

Contents

I	Security Model	2
1	Blockchains as Adversarial State Machines	3
1.1	Purpose	3
1.2	Learning Objectives	4
1.3	The Core Model	4
1.4	State, State Transitions, and Validity Rules	6
1.4.1	State Is Everything the Rules Need to Remember	6
1.4.2	Transitions Are the Only Way State Changes	6
1.4.3	Validity Is Not the Same as Safety	8
1.4.4	Validity Rules Define Reachable States	8
1.4.5	Invariants Are Properties That Must Survive Every Transition	9
1.4.6	State Includes the Rules That Can Change the Rules	9
1.5	Determinism, Replication, and Global Agreement	10
1.5.1	Replication Requires Determinism	10
1.5.2	Consensus Orders Inputs; Execution Applies Them	11
1.5.3	Blocks Are Commitment Devices	11
1.5.4	Global Agreement Is Not Instant Agreement	12
1.5.5	Determinism Does Not Eliminate Implementation Risk	13
1.6	Permissionless Inputs and Adversarial Execution	14
1.6.1	Permissionless Means Attackers Are Normal Users	14
1.6.2	Attackers Choose Edge Cases, Not Happy Paths	14
1.6.3	Public State Changes the Defender's Burden	15
1.6.4	The Adversary Controls More Than Transaction Parameters	15
1.6.5	Rejection Is a Security Boundary	16

1.6.6	Adversarial Execution Includes Honest-Looking Transactions	17
1.7	Safety, Liveness, and Fault Assumptions	17
1.7.1	Safety: Bad States Must Not Happen	17
1.7.2	Liveness: Necessary Progress Must Remain Possible	18
1.7.3	Safety and Liveness Pull Against Each Other	18
1.7.4	Fault Assumptions Are Part of the Security Model	19
1.7.5	Byzantine, Rational, and Accidental Faults Are Different	20
1.7.6	Time and Finality Are Security Parameters	20
1.8	Security Failures as Broken State Transitions	21
1.8.1	The Most Important Distinction	21
1.8.2	A Taxonomy of Broken Transitions	21
1.8.3	Valid Exploits Are Still Exploits	22
1.8.4	The Chain Protects Its Own Rules, Not Your Intent	23
1.8.5	Reorgs and Ordering Are State-Transition Problems	24
1.8.6	Security Review Starts With Transition Questions	24
1.9	Worked Example: A Minimal Escrow State Machine	25
1.10	Practical Discipline: Do Not Start With Bug Names	27
1.11	Review Questions	28
1.12	Applied Exercises	29
1.12.1	Exercise 1: Model a Token Transfer	29
1.12.2	Exercise 2: Separate Rule Validity From Intent	29
1.12.3	Exercise 3: Identify Liveness Risk	29
1.12.4	Exercise 4: Broken Transition Taxonomy	30
1.13	Key Takeaways	30
1.14	Further Reading	31
2	Assets, Authority, and Trust Boundaries	32
2.1	Purpose	32
2.2	Learning Objectives	33
2.3	The Core Distinction	33
2.4	What Counts as an Asset On-Chain	34
2.4.1	Assets Are Enforced Claims Over State	34
2.4.2	Asset Categories	34

2.4.3	Native Assets and Contract Assets Differ	36
2.4.4	An Asset Can Be a Right, Not a Balance	36
2.4.5	Off-Chain Claims Are Not Made On-Chain by Naming Them	37
2.4.6	Control Surfaces Are Assets	38
2.5	Ownership: Keys, Accounts, UTXOs, Objects, and Claims	39
2.5.1	Ownership Is a State-Machine Relationship	39
2.5.2	UTXO Ownership: Spend Conditions, Not Account Balances	39
2.5.3	Account Ownership: Keys, Code, and Nonces	40
2.5.4	Contract Ownership Is Just State	40
2.5.5	Token Ownership Is Defined by the Token's Rules	41
2.5.6	Solana Ownership: Account Owner Program and Signers	42
2.5.7	Move Ownership: Resources and Module Boundaries	43
2.5.8	Legal Ownership and Protocol Control Can Diverge	43
2.6	Authority: Users, Admins, Validators, Governance, and Custodians	44
2.6.1	Authority Is the Ability to Cause a Transition	44
2.6.2	User Authority	45
2.6.3	Admin Authority	45
2.6.4	Validator, Miner, Builder, and Sequencer Authority	46
2.6.5	Governance Authority	47
2.6.6	Custodial and Operational Authority	48
2.6.7	Authority Can Be Delegated, Composed, or Hidden	49
2.7	Trust Boundaries Across On-Chain, Off-Chain, and Human Systems	50
2.7.1	A Trust Boundary Marks a Change in Verification	50
2.7.2	On-Chain Does Not Mean Trustless	51
2.7.3	Off-Chain Does Not Mean Insecure	51
2.7.4	Cross-Chain Systems Create Stacked Trust Boundaries	52
2.7.5	Human Systems Are Part of the Boundary	53
2.7.6	Trust Boundaries Should Be Written as Assumptions	53
2.8	Mapping Who Can Move, Freeze, Mint, Burn, Upgrade, or Censor Assets	54
2.8.1	Authority Mapping Is the Practical Output	54
2.8.2	The Core Authority Verbs	55
2.8.3	Authority Matrix	56

2.8.4	Example: ERC-20 Token Authority Map	57
2.8.5	Example: Vault Authority Map	58
2.8.6	Example: Bridge Authority Map	59
2.8.7	Example: Governance Authority Map	60
2.8.8	Minimal Authority Mapping Template	60
2.9	Worked Example: Mapping a Tokenized Vault	61
2.10	Practical Discipline: Replace Nouns With Verbs	64
2.11	Review Questions	64
2.12	Applied Exercises	65
2.12.1	Exercise 1: Authority Map a Token	65
2.12.2	Exercise 2: Trust Boundary and Custody Review	65
2.12.3	Exercise 3: Bridge Claim Analysis	66
2.12.4	Exercise 4: Replace Nouns With Verbs	66
2.13	Key Takeaways	67
2.14	Further Reading	67
3	Transactions and State Transitions	68
3.1	Purpose	68
3.2	Learning Objectives	68
3.3	The Core Model	69
3.4	Transaction Anatomy Across Chain Models	70
3.4.1	Transaction Anatomy Is a Security Interface	70
3.4.2	UTXO Transactions: Consume Outputs, Create Outputs	71
3.4.3	Account Transactions: Mutate Account and Contract State	72
3.4.4	Instruction-Based Transactions: Multiple Operations, Shared Atomicity	73
3.4.5	Object-Based Transactions: Inputs, Versions, and Effects	74
3.4.6	Cosmos-Style Transactions: Messages Through an Application Pipeline	75
3.4.7	Cross-Model Comparison	76
3.5	Signatures, Nonces, Sequence Numbers, and Replay Prevention	77
3.5.1	Signatures Authorize a Specific Message, Not a Vague Intention	77
3.5.2	Replay Is Reuse of Authority in the Wrong Context	78
3.5.3	Ethereum Nonces Serialize Account Transactions	78
3.5.4	Chain IDs and Domain Separation Prevent Cross-Chain Replay	79

3.5.5	UTXO Replay Prevention Comes From Consumption	80
3.5.6	Solana Recent Blockhashes and Durable Nonces	80
3.5.7	Cosmos Sequence Numbers and Timeout Heights	81
3.5.8	Object Versions and Expiration	81
3.5.9	Replay Protection Must Match the Asset	82
3.6	Fees, Gas, Compute Budgets, and Resource Limits	82
3.6.1	Resource Pricing Is Part of Consensus Security	82
3.6.2	Ethereum Gas	83
3.6.3	Solana Compute Budgets and Fees	83
3.6.4	Cosmos Gas and Local Mempool Fee Policy	84
3.6.5	UTXO Fees and Transaction Pinning	85
3.6.6	Resource Limits Create Liveness Risks	85
3.6.7	Fees Shape Adversarial Strategy	86
3.7	Execution Results: Receipts, Logs, Events, and Reverts	86
3.7.1	Inclusion Is Not Execution Success	86
3.7.2	Receipts Summarize Execution Outcomes	88
3.7.3	Logs and Events Are Communication, Not Authority	88
3.7.4	Return Data and Revert Data Matter	90
3.7.5	Atomicity Is Model-Specific	90
3.7.6	Simulation Is Not Execution	91
3.8	When a Valid Transaction Still Produces an Unsafe State	92
3.8.1	Validity Is Layered	92
3.8.2	Successful Execution Can Encode a Failed Assumption	92
3.8.3	Unsafe State From Correct Ordering Rules	93
3.8.4	Unsafe State From Partial Mental Models	94
3.8.5	Unsafe State From Stale Context	94
3.8.6	Unsafe State From Successful Failure	95
3.8.7	Transaction Safety Checklist	96
3.9	Worked Example: A Swap Transaction	97
3.10	Practical Discipline: Transaction Review Notes	99
3.11	Review Questions	100
3.12	Applied Exercises	100

3.12.1	Exercise 1: Transaction Anatomy and Evidence	100
3.12.2	Exercise 2: Replay Scope	101
3.12.3	Exercise 3: Event vs State	101
3.12.4	Exercise 4: Resource Limits and Valid-But-Unsafe Outcomes	101
3.13	Key Takeaways	102
3.14	Further Reading	102
4	Keys, Signatures, and Authorization	104
4.1	Purpose	104
4.2	Learning Objectives	105
4.3	The Core Model	105
4.4	Private Keys, Public Keys, and Address Derivation	107
4.4.1	Private Keys Are Authority Material	107
4.4.2	Public Keys Verify; Addresses Identify	108
4.4.3	Key Generation Depends on Entropy	108
4.4.4	Seed Phrases Are Root Authority	109
4.4.5	Hierarchical Deterministic Wallets Create Authority Trees	110
4.4.6	Wallets Are Interfaces to Authority	111
4.4.7	Key Rotation and Revocation Are Hard	111
4.5	ECDSA, EdDSA, BLS, Schnorr, and Signature Assumptions	112
4.5.1	Digital Signatures Provide Authenticity and Integrity	112
4.5.2	ECDSA	113
4.5.3	ECDSA Malleability and Low-s Rules	113
4.5.4	EdDSA	114
4.5.5	BLS	115
4.5.6	Schnorr Signatures	116
4.5.7	Signature Assumptions Are Broader Than Algorithms	117
4.6	Message Signing, Transaction Signing, and Domain Separation	117
4.6.1	Transaction Signing Authorizes a Chain-Level Request	117
4.6.2	Message Signing Authorizes Off-Chain or Contract-Interpreted Meaning	118
4.6.3	EIP-191 and the Problem of Raw Bytes	118
4.6.4	EIP-712 Typed Data	119
4.6.5	Permits and Signed Approvals	120

4.6.6	Contract Signatures and ERC-1271	121
4.6.7	Domain Separation Checklist	121
4.7	Multisig, Threshold Signatures, MPC, and Hardware Signing	122
4.7.1	Single-Key Control Is a Concentrated Failure Mode	122
4.7.2	On-Chain Multisig	123
4.7.3	Native Multisig and Script Policies	124
4.7.4	Threshold Signatures	124
4.7.5	MPC Signing	125
4.7.6	Hardware Wallets and HSMs	126
4.7.7	Choosing the Right Signing Architecture	127
4.8	Authorization Failures: Replay, Malleability, Confused Signing, and Key Compromise	128
4.8.1	Signature Validity Is Not Authorization Completeness	128
4.8.2	Replay Failures	128
4.8.3	Malleability Failures	129
4.8.4	Confused Signing	129
4.8.5	Wrong Signer and Wrong Authority	130
4.8.6	Stale Authority	131
4.8.7	Key Compromise	131
4.8.8	Authorization Review Checklist	132
4.9	Worked Example: Permit as Delegated Authority	133
4.10	Practical Discipline: Translate Signatures Into Authority	134
4.11	Review Questions	135
4.12	Applied Exercises	135
4.12.1	Exercise 1: Signed-Payload Anatomy	135
4.12.2	Exercise 2: Replay and Domain-Separation Review	136
4.12.3	Exercise 3: Compare Signing Arrangements	136
4.12.4	Exercise 4: Authorization Finding Draft	136
4.13	Key Takeaways	137
4.14	Further Reading	137
5	Cryptographic Commitments and Proofs	139
5.1	Purpose	139
5.2	Learning Objectives	140

5.3	The Core Model	140
5.4	Hash Functions, Preimages, and Collision Resistance	142
5.4.1	A Cryptographic Hash Is Not Just a Checksum	142
5.4.2	Preimage Resistance	143
5.4.3	Second-Preimage Resistance	143
5.4.4	Collision Resistance	144
5.4.5	Hash Security Depends on the Application	145
5.4.6	Hashes Are Identifiers, Not Authority	145
5.4.7	Hash Functions Do Not Fix Bad Encoding	146
5.5	Commitments, Binding Assumptions, and Domain Separation	147
5.5.1	A Commitment Binds a Claim to Data	147
5.5.2	Binding Depends on the Whole Construction	147
5.5.3	Domain Separation Prevents Context Confusion	149
5.5.4	Canonical Encoding Is Part of the Commitment	149
5.5.5	Hiding Requires Entropy	150
5.5.6	Commitments Can Become Stale	150
5.6	Merkle Trees, Patricia Tries, State Roots, and Inclusion Proofs	151
5.6.1	Merkle Trees Compress Many Claims Into One Root	151
5.6.2	Inclusion Proofs Prove Membership Relative to a Root	152
5.6.3	Tree Shape and Leaf Meaning Matter	153
5.6.4	Patricia Tries Commit to Key/Value State	154
5.6.5	State Proofs and Storage Proofs	155
5.6.6	Receipt Proofs and Event Proofs	156
5.6.7	SSZ Merkleization and Consensus Objects	156
5.7	Proof Verification, Proof Forgery Risk, and Trust-Minimized Claims	157
5.7.1	A Proof Is Only as Good as Its Root	157
5.7.2	Proof Verification Is a State Transition	158
5.7.3	Common Proof Verification Failures	158
5.7.4	Proof of Inclusion Is Not Proof of Validity	159
5.7.5	Trust-Minimized Does Not Mean Trust-Free	160
5.7.6	Proofs Need Freshness and Expiration	160
5.7.7	Proof Verification Can Be a Denial-of-Service Surface	161

5.7.8	The Verification Checklist	161
5.8	Worked Example: Verifying a Cross-Chain Deposit Claim	162
5.9	Practical Discipline: Root, Proof, Claim	164
5.10	Review Questions	165
5.11	Applied Exercises	165
5.11.1	Exercise 1: Manual Merkle Verification	165
5.11.2	Exercise 2: Root and Domain Review	165
5.11.3	Exercise 3: Proof-Verification Pseudocode	165
5.11.4	Exercise 4: Trust-Minimized Claim Statement	166
5.12	Key Takeaways	166
5.13	Further Reading	167
6	Threat Modeling Blockchain Systems	168
6.1	Purpose	168
6.2	Learning Objectives	169
6.3	The Core Model	170
6.4	Turning a Protocol Description into a Security Model	171
6.4.1	Start With Scope	171
6.4.2	Collect the Actual System, Not the Marketing System	172
6.4.3	Extract Security Claims	173
6.4.4	Translate Features Into State Transitions	174
6.4.5	Build a System Context Diagram	175
6.4.6	Threat Modeling Is Iterative	176
6.5	Mapping Actors, Assets, Entry Points, and Privileged Roles	177
6.5.1	Actor Inventory	177
6.5.2	Asset Inventory	179
6.5.3	Entry Point Inventory	180
6.5.4	Privileged Role Inventory	181
6.5.5	Data and Control Flows	183
6.5.6	The Minimum Map	184
6.6	Separating Users, Attackers, Rational Participants, and Trusted Operators	184
6.6.1	Do Not Use One Generic Attacker	184
6.6.2	Users Can Be Adversarial Without Being “Hackers”	185

6.6.3	Rational Participants Are Not Byzantine Actors	186
6.6.4	Trusted Operators Are Threat Sources Too	186
6.6.5	Insiders Have Different Access	187
6.6.6	Actor Table Template	188
6.7	Identifying Trust Assumptions and External Dependencies	189
6.7.1	Assumptions Are Security Debt	189
6.7.2	Dependency Inventory	189
6.7.3	Safety vs Liveness Assumptions	190
6.7.4	External Contracts Are Dependencies, Not Background	191
6.7.5	Human Dependencies Are Real Dependencies	192
6.7.6	Assumption Register	193
6.8	Mapping Attack Surfaces Across Chain, Contract, Infrastructure, and Human Layers	194
6.8.1	Blockchain Attack Surface Is Layered	194
6.8.2	STRIDE as a Prompt, Not a Cage	194
6.8.3	Contract and Program Attack Surface	195
6.8.4	Chain and Ordering Attack Surface	196
6.8.5	Economic Attack Surface	197
6.8.6	Infrastructure Attack Surface	197
6.8.7	Human and Governance Attack Surface	198
6.9	Producing a Threat Model That Guides Review Instead of Decorating It	199
6.9.1	The Threat Model Must Produce Work	199
6.9.2	Prioritize by Impact, Feasibility, and Uncertainty	200
6.9.3	Turn Threats Into Invariants and Tests	201
6.9.4	Decide Mitigation, Acceptance, Transfer, or Elimination	202
6.9.5	Threat Model Deliverable Template	203
6.9.6	Anti-Patterns	204
6.9.7	What Good Looks Like	205
6.10	Worked Example: Threat Modeling a Lending Market	205
6.10.1	Step 1: Scope	206
6.10.2	Step 2: Assets	206
6.10.3	Step 3: Actors	206
6.10.4	Step 4: Entry Points	207

6.10.5	Step 5: Assumptions	207
6.10.6	Step 6: Threat Scenarios	208
6.10.7	Step 7: Review Plan	209
6.11	Practical Discipline: The One-Page Threat Model	210
6.12	Review Questions	210
6.13	Applied Exercises	211
6.13.1	Exercise 1: One-Page Threat Model	211
6.13.2	Exercise 2: Data and Control Flow	211
6.13.3	Exercise 3: Attack Tree to Review Plan	211
6.13.4	Exercise 4: Threat Model Critique	212
6.14	Key Takeaways	212
6.15	Further Reading	212
7	Invariants and Exploitability	214
7.1	Purpose	214
7.2	Learning Objectives	215
7.3	The Core Model	215
7.4	What an Invariant Is and Why It Finds Deep Bugs	217
7.4.1	Invariants Are Properties, Not Wishes	217
7.4.2	Reachability Is the Point	217
7.4.3	Inductive Invariants vs Intended Invariants	218
7.4.4	Invariants Can Be About States, Transitions, or Histories	219
7.4.5	A Broken Invariant Is an Attack Hypothesis	219
7.4.6	Tools Check Properties; People Must Choose Them	220
7.4.7	The Invariant Quality Test	221
7.5	Conservation, Supply, and Asset-Balance Invariants	222
7.5.1	Conservation Is the First Accounting Question	222
7.5.2	Exact Conservation vs Bounded Drift	222
7.5.3	Total Supply Is Not Enough	223
7.5.4	Direct Balance Changes Are a Security Surface	223
7.5.5	Bridges Need Cross-Domain Conservation	224
7.5.6	Conservation Invariants Should Name Units	225
7.5.7	Common Conservation Failure Modes	225

7.6	Authorization, Ownership, and Capability Invariants	226
7.6.1	Authorization Is a State Property	226
7.6.2	Ownership Must Be Bound to the Right Object	227
7.6.3	Capabilities Are Assets	227
7.6.4	Negative Authorization Matters	228
7.6.5	Cross-Contract Authorization Is Where Bugs Hide	229
7.6.6	Authorization Invariants Should Include Time	229
7.7	Solvency, Collateralization, Reserve, and Share-Price Invariants	230
7.7.1	Solvency Is About Claims vs Recoverable Assets	230
7.7.2	Lending Markets Need Collateralization Invariants	231
7.7.3	Lending Invariants Depend on Oracles	232
7.7.4	Reserve Invariants Separate User Claims from Protocol Equity	232
7.7.5	Share-Price Invariants Are Subtle	233
7.7.6	Donation and Inflation Attacks Are Broken Share Invariants	233
7.7.7	AMM Invariants Are Pricing Rules, Not Magic Shields	234
7.7.8	Solvency Invariants Must Name Failure Modes	235
7.8	Temporal, Liveness, Exit, Withdrawal, and Recovery Invariants	236
7.8.1	Not Every Security Failure Is Theft	236
7.8.2	Temporal Properties Need Clocks and Conditions	237
7.8.3	Exit Invariants Are Security Boundaries	237
7.8.4	Withdrawal Queues Need Ordering Invariants	238
7.8.5	Pause and Recovery Controls Need Invariants Too	239
7.8.6	Liveness Invariants Are Hard to Test	239
7.9	Exploitability: Capital, Timing, Liquidity, Reliability, and Profit	240
7.9.1	Vulnerability Is Not the Same as Exploit	240
7.9.2	Exploitability Has Preconditions	240
7.9.3	Capital Requirement	241
7.9.4	Timing and Ordering	242
7.9.5	Liquidity and Market Depth	242
7.9.6	Reliability and Repeatability	243
7.9.7	Profit Is Not the Only Impact	244
7.9.8	“Not Exploitable” Must Be Proven Narrowly	244

7.9.9	Exploitability Worksheet	245
7.10	Worked Example: From Protocol Intent to Attack Hypotheses	246
7.10.1	Protocol Description	246
7.10.2	Step 1: Asset and Share Conservation	246
7.10.3	Step 2: Deposit Fairness Invariant	246
7.10.4	Step 3: Empty-Vault Attack Hypothesis	247
7.10.5	Step 4: Exploitability Evaluation	248
7.10.6	Step 5: Mitigation Invariants	249
7.11	Practical Discipline: Building an Invariant Catalogue	249
7.12	Review Questions	251
7.13	Applied Exercises	252
7.13.1	Exercise 1: Write Invariants from Intent	252
7.13.2	Exercise 2: Find the Missing Assumption	252
7.13.3	Exercise 3: Exploitability Worksheet	252
7.13.4	Exercise 4: Counterexample to Finding	253
7.14	Key Takeaways	253
7.15	Part I Handoff	254
7.16	Further Reading	254

Part I Integrated Lab: Security Model Review

256

Part I

Security Model

Chapter 1

Blockchains as Adversarial State Machines

1.1 Purpose

This chapter establishes the central mental model for the book:

A blockchain is a replicated state machine whose inputs are chosen in an adversarial environment.

That sentence is more useful than most slogans about decentralization, immutability, or trustlessness. It gives us a way to reason about security without getting trapped inside the vocabulary of any single chain. Bitcoin, Ethereum, Solana, Move-based systems, appchains, bridges, rollups, and smart contracts all differ in execution model and consensus mechanism. But they all have to answer the same first-principles questions:

- What is the state?
- What inputs can change it?
- Which transitions are valid?
- Who gets to submit, order, censor, or finalize those inputs?
- Which properties must remain true across every reachable state?
- What assumptions make replication, progress, and agreement possible?

This chapter is deliberately not a tour of blockchain buzzwords. We will not yet study signatures in depth, consensus protocols, token standards, reentrancy, MEV, or bridges. Those topics come later. Here we build the frame that makes those topics teachable.

If you learn one habit from this chapter, make it this:

Never review a blockchain system as a collection of features. Review it as a set of possible state transitions under adversarial input and ordering.

1.2 Learning Objectives

By the end of this chapter, you should be able to:

- Explain a blockchain as a state machine rather than as a database or a payment network.
- Distinguish state, transaction, block, transition function, validity rule, and invariant.
- Explain why deterministic execution is necessary for replicated validation.
- Identify where adversarial inputs enter a blockchain system.
- Separate safety failures from liveness failures.
- Explain why many serious exploits are not “invalid transactions” but valid transitions that violate protocol intent.

1.3 The Core Model

A state machine has four basic ingredients:

- A set of possible states.
- A set of possible inputs.
- A transition function that maps a current state and an input to a next state.
- Rules for rejecting inputs or transitions that are not allowed.

In abstract form:

```
next_state = apply(current_state, input)
```

For blockchains, the important inputs are usually transactions, messages, blocks, votes, proofs, signatures, and governance actions. A block is a batch of ordered inputs. A chain is a history of such batches. A node validates the chain by starting from an initial state and applying the accepted inputs in the accepted order.

Ethereum’s formal specification explicitly frames Ethereum as a transaction-based state machine: a genesis state is transformed into a current state by executing transactions.¹ Ethereum.org’s developer documentation makes the same operational point in simpler terms: transactions are signed instructions from accounts that update network state, and blocks batch transactions so participants can synchronize around a common history.^{2 3}

Bitcoin is often described as a ledger rather than a state machine, but the same model applies. The state is the set of unspent transaction outputs, along with consensus-relevant chain data. A transaction consumes existing outputs and creates new outputs. A block proposes an ordered set of

¹Gavin Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger”, <https://ethereum.github.io/yellowpaper/paper.pdf>.

²ethereum.org, “Transactions”, <https://ethereum.org/developers/docs/transactions/>.

³ethereum.org, “Blocks”, <https://ethereum.org/developers/docs/blocks/>.

such transactions. Nodes accept a block only if its transactions satisfy the validity rules, including the rule that the same output is not spent twice.⁴

This generalizes:

```
genesis_state
  --tx_1--> state_1
  --tx_2--> state_2
  --tx_3--> state_3
  ...
```

For review work, draw the machine before naming bug classes:

```
flowchart LR
  S[Current state] --> I[Input]
  I --> V{Validity check}
  V -- reject --> R[No state change]
  V -- accept --> T[Transition function]
  T --> N[Next state]
  N --> P{Invariant check}
  P -- holds --> A[Accept transition]
  P -- broken --> B[Unsafe reachable state]
```

The same idea can be written as minimal review pseudocode:

```
apply(state, input):
  check syntax and encoding
  check authority
  check validity in current state
  compute next_state
  check required invariants
  return next_state or reject
```

For blocks:

```
state_after_block = apply_block(parent_state, [tx_0, tx_1, tx_2, ...])
```

And for a chain:

```
current_state = replay(genesis_state, canonical_history)
```

The security question is not merely whether a transaction is formatted correctly. The question is whether the system's rules ensure that every reachable state is acceptable under the intended security model.

That distinction will carry the whole book.

⁴Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", <https://bitcoin.org/bitcoin.pdf>.

1.4 State, State Transitions, and Validity Rules

1.4.1 State Is Everything the Rules Need to Remember

State is the information that matters for future validity and behavior.

In a blockchain system, state may include:

- Account balances.
- UTXO sets.
- Contract storage.
- Ownership records.
- Nonces and sequence numbers.
- Validator sets and stakes.
- Governance configuration.
- Pending withdrawals.
- Oracle prices cached on-chain.
- Token supplies.
- Vault share balances.
- Bridge message statuses.
- Rollup output roots.
- Protocol parameters such as fee rates, limits, and emergency flags.

Do not reduce state to “balances.” Balances are only one kind of state. A paused flag is state. An upgrade admin is state. A stored oracle timestamp is state. A consumed bridge message nonce is state. A validator’s slashing status is state. A record that a withdrawal has already been claimed is state.

For security analysis, state is whatever the attacker wants to change, depend on, corrupt, desynchronize, or freeze.

1.4.2 Transitions Are the Only Way State Changes

A state transition is a change from one state to another. In blockchain systems, transitions are usually caused by accepted inputs:

- A user signs and submits a transaction.
- A contract calls another contract.
- A validator proposes a block.
- A governance proposal executes.
- A bridge relayer submits a message.
- A sequencer posts a rollup batch.

- A withdrawal proof is verified.
- An oracle update is accepted.
- A protocol upgrade changes the transition rules themselves.

The transition is not the transaction object. The transaction is the input. The transition is the state change produced by applying the input under the rules.

Consider a tiny token ledger:

Element	Token ledger model
State	<code>balances[address], total_supply</code>
Input	<code>transfer(sender, recipient, amount, signature)</code>
Authority	<code>signature</code> must authorize <code>sender</code>
Validity	<code>amount > 0</code> and <code>balances[sender] >= amount</code>
Transition	Decrease <code>sender</code> , increase <code>recipient</code> , leave <code>total_supply</code> unchanged
Invariant	Sum of balances remains equal to <code>total_supply</code>

The transfer transition table is:

Step	Check or update	Security purpose
1	Decode <code>sender, recipient, amount, signature</code>	Reject malformed inputs before state changes
2	Verify the signature authorizes <code>sender</code>	Prevent unauthorized movement
3	Check <code>amount > 0</code>	Reject nonsensical or ambiguous movement
4	Check <code>balances[sender] >= amount</code>	Prevent negative balances
5	Subtract from <code>sender</code> and add to <code>recipient</code>	Move value without changing total supply
6	Recheck conservation	Catch implementation or integration errors

The transition is acceptable only if the validity rules hold. If `balances[sender]` is less than `amount`, the input must not produce the transfer. If the signature does not authorize the sender, the input must not move funds. If the transition decreases the sender but forgets to increase the recipient, the transition violates asset conservation even if the transaction looked well formed.

Security starts with this discipline:

For every accepted input, what state changes?
For every rejected input, why is it rejected?
For every state change, what must remain true?

1.4.3 Validity Is Not the Same as Safety

The word “valid” is dangerous because it is often used at different layers.

A transaction may be:

- Syntactically valid: it is encoded correctly.
- Authenticated: the signature or authorization data checks out.
- Executable: the current state has enough resources for execution.
- Protocol-valid: it satisfies the base chain’s validity rules.
- Application-valid: it satisfies a smart contract or program’s explicit checks.
- Intent-safe: it preserves the economic or security properties the protocol was supposed to protect.

The last category is where many expensive failures live.

A lending protocol may accept a transaction that is syntactically valid, signed by the attacker, included in a valid block, and allowed by the contract code. Yet the same transaction may still leave the protocol insolvent because the oracle price was manipulable or the collateral accounting rounded in the attacker’s favor. The chain did not fail to validate the transaction. The application failed to encode the right validity constraints or invariants.

This is why “the transaction was valid” is not a defense of a protocol. It is often the description of the exploit.

1.4.4 Validity Rules Define Reachable States

The set of reachable states is the set of states the system can enter by starting from the initial state and applying accepted transitions.

Informally:

```
genesis_state is reachable

if state is reachable
and input is valid in state
then apply(state, input) is reachable
```

A security property must hold over reachable states, not imagined states. If a state cannot occur under any accepted transition, it may be irrelevant. If a dangerous state can occur through any valid sequence of inputs, it matters even if the sequence looks unusual.

Attackers specialize in unusual valid sequences.

They will not use your protocol according to the product story. They will use it according to the transition rules.

1.4.5 Invariants Are Properties That Must Survive Every Transition

An invariant is a property that should remain true across all reachable states.

Examples:

- Total token balances equal total supply.
- No account balance is negative.
- A withdrawal cannot be claimed twice.
- A user cannot spend funds without authorization.
- A vault's assets are sufficient to redeem outstanding shares according to the redemption rules.
- A bridge message cannot be executed on the destination chain unless it was committed on the source chain under the required finality assumptions.
- A governance executor cannot bypass the configured delay unless emergency powers explicitly allow it.

The invariant mindset comes from state-machine reasoning. Lamport's treatment of state machines emphasizes invariants as predicates that hold in all reachable states, and proves them by showing they hold initially and are preserved by transitions.⁵ This is not academic ornamentation. It is exactly the reasoning security reviewers need.

For a blockchain protocol, the practical version is:

If the invariant is true before this transaction,
is it still true after this transaction?

If the answer is no, you have either found a bug or discovered that your invariant is not actually part of the intended design.

Both outcomes are valuable.

1.4.6 State Includes the Rules That Can Change the Rules

Many blockchain systems have upgrade mechanisms, governance modules, feature flags, admin keys, pausing controls, validator-set updates, and parameter changes. These are not external to the state machine. They are transitions that alter future transition rules or future authority.

For example:

State:

`implementation_address`

⁵Leslie Lamport, "Computation and State Machines", <https://lamport.azurewebsites.net/pubs/state-machine.pdf>.

```

admin
timelock_delay
paused

```

Input:

```

upgrade_to(new_implementation)

```

Validity:

```

caller == admin
timelock elapsed
new implementation passes required checks

```

Transition:

```

implementation_address = new_implementation

```

An upgrade does not merely change a value. It changes the code that determines future state transitions. That makes upgrade authority one of the most important security boundaries in any system that has it.

Do not treat governance and upgrades as operational details. They are part of the state machine.

1.5 Determinism, Replication, and Global Agreement

1.5.1 Replication Requires Determinism

A blockchain is replicated. Many nodes maintain or verify the same system. They may be run by different people, on different machines, in different countries, using different client implementations.

Replication only works if honest validators can independently reach the same result.

If two honest nodes start from the same state and apply the same valid input sequence, they must compute the same next state. Otherwise the network does not have one replicated state machine. It has a collection of machines drifting apart.

This requirement can be stated simply:

```

same initial state
+ same ordered inputs
+ same transition rules
= same resulting state

```

Determinism is why blockchain execution environments restrict or carefully define sources of environmental data. A contract cannot ask an arbitrary web server for the current weather during consensus execution. A validator cannot use its local machine clock as an unbounded source of truth. A program cannot depend on uncommitted local randomness and still expect all validators to agree.

When a blockchain exposes “environmental” values such as block number, timestamp, slot, block hash, base fee, recent blockhash, or chain ID, those values are not private local facts. They are consensus-visible inputs or protocol-defined values. They are part of the transition context.

This has a security consequence:

If a value is available to on-chain execution, assume it is public, constrained by protocol rules, and potentially influenced by actors who participate in block production.

That does not mean every such value is useless. It means the security assumption must be explicit.

1.5.2 Consensus Orders Inputs; Execution Applies Them

For teaching purposes, separate two jobs:

- Consensus, fork choice, or block production determines which ordered history is canonical.
- Execution applies the transition rules to that ordered history.

Different chains combine these jobs differently, but the distinction matters.

Bitcoin’s proof-of-work chain selection gives nodes a way to converge on a history of valid blocks under a resource assumption. Ethereum’s modern proof-of-stake protocol uses validators, attestations, fork choice, and finality checkpoints. BFT-style appchains may use voting rounds and finality certificates. Rollups may use sequencers, L1 contracts, fraud proofs, validity proofs, or data availability assumptions.

Those mechanisms differ. The state-machine question remains:

Which ordered inputs are accepted as canonical?

What state results from applying them?

When can users treat that result as settled?

Chapter 8 will study consensus and finality in detail. For now, do not confuse “valid transition” with “canonical transition.” A transaction can be locally valid and still not land in the canonical chain. A block can contain valid transactions and still lose a fork-choice race. A withdrawal can be valid on an L2 but not yet settled on L1.

Security depends on both execution validity and history selection.

1.5.3 Blocks Are Commitment Devices

A block is not merely a container. It commits to an ordered set of data and to a relationship with prior history.

Ethereum documentation describes blocks as batches of transactions linked by hashes of previous blocks, so that changing prior data affects later commitments.⁶ The Ethereum Yellow Paper makes a more formal point: blocks record transactions, reference the prior block, and include an identifier of the resulting state rather than storing the entire state itself.⁷

This matters for security because nodes do not need to trust a block producer’s claim about the final state. They can execute the block and verify that the resulting state commitment is correct.

At a high level:

proposed block:

```
parent reference
ordered transactions
state commitment
other consensus data
```

validator checks:

```
parent is acceptable
transactions are valid in order
execution result matches commitment
consensus rules are satisfied
```

If the block producer lies about the state result, honest validating nodes should reject the block. If clients disagree about the transition rules, a chain split can occur. If the commitment scheme has a flaw, proofs and light-client assumptions can fail. Those are different failure modes, but all are visible in the state-machine frame.

1.5.4 Global Agreement Is Not Instant Agreement

“The blockchain state” sounds singular. In reality, nodes observe the network through time, latency, forks, local mempools, peer choices, RPC providers, and client software. Agreement is a property achieved under assumptions, not magic.

At a given moment:

- Two nodes may have seen different pending transactions.
- Two nodes may have different mempool contents.
- Two nodes may temporarily prefer different chain heads.
- A block that appears confirmed may later be reorged in probabilistic-finality systems.
- A rollup transaction may be accepted by a sequencer before it is settled on the base layer.
- A user interface may show state from an indexer that is behind, wrong, or pointed at a minority fork.

⁶ethereum.org, “Blocks”, <https://ethereum.org/developers/docs/blocks/>.

⁷Gavin Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger”, <https://ethereum.github.io/yellowpaper/paper.pdf>.

Security reviews must be precise about which level of agreement is being assumed:

- Local execution result.
- Mempool observation.
- Included in a block.
- Confirmed after some depth.
- Economically finalized.
- BFT-finalized.
- Settled on another domain.
- Reflected correctly by an RPC, indexer, or frontend.

Many application bugs come from treating one level as if it were another.

Example: a bridge that releases funds on chain B after seeing an event on chain A must define how final that event must be. Seeing a transaction in a block is not the same as accepting that the event can never be removed from the canonical history. If the bridge treats tentative state as final state, the bridge has imported the source chain's reorg risk into its own asset accounting.

1.5.5 Determinism Does Not Eliminate Implementation Risk

A protocol can be deterministic on paper and still fail in implementation.

Possible causes include:

- Client bugs.
- Parser differences.
- Integer overflow or underflow in a client implementation.
- Inconsistent handling of malformed data.
- Different interpretations of serialization rules.
- Non-deterministic iteration over maps or sets.
- Platform-specific behavior.
- Incorrect gas or resource accounting.
- Database corruption or state-pruning errors.

This is why blockchain security is not only smart contract security. The replicated state machine depends on correct clients, correct specifications, compatible implementations, and robust handling of adversarial data.

If a malformed block causes one client to accept and another to reject, the failure is not at the application layer. The replicated machine has split on the meaning of its transition rules.

1.6 Permissionless Inputs and Adversarial Execution

1.6.1 Permissionless Means Attackers Are Normal Users

In many traditional systems, a backend receives requests from authenticated users through controlled interfaces. Attackers may still send malicious requests, but the system often has additional chokepoints: web servers, rate limits, IP-based controls, account review, fraud teams, manual rollback, or privileged database access.

Public blockchains change the default.

If a function, instruction, transaction type, or message path is public, an attacker can usually call it. If a contract can be composed with, an attacker can compose with it. If a state can be read, an attacker can simulate against it. If a transaction can be frontrun, copied, sandwiched, delayed, or backrun, someone may try.

The attacker is not outside the system. The attacker is a participant using the system's own rules.

This is the permissionless input model:

Any allowed actor may submit any syntactically valid input,
at any allowed time,
with any allowed parameters,
in any allowed sequence,
to any public entry point,
as long as they can pay the required costs.

That is the baseline. Then you add the chain-specific details: mempools, fees, account models, signer semantics, compute limits, validator behavior, sequencer rules, relayers, governance, and off-chain infrastructure.

1.6.2 Attackers Choose Edge Cases, Not Happy Paths

A user story says:

Alice deposits collateral, borrows conservatively, repays later, and withdraws.

An attacker asks:

Can I deposit and withdraw in the same transaction?
Can I manipulate the price before borrowing?
Can I enter with zero shares?
Can I donate assets directly to skew accounting?
Can I call through a contract that reenters?
Can I create many accounts?
Can I force rounding to favor me?
Can I make another user's operation fail?
Can I trigger a state where exits are impossible?

Can I exploit a stale cache?
Can I submit the same message on another chain?
Can I reorder operations so the invariant fails?

The difference is not attitude. It is method.

Security review is adversarial state exploration. It asks what the transition system permits, not what the designers expected honest users to do.

1.6.3 Public State Changes the Defender's Burden

On-chain systems often expose state, code, events, pending transactions, liquidity, prices, governance proposals, and privileged roles. Attackers can inspect them cheaply. They can simulate transactions before sending them. They can fork mainnet locally. They can watch pending transactions. They can build bots that react faster than humans.

Public state has benefits: auditability, verifiability, composability, and independent monitoring. But it also changes the defensive burden.

Do not rely on:

- Hidden contract logic.
- Attackers not knowing the current state.
- Attackers not finding a profitable ordering.
- Attackers not composing your protocol with another.
- Attackers not noticing a governance proposal.
- Attackers not simulating the outcome.
- Attackers behaving like UI users.

In public systems, obscurity is usually temporary. Security must live in the transition rules, economic assumptions, and operational controls.

1.6.4 The Adversary Controls More Than Transaction Parameters

At minimum, an attacker controls the transactions they submit. Often they can influence more:

- Timing: when to submit, delay, replace, or cancel.
- Ordering: through priority fees, private order flow, builder relationships, or sequencer influence.
- Atomicity: bundling multiple operations in one transaction.
- Call graph: routing through contracts or programs that trigger callbacks.
- Identity surface: creating many accounts or addresses.
- Liquidity: moving prices temporarily.
- Governance power: buying, borrowing, or bribing votes.

- Infrastructure view: choosing which RPC, relayer, indexer, or frontend to interact with.
- Cross-domain timing: exploiting finality gaps between chains or layers.

The exact powers differ by system. The habit is the same: make the adversary's powers explicit.

Bad security reasoning says:

No rational user would do that.

Better security reasoning says:

Can any valid actor do that, and would doing it improve their position?

1.6.5 Rejection Is a Security Boundary

A blockchain system is defined as much by what it rejects as by what it accepts.

Rejection can happen at several layers:

- A peer rejects malformed network data.
- A node rejects an invalid transaction.
- A block builder refuses to include a transaction.
- A validator rejects an invalid block.
- A contract reverts.
- A program returns an error.
- A bridge rejects a proof.
- A governance executor rejects a payload that has not passed the timelock.

For reviewers, every rejection path deserves attention:

- Is the rejection condition complete?
- Is it checked before any irreversible side effect?
- Does failure revert all state changes or only some?
- Can the rejection be triggered to grief others?
- Is rejection deterministic across validators?
- Does the caller learn enough to adapt and exploit?
- Does the system fail closed or fail open?

In many smart contract exploits, the missing line is a missing rejection condition:

```
require(caller == owner)
require(amount <= balance)
require(message_not_already_used)
require(price_is_fresh)
require(account.owner == expected_program)
require(output_root_is_final)
```

But a rejection condition is not automatically correct just because it exists. It may check the wrong authority, use stale state, validate the wrong account, trust the wrong domain, or run after an external call.

1.6.6 Adversarial Execution Includes Honest-Looking Transactions

Not every harmful transaction is obviously malicious.

A transaction can be:

- Signed by the legitimate key.
- Routed through a normal UI.
- Included by an honest validator.
- Valid under the protocol rules.
- Economically profitable because of an unintended state interaction.

This is common in wallet drainers, approval phishing, governance manipulation, oracle manipulation, liquidation races, and MEV extraction. The security failure may not be “someone hacked consensus.” It may be that the system allowed a user, bot, admin, relayer, or contract to authorize a transition whose consequences were not understood.

This is why the next chapter treats assets, ownership, authority, and trust boundaries as first-class topics.

1.7 Safety, Liveness, and Fault Assumptions

1.7.1 Safety: Bad States Must Not Happen

Safety properties say that certain bad things must never happen. In blockchain security, safety is about preventing unacceptable states or conflicting histories.

Examples:

- No one can spend the same coin twice in the finalized history.
- No account can transfer funds without authorization.
- No accepted block can create coins outside the issuance rules.
- No bridge message can be executed twice.
- No finalized checkpoint conflicts with another finalized checkpoint under the protocol’s assumptions.
- No vault can issue claims that exceed redeemable assets under its accounting rules.
- No governance action can execute before the required delay.

Safety failures are usually severe because the bad state has already happened. A later transaction may repair accounting, compensate users, or pause the system, but the safety violation itself is part of history unless the chain or application has an explicit recovery mechanism.

In state-machine terms, safety asks:

Can any valid sequence of transitions reach a forbidden state?

If yes, the system is unsafe with respect to that property.

1.7.2 Liveness: Necessary Progress Must Remain Possible

Liveness properties say that something required can eventually happen.

Examples:

- Valid transactions can eventually be included.
- A user can eventually withdraw funds.
- A rollup user can eventually force inclusion or exit.
- A bridge message can eventually be relayed.
- A liquidation system can eventually clear unhealthy debt.
- A governance proposal can eventually execute after approval.
- A paused protocol can eventually recover through a defined path.

Liveness failures can be just as destructive as safety failures. A protocol that never loses accounting correctness but traps user funds forever is not secure in any meaningful sense. A chain that preserves safety by halting may be preferable to accepting conflicting histories, but the halt is still a liveness failure with economic consequences.

In state-machine terms, liveness asks:

From an allowed state, can the system still make required progress under the stated assumptions?

If no, the system is live only on paper.

1.7.3 Safety and Liveness Pull Against Each Other

Distributed systems often face a tradeoff between preserving safety and maintaining liveness under adverse network conditions.

If the network is partitioned, a system may choose to stop finalizing rather than finalize conflicting histories. That preserves safety but sacrifices liveness. A system that continues accepting conflicting histories may preserve local availability but violate global safety.

Blockchain designs make different choices:

- Nakamoto-style systems may continue producing competing branches and resolve them probabilistically through fork choice.
- BFT-style systems may stop finalizing if they cannot gather enough votes.
- Rollups may allow users to exit through L1 mechanisms if the sequencer stops cooperating, but only if the escape hatch is actually implemented and usable.
- Bridges may wait for deeper finality before releasing funds, sacrificing speed to reduce reorg risk.

The security reviewer should not ask “Is it decentralized?” as a vague comfort phrase. Ask:

Under which network and adversary assumptions is safety preserved?

Under which assumptions is progress guaranteed?

What happens outside those assumptions?

1.7.4 Fault Assumptions Are Part of the Security Model

No blockchain system is secure without assumptions. The assumptions may be cryptographic, economic, network-level, implementation-level, or human.

Common assumptions include:

- Hash functions are collision resistant for practical adversaries.
- Signature schemes cannot be forged.
- Honest nodes execute the protocol rules correctly.
- A sufficient fraction of block production power follows the protocol.
- The network eventually delivers messages.
- Validators with enough stake are not colluding to violate safety.
- At least one honest relayer exists for liveness.
- A sequencer cannot permanently censor without users escaping through L1.
- A multisig threshold is not compromised.
- An oracle committee reports fresh and accurate prices within defined bounds.
- A client bug does not affect a supermajority of economic weight.
- Governance voters have enough time and information to react to malicious proposals.

These are not decorative. They determine the boundary of the security claim.

Weak writing says:

The protocol is secure because it is decentralized.

Serious writing says:

The protocol preserves safety if fewer than one-third of voting power is Byzantine during the relevant finality window, clients agree on the transition rules, and the network eventually delivers messages. It preserves withdrawal liveness if at least one honest actor can submit the exit transaction and the base layer remains live.

The second statement can be attacked, tested, debated, and improved. The first cannot.

1.7.5 Byzantine, Rational, and Accidental Faults Are Different

Faults are not all the same.

A crash fault occurs when a component stops. A Byzantine fault occurs when a component behaves arbitrarily: lying, equivocating, censoring, fabricating data, or behaving inconsistently toward different peers. A rational adversary behaves strategically to maximize payoff. An accidental fault may come from a bug, misconfiguration, operator error, or misunderstood integration.

Blockchain systems often face all of these at once:

- A validator can go offline.
- A validator can equivocate.
- A block builder can reorder transactions for profit.
- A user can sign a malicious approval by mistake.
- A developer can deploy with the wrong address.
- A client can parse a malformed block incorrectly.
- A governance participant can vote for a payload they did not inspect.
- A relayer can stop submitting messages because fees are uneconomic.

Do not collapse these into one generic “attacker.” Each fault type requires different controls.

- Crash faults need redundancy and recovery.
- Byzantine faults need validation, quorum assumptions, slashing, fraud proofs, or cryptographic proofs.
- Rational faults need incentive analysis and payoff bounds.
- Human faults need operational controls, interfaces, delays, and review processes.

1.7.6 Time and Finality Are Security Parameters

Many blockchain systems are secure only relative to time.

Examples:

- A Bitcoin payment becomes harder to reverse as more blocks are built after it, under the assumption that the attacker controls less hash power than the honest network.⁸
- A proof-of-stake chain may justify or finalize checkpoints after a voting process.
- A rollup withdrawal may require a challenge window.
- An oracle price may be safe only if it is fresh.
- A governance delay may be safe only if users can detect and respond before execution.
- A liquidation may be safe only if keepers can act before bad debt accumulates.

⁸Satoshi Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System”, <https://bitcoin.org/bitcoin.pdf>.

Security review should therefore ask:

When is this state considered final enough for the next action?

How long can an attacker delay progress?

How stale can this input become before it is unsafe?

How much time do defenders have to react?

Time is not an implementation detail. It is part of the threat model.

1.8 Security Failures as Broken State Transitions

1.8.1 The Most Important Distinction

There are two broad classes of state-machine failure:

1. The system accepts a transition that should be invalid under its explicit rules.
2. The system accepts a transition that is valid under its explicit rules but violates the intended security properties.

The first class is usually a protocol or implementation failure. Examples include a signature verification bug, a consensus client bug, a parser discrepancy, or an invalid block accepted by validators.

The second class is the center of application-layer blockchain security. The chain does exactly what the code says. The code does not say what the protocol actually needed.

Most smart contract exploits are in the second class.

That is why this book will repeatedly separate:

```
rule validity
from
security intent
```

1.8.2 A Taxonomy of Broken Transitions

The state-machine model gives us a clean failure taxonomy.

Failure type	State-machine description	Example
Invalid transition accepted	The validator accepts a transition outside the rules	Invalid block, forged signature, client consensus bug
Valid transition violates intent	The rules allow a reachable state that breaks an invariant	Accounting bug, oracle manipulation, share inflation
Unauthorized transition	State changes under the wrong authority	Missing owner check, wrong signer, account substitution

Failure type	State-machine description	Example
Duplicate transition	A one-time transition executes more than once	Replay, double claim, bridge message reused
Missing transition	A required state change cannot happen	Stuck withdrawal, frozen queue, no recovery path
Wrong ordering	The same inputs in a different order produce harmful state	Front-running, sandwiching, liquidation race
Divergent replicas	Honest nodes compute different states from the same history	Client bug, nondeterminism, parser inconsistency
Unsafe external assumption	Transition relies on a fact not guaranteed by the system	Stale oracle, unfinalized source-chain event, compromised relayer

This taxonomy is not exhaustive, but it is a better starting point than memorizing bug names. Bug names are symptoms. Broken transitions are causes.

1.8.3 Valid Exploits Are Still Exploits

Suppose a vault has this simplified design:

State:

```
asset_balance
total_shares
shares[user]
```

deposit(amount):

```
if total_shares == 0:
    minted = amount
else:
    minted = amount * total_shares / asset_balance
```

```
transfer amount from user to vault
shares[user] += minted
total_shares += minted
```

withdraw(shares_to_burn):

```
amount = shares_to_burn * asset_balance / total_shares
shares[user] -= shares_to_burn
total_shares -= shares_to_burn
transfer amount from vault to user
```

At first glance, the transition rules look reasonable. Deposits mint shares. Withdrawals burn shares. The exchange rate comes from assets divided by shares.

Now ask state-machine questions:

- Can `asset_balance` change without `total_shares` changing?
- What happens when `total_shares` is very small?
- Which way does integer division round?
- Can a deposit mint zero shares?
- Does the first depositor get special power over the exchange rate?
- Is direct donation to the vault part of the transition model?

If an attacker can become the first depositor with a tiny amount, then donate assets directly to increase `asset_balance`, they may manipulate the share price so later depositors receive too few shares because of rounding. Every individual transaction may be valid. The exploit is a valid path through the state machine that violates the intended fairness and accounting properties.

The lesson is not “all vaults are broken.” The lesson is that security properties must be encoded and tested against all reachable states, including states produced by unusual but valid sequences.

1.8.4 The Chain Protects Its Own Rules, Not Your Intent

Base-layer validation enforces base-layer rules. It does not know your business logic unless your business logic is encoded in executable rules.

A chain can verify:

- This signature is valid.
- This nonce is correct.
- This transaction pays enough fee.
- This block follows consensus rules.
- This contract execution ended in a valid state according to the VM.

The chain generally cannot verify:

- This lending market’s risk parameters are economically safe.
- This oracle reflects a fair market price.
- This governance proposal is not malicious.
- This bridge message corresponds to a source event that your protocol should trust.
- This user understood the approval they signed.
- This vault share price is fair to later depositors.

Those properties must be designed, constrained, monitored, or explicitly accepted as trust assumptions.

1.8.5 Reorgs and Ordering Are State-Transition Problems

A reorg is not only a chain-layer curiosity. It changes which state transitions are part of canonical history.

Suppose a protocol observes:

```
Block A includes deposit.  
Protocol credits user on another domain.  
Block A is reorged out.  
Deposit no longer exists in canonical history.  
Credit remains on other domain.
```

The failure is a mismatch between state machines. One domain treated a transition as final before the other domain's finality assumptions justified it.

Ordering has similar consequences. Consider two transactions:

```
tx_1: oracle price update  
tx_2: borrow against collateral
```

The resulting state may differ depending on whether `tx_1` happens before `tx_2` or after it. If attackers can influence ordering, they can sometimes choose which state transition path the system takes.

That is why mempools, block production, finality, bridges, and MEV belong in a blockchain security curriculum. They are not separate from smart contract security. They determine the input order and history that the state machine accepts.

1.8.6 Security Review Starts With Transition Questions

When you encounter a new blockchain system, start with these questions:

1. What is the state?
2. What are the entry points that can change it?
3. What are the validity rules for each entry point?
4. Who or what provides authority for each transition?
5. What external facts does the transition trust?
6. What ordering assumptions does the transition rely on?
7. What must remain true after every transition?
8. What required transitions must remain possible?
9. What happens if the input is delayed, duplicated, reordered, replayed, or removed from history?
10. What happens if an authorized actor behaves maliciously or irrationally?

These questions are intentionally repetitive. Repetition is how you avoid being fooled by surface complexity.

1.9 Worked Example: A Minimal Escrow State Machine

Consider a simple escrow:

State:

```

buyer
seller
arbiter
amount
funded: boolean
released: boolean
refunded: boolean

```

Transitions:

```

fund()
release()
refund()
arbitrate_release()
arbitrate_refund()

```

The state diagram is:

```

stateDiagram-v2
    [*] --> Unfunded
    Unfunded --> Funded: fund
    Funded --> Released: release / arbitrate_release
    Funded --> Refunded: refund / arbitrate_refund
    Released --> [*]
    Refunded --> [*]

```

A product description might say:

The buyer funds the escrow. If delivery succeeds, the seller is paid.

If delivery fails, the buyer is refunded. The arbiter resolves disputes.

That is not yet a security model.

A state-machine review asks for explicit transition rules:

Transition	Caller	Preconditions	State change	Value movement
fund()	buyer	funded == false and payment received	funded = true	Buyer deposits amount into escrow

Transition	Caller	Preconditions	State change	Value movement
release()	buyer	funded == true, released == false, refunded == false	released = true	Escrow pays seller
refund()	seller	funded == true, released == false, refunded == false	refunded = true	Escrow pays buyer
arbitrate_release ()	arbiter	funded == true, released == false, refunded == false	released = true	Escrow pays seller
arbitrate_refund ()	arbiter	funded == true, released == false, refunded == false	refunded = true	Escrow pays buyer

Now write invariants:

released and refunded cannot both be true

```
if released == true:
    seller has been paid exactly once
```

```
if refunded == true:
    buyer has been refunded exactly once
```

```
if funded == false:
    release and refund transitions cannot transfer funds
```

only buyer can voluntarily release
only seller can voluntarily refund
only arbiter can resolve disputes

The forbidden states should be visible before code review begins:

Forbidden state	Why it matters	Required guard
<code>released == true</code> and <code>refunded == true</code>	Pays both sides from one escrow	Every payout transition checks both terminal flags
<code>funded == false</code> and funds transferred out	Pays from an unfunded escrow	Payouts require successful funding
Seller paid more than once	Double spend against escrow balance	Mark terminal state before unsafe external control flow
Buyer refunded more than once	Duplicate refund	Consume withdrawal/refund state exactly once
Arbiter changed by one party during dispute	Authority capture	Arbiter changes require explicit, delayed, reviewable authority
Terminal state reached but transfer failed	Accounting says paid while value did not move	State change and value movement must have consistent failure behavior

Now look for broken transitions:

- What if `release()` transfers funds before setting `released = true` and the recipient can reenter?
- What if `arbitrate_release()` forgets to check `refunded == false`?
- What if `fund()` can be called twice and overwrites `amount`?
- What if `refund()` sends funds to `msg.sender` instead of `buyer`?
- What if the arbiter address can be changed by the seller?
- What if the token used for payment takes a fee on transfer?
- What if the transfer fails but the state is marked released?
- What if neither buyer nor seller cooperates and the arbiter key is lost?

Notice the range: authorization, accounting, external calls, token semantics, liveness, and operational risk all appear in a tiny state machine. The same method scales to larger systems.

1.10 Practical Discipline: Do Not Start With Bug Names

Beginners often ask:

Is this vulnerable to reentrancy?

Is this vulnerable to oracle manipulation?

Is this vulnerable to replay?

Those are useful questions, but they are not the first questions.

Start earlier:

What transition changes value?

What authority permits it?

What invariant should it preserve?

What external fact does it rely on?

What happens if the transition occurs twice?

What happens if it occurs before or after another transition?

What happens if it never occurs?

Then bug classes become labels for specific ways the transition model can fail.

Bug label	State-machine interpretation
Reentrancy	External control flow allows another transition before invariants are restored
Replay	An input is accepted again after it should have been consumed
Oracle manipulation	A transition depends on an external value whose production rules do not support the claimed property
MEV	Observation, inclusion, or ordering power changes which valid transition sequence executes
Bridge failure	State, proof, finality, domain separation, or trust assumptions do not line up across machines

This is how the rest of the book will proceed.

1.11 Review Questions

1. What information belongs in “state” for a UTXO system and for an account-based smart contract system?
2. What is the difference between a transaction object and the state transition it causes?
3. Why can a transaction be valid under explicit rules but unsafe under protocol intent?
4. Why must replicated blockchain execution be deterministic?
5. Give one safety property and one liveness property for a bridge or lending protocol.
6. Why does “the chain accepted the transaction” not prove that the application was secure?
7. What assumption is being made when a system claims safety below a one-third Byzantine threshold?

1.12 Applied Exercises

1.12.1 Exercise 1: Model a Token Transfer

Write a state-machine model for a simple token with:

- `balances[address]`
- `total_supply`
- `transfer(sender, recipient, amount)`
- `mint(recipient, amount)`
- `burn(holder, amount)`

For each transition, write:

- Required authority.
- Validity conditions.
- State updates.
- At least two invariants.
- A transition table.

Then answer: which invariants depend on the mint authority being honest or constrained?

1.12.2 Exercise 2: Separate Rule Validity From Intent

Consider a protocol that allows anyone to deposit collateral and borrow up to 80 percent of the collateral's oracle value.

The contract checks:

```
collateral_value * 80 / 100 >= borrowed_value
```

Assume the oracle price can be manipulated for one block using a low-liquidity pool.

Answer:

- Is the borrow transaction valid under the contract's explicit rule?
- Which security property is violated?
- What external assumption was false?
- Is this a safety failure, liveness failure, or both?

1.12.3 Exercise 3: Identify Liveness Risk

A rollup says users can withdraw through L1 if the sequencer censors them. The withdrawal path requires a proof generated by an off-chain service run by the rollup team.

Answer:

- What safety property is the escape hatch supposed to protect?
- What liveness assumption does it introduce?
- What happens if the sequencer is censoring and the proof service is down?
- How would you rewrite the security claim more honestly?

1.12.4 Exercise 4: Broken Transition Taxonomy

Classify each failure:

1. A bridge accepts the same message twice.
2. A validator client accepts a block another client rejects.
3. A vault lets the first depositor manipulate share price through direct donation.
4. A governance proposal executes before the timelock expires.
5. A withdrawal queue can become permanently stuck because one recipient reverts.
6. A frontend asks users to sign a transaction that grants unlimited token approval to an attacker.

For each, identify:

- The relevant state.
- The broken transition.
- The missing or false assumption.
- Whether the primary failure is safety, liveness, authorization, ordering, or user-side authority.

The Part I Integrated Lab will combine this chapter’s state-machine method with assets, authority, threat modeling, and invariants. These exercises are intentionally smaller: they train the pieces.

1.13 Key Takeaways

- A blockchain is best understood as a replicated state machine operating under adversarial inputs.
- State is not only balances. It includes every variable, commitment, role, parameter, queue, nonce, proof status, and authority record that affects future behavior.
- Transactions are inputs. State transitions are the changes caused by applying inputs under rules.
- Validity is layered. A transaction can be syntactically valid, signed, included, and executable while still violating protocol intent.
- Replication requires deterministic execution. Honest nodes must compute the same result from the same starting state and ordered inputs.
- Consensus and fork choice select history. Execution applies transition rules to that history.
- Safety means forbidden states or conflicting histories do not happen. Liveness means required progress remains possible.
- Fault assumptions are part of the security claim. If they are vague, the security claim is vague.

- Most serious application-layer exploits are valid paths through an underspecified or incorrectly constrained state machine.

1.14 Further Reading

- Leslie Lamport, “Computation and State Machines”. A concise foundation for thinking about computation as state machines and reasoning about invariants.
- Gavin Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger”. The formal Ethereum specification, especially the blockchain paradigm and state-transition framing.
- ethereum.org, “Transactions”. A practical introduction to Ethereum transactions as signed state-changing instructions.
- ethereum.org, “Blocks”. A practical introduction to blocks as ordered batches of transactions linked by hashes.
- Satoshi Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System”. The original Bitcoin paper, especially the network, proof-of-work, and double-spend sections.
- Bowen Alpern and Fred B. Schneider, “Defining Liveness” and “Recognizing Safety and Liveness.” These papers are the classical formal background for the safety/liveness distinction used throughout distributed systems.

Chapter 2

Assets, Authority, and Trust Boundaries

2.1 Purpose

Chapter 1 framed blockchain systems as adversarial state machines. Chapter 2 asks the next question:

Which parts of the state are valuable, and who has the authority to change them?

Most blockchain security failures involve confusion about assets, authority, or trust boundaries. A protocol team says “users own their funds,” but an upgrade key can replace the withdrawal logic. A token claims to be backed by off-chain reserves, but the reserve proof is only an attestation. A bridge mints a wrapped asset, but the wrapped asset is only as strong as the source-chain finality, validator set, relayers, and message verification path. A governance system looks decentralized, but a small quorum, delegated voting, emergency guardian, or timelock admin can still decide outcomes.

Security analysis becomes unserious when it treats “ownership” as a word rather than a state-machine fact.

For this book, ownership means nothing until you can answer:

Which state variable, object, output, role, or claim records the asset?

Which transition moves or modifies it?

Which authority can cause that transition?

Which assumptions make that authority trustworthy enough?

This chapter builds the vocabulary for those answers.

2.2 Learning Objectives

By the end of this chapter, you should be able to:

- Define on-chain assets as enforceable claims over state, not just tokens.
- Distinguish ownership, possession, authority, custody, control, and economic exposure.
- Compare ownership models across UTXOs, accounts, contracts, Solana accounts, and Move resources.
- Identify privileged authorities such as admins, governors, validators, sequencers, custodians, oracles, relayers, and upgrade controllers.
- Draw trust boundaries across on-chain code, off-chain infrastructure, human processes, legal claims, and cross-chain systems.
- Build an authority map showing who can move, freeze, mint, burn, upgrade, censor, or otherwise affect assets.

2.3 The Core Distinction

An asset is something valuable represented by, controlled by, or dependent on system state.

Authority is the ability to cause state transitions that affect that asset.

These are related, but not the same.

An address may hold tokens but have granted a spender allowance. A user may own vault shares but depend on a strategist, oracle, withdrawal queue, and admin pause key. A DAO may own a treasury but a timelock executor may be able to trigger the transfer. A stablecoin holder may own an on-chain balance but depend on off-chain reserve custody and redemption policy. A rollup user may own an L2 balance but depend on the L1 bridge contract, proof system, data availability, and exit path.

The basic security questions are:

Asset:

What has value?

State:

Where is that value represented?

Authority:

Who or what can change that representation?

Boundary:

Which facts must be trusted rather than verified locally?

If you cannot answer those questions, you do not understand the system's security model yet.

2.4 What Counts as an Asset On-Chain

2.4.1 Assets Are Enforced Claims Over State

An on-chain asset is not a tiny object sitting inside an address. It is a claim recognized by a state-transition system.

For example:

- A Bitcoin coin is spendable because an unspent transaction output exists and a future transaction can satisfy its spending condition.
- ETH is spendable because an Ethereum account balance exists and a valid transaction can debit it.
- An ERC-20 token balance is spendable because a contract’s state maps an address to a number and the contract’s transfer rules allow that number to decrease and another to increase.
- An ERC-721 NFT is transferable because a contract maps a token ID to an owner and its transfer rules recognize the caller as owner or authorized operator.
- A Solana token account records token state inside an account whose data is governed by a program.
- A Move resource represents value partly because the language and module system restrict copying, dropping, storing, and unpacking.
- A bridged asset is a claim in one domain that depends on locked, burned, proven, or otherwise accounted value in another domain.
- A governance token is both an economic asset and a control surface.

The asset exists because the system’s rules say certain future transitions are allowed and others are not.

That is why security review cannot stop at “who owns the token.” The real question is:

Which transitions can affect the claim?

2.4.2 Asset Categories

A useful asset inventory should include more than obvious balances.

Common blockchain asset categories:

Category	Examples	Security question
Native assets	BTC, ETH, SOL, staking rewards	Who can spend, slash, censor, or reorder transactions affecting them?

Category	Examples	Security question
Fungible tokens	ERC-20, SPL tokens, stablecoins	Who can transfer, approve, mint, burn, freeze, or blacklist?
Non-fungible assets	ERC-721 tokens, game items, account-bound records	Who controls transfer, metadata, approvals, and operator authority?
Semi-fungible assets	ERC-1155-style assets, batch-minted items	Are balances, IDs, and batch operations checked consistently?
Shares and claims	LP tokens, vault shares, receipt tokens, restaking positions	What do shares claim, and can the exchange rate be manipulated?
Debt and liabilities	Borrow positions, margin accounts, undercollateralized debt	Who can create, liquidate, forgive, or socialize debt?
Capabilities and roles	Admin roles, minter roles, pause guardians, upgrade keys	What privileged transitions do they unlock?
Governance power	Voting tokens, delegation, proposal rights	Can governance change the rules or seize assets?
Cross-domain claims	Wrapped assets, bridge receipts, rollup withdrawals	Which external state and finality assumptions are trusted?
Off-chain claims	Fiat-backed stablecoins, RWAs, redemption rights	Which legal, custody, and reporting systems are outside the chain?
Information assets	Oracle prices, validator keys, private order flow, signer policies	Who can distort or leak information that controls value?

The point of this table is not classification for its own sake. It is to force a reviewer to notice assets that are not called “tokens.”

A governance role is an asset. An oracle key is an asset. A bridge validator key is an asset. An unexecuted withdrawal claim is an asset. A pending governance proposal may become an asset or a liability depending on who can execute it.

2.4.3 Native Assets and Contract Assets Differ

Native assets are accounted for by the base protocol. Contract assets are accounted for by contract or program logic.

On Ethereum, ETH balances are part of account state, while ERC-20 balances are state inside a token contract. Ethereum’s documentation describes accounts as entities with ETH balances that can send messages, and distinguishes externally owned accounts controlled by private keys from contract accounts controlled by code.¹

That distinction matters.

If a user sends ETH, the base protocol updates account balances according to transaction rules. If a user transfers an ERC-20 token, the token contract updates its internal balance mapping according to its own implementation. The ERC-20 standard specifies functions such as `balanceOf`, `transfer`, `transferFrom`, `approve`, and `allowance`, but different implementations can still behave differently within or around that interface.²

Security consequences:

- A wallet may display both ETH and ERC-20 balances, but those balances are enforced by different rule systems.
- A transfer of ETH and a transfer of a token can have different failure behavior.
- A token can have minting, burning, pausing, blacklisting, rebasing, transfer fees, hooks, or nonstandard return values.
- A protocol that treats all tokens as if they were identical ERC-20s will eventually be wrong.

Do not say “the user has funds” until you know which accounting system recognizes the claim.

2.4.4 An Asset Can Be a Right, Not a Balance

Many important assets are rights to do something later.

Examples:

- A withdrawal ticket.
- A bridge message that has been proven but not executed.
- A governance proposal that can be queued.
- A validator withdrawal credential.
- A liquidation right.
- A claim on future rewards.
- A timelocked allocation.

¹ethereum.org, “Ethereum Accounts”, <https://ethereum.org/developers/docs/accounts/>.

²Fabian Vogelsteller and Vitalik Buterin, “ERC-20: Token Standard”, <https://eips.ethereum.org/EIPS/eip-20>.

- A vesting stream.
- A pending unstake.
- A signature that authorizes a permit or order.

These rights may not look like balances, but they are still valuable. Their security depends on whether they can be created, transferred, consumed, canceled, replayed, or blocked.

For every such right, ask:

How is the right represented?

Who can create it?

Who can consume it?

Can it be consumed more than once?

Can it expire?

Can it be canceled?

Can it be transferred?

Can it be made unusable?

Rights are especially important in cross-chain, lending, governance, and staking systems, where value often exists in an intermediate state.

2.4.5 Off-Chain Claims Are Not Made On-Chain by Naming Them

Some assets are on-chain records of off-chain claims.

Examples:

- Fiat-backed stablecoins.
- Tokenized treasury bills.
- Real-world asset tokens.
- Custodial wrapped BTC.
- Claims on carbon credits, invoices, real estate, or commodities.

The on-chain token may be transferred by code, but the underlying claim depends on legal agreements, custody, redemption policies, banking access, auditors, administrators, and sometimes court systems.

The chain can enforce:

`address A has 100 tokens`

`address A can transfer 100 tokens if not blocked`

The chain usually cannot enforce:

`the issuer has matching reserves`

`the custodian did not rehypothecate assets`

`the bank account remains accessible`

the token holder has legally enforceable redemption rights
the reserve report is complete and current

This does not make off-chain claims worthless. It means they cross a trust boundary. A serious security analysis states that boundary directly.

Bad:

The token is backed 1:1.

Better:

The token contract records transferable claims. Redemption depends on issuer solvency, reserve custody, banking access, legal enforceability, and the issuer's honoring of redemptions.

The second statement is less marketable. It is also more honest.

2.4.6 Control Surfaces Are Assets

Attackers often target control before they target balances.

Control surfaces include:

- Upgrade authority.
- Mint authority.
- Burn authority.
- Pause authority.
- Blacklist or freeze authority.
- Oracle update authority.
- Governance proposal rights.
- Timelock proposer, executor, and canceller roles.
- Bridge validator or relayer keys.
- Sequencer control.
- Validator keys.
- Backend signer keys.
- Deployment keys and CI/CD secrets.
- Frontend domains and package-publishing accounts.

These are assets because controlling them can lead to asset movement, denial of service, governance capture, or reputation damage.

If a protocol has \$10 million in a vault and a single key can upgrade the vault to drain funds, the key is economically equivalent to control over the vault. Treat it accordingly.

2.5 Ownership: Keys, Accounts, UTXOs, Objects, and Claims

2.5.1 Ownership Is a State-Machine Relationship

In ordinary language, ownership sounds legal or moral. In blockchain security, ownership is operational:

An actor owns an asset to the extent that the system recognizes that actor,
or an authority controlled by that actor,
as able to exercise the relevant transitions over the asset.

This definition is intentionally dry. It prevents category errors.

A private key may control an account. The account may hold a token. The token may represent a claim on reserves. The reserves may be held by a custodian. The custodian may be subject to law. The user may have legal rights against the issuer. These are different layers of ownership and authority.

Do not collapse them.

2.5.2 UTXO Ownership: Spend Conditions, Not Account Balances

In a UTXO system, the state contains unspent outputs. Each output has a value and a spending condition. To “own” the output is to be able to satisfy that condition in a valid transaction.

Bitcoin’s whitepaper describes coins as transferred through digital signatures and then immediately identifies the hard problem: the recipient needs assurance that an earlier owner did not double-spend.³ That is why ownership in Bitcoin is not just “this public key appears in a chain of signatures.” It also depends on the network agreeing which spend came first.

Security consequences:

- The same key may control many outputs.
- A wallet balance is a derived view over a set of spendable outputs.
- Spending one output usually consumes it and creates new outputs.
- Coin selection affects privacy, fees, and sometimes security.
- Script conditions can express multisig, timelocks, hashlocks, and other constraints.
- A UTXO can be unspendable if its script is impossible to satisfy or if required keys are lost.

The security question is not:

Which account owns the money?

It is:

Which unspent outputs exist, and what conditions allow each to be spent?

³Satoshi Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System”, <https://bitcoin.org/bitcoin.pdf>.

2.5.3 Account Ownership: Keys, Code, and Nonces

Account-based systems present a different interface. On Ethereum, externally owned accounts are controlled by private keys; contract accounts are controlled by code.⁴

This distinction is foundational.

An EOA can initiate transactions because someone can produce signatures for it. A contract account cannot initiate transactions on its own in the same way; it executes when called, and its authority is defined by code and state.

Security consequences:

- Losing an EOA private key usually means losing practical control over the account's assets.
- A compromised EOA can sign valid transactions that are disastrous.
- A contract can hold assets but expose only specific methods for moving them.
- A contract can be the owner of another contract.
- Nonces prevent simple replay of transactions from the same account.
- Smart accounts and account abstraction blur the interface, but not the underlying question: which validation logic authorizes the transition?

Ethereum.org also notes that an account is not a wallet; a wallet is an interface for interacting with an account.⁵ That distinction becomes important in user-side security. A wallet can mislead a user, fail to simulate consequences, connect to a malicious site, or sign the wrong message. The account is the state object; the wallet is a tool that controls or mediates authority.

2.5.4 Contract Ownership Is Just State

When a Solidity contract has an `owner`, that owner is usually a state variable checked by privileged functions.

Example:

```
address public owner;

modifier onlyOwner() {
    require(msg.sender == owner, "not owner");
    _;
}

function setFee(uint256 newFee) external onlyOwner {
    fee = newFee;
}
```

⁴ethereum.org, “Ethereum Accounts”, <https://ethereum.org/developers/docs/accounts/>.

⁵ethereum.org, “Ethereum Accounts”, <https://ethereum.org/developers/docs/accounts/>.

This is not mystical. It means:

State:

```
owner
fee
```

Transition:

```
setFee(newFee)
```

Validity:

```
caller == owner
```

Effect:

```
fee = newFee
```

OpenZeppelin’s access-control documentation emphasizes that access control governs who can perform actions such as minting, voting, freezing transfers, or administrative tasks.⁶ It also distinguishes simple ownership from role-based access control, where separate roles can authorize different actions.

The security reviewer should ask:

- Who is the owner?
- Can ownership be transferred?
- Is transfer one-step or two-step?
- Can ownership be renounced?
- If ownership is renounced, does the system lose necessary recovery powers?
- If ownership is transferred to a contract, what controls that contract?
- If ownership is transferred to a multisig, who controls the signer keys?
- If ownership is transferred to governance, can governance be captured?
- If ownership is transferred to a timelock, who can propose, cancel, and execute?

An `owner` variable is rarely the end of the authority chain. It is usually the beginning.

2.5.5 Token Ownership Is Defined by the Token’s Rules

Token standards define interfaces, not complete security models.

ERC-20 defines basic functions for balances, transfers, approvals, and allowances. It explicitly includes delegated spending through `approve`, `transferFrom`, and `allowance`.⁷ That means a user’s token balance is not the only state relevant to movement. Allowances are authority too.

For an ERC-20-like token, asset movement may be authorized by:

⁶OpenZeppelin, “Access Control”, <https://docs.openzeppelin.com/contracts/5.x/access-control>.

⁷Fabian Vogelsteller and Vitalik Buterin, “ERC-20: Token Standard”, <https://eips.ethereum.org/EIPS/eip-20>.

- The holder calling `transfer`.
- A spender calling `transferFrom` using allowance.
- A minter increasing supply.
- A burner reducing supply or destroying a user’s balance.
- An admin pausing transfers.
- A blacklist authority blocking addresses.
- A rebasing mechanism changing balances.
- A bridge contract minting or burning representations.

ERC-721 adds different authority semantics. The standard tracks ownership by token ID, includes approvals, and includes operators that can manage all NFTs of an owner.⁸ A security review of NFTs therefore must inspect not only `ownerOf(tokenId)` but also per-token approvals, operator approvals, transfer hooks, metadata authority, and marketplace integration.

The general rule:

Do not infer token authority from the word "owner."
Read the token’s transition rules.

2.5.6 Solana Ownership: Account Owner Program and Signers

Solana uses “owner” in a way that can confuse EVM-trained reviewers.

Solana documentation describes accounts as the fundamental data units storing state, with each account identified by a 32-byte address. It also states that only the account’s owner program can modify its data or debit lamports, while any program can credit lamports to a writable account.⁹

This is not the same as “the user owns this account” in ordinary language.

For this chapter, keep only the ownership lesson: separate the program that can modify account data, the signer that authorized the instruction, and the account constraints that prove the passed accounts are the intended accounts. Part III will study the detailed Solana execution model.

For Solana, ask:

Which program owns this account’s data?

Which signer authorized this instruction?

Which accounts are writable?

Which account constraints prove that these are the intended accounts?

Which PDA seeds establish program authority?

⁸William Entriken et al., “ERC-721: Non-Fungible Token Standard”, <https://eips.ethereum.org/EIPS/eip-721>.

⁹Solana Foundation, “Accounts”, <https://solana.com/docs/core/accounts>.

2.5.7 Move Ownership: Resources and Module Boundaries

Move was designed around resource-like values whose abilities control whether they can be copied, dropped, stored, or used as storage keys. The current Move Book describes abilities as the typing feature that controls what actions are permissible for values of a given type, including linear behavior and storage use.¹⁰

Move changes the shape of some asset bugs. It can make accidental duplication or dropping of resources harder than in less restrictive models. It does not remove the need to map creation, destruction, transfer, privileged capabilities, and module boundaries. Part III will handle the execution-model details.

You still need to ask:

- Which module defines the resource?
- Which functions can create it?
- Which functions can destroy it?
- Which functions can transfer or store it?
- Which capabilities authorize privileged actions?
- Which invariants are enforced by the type system, and which remain protocol-level assumptions?

Move’s resource model is a strong tool for representing assets. It is not a substitute for designing correct authority.

2.5.8 Legal Ownership and Protocol Control Can Diverge

Sometimes the person legally entitled to an asset is not the actor who can move it on-chain.

Examples:

- A user deposits into a centralized exchange. The exchange controls the withdrawal wallet.
- A stablecoin holder may have an on-chain balance but redemption depends on issuer terms.
- A custodian signs transactions for an institution.
- A DAO treasury may be legally associated with a foundation while on-chain control sits with a multisig or governance contract.
- A court order may require freezing assets, but the chain only enforces that if some authority can freeze them.
- A compromised key can move assets even though the attacker is not the rightful owner.

For security work, distinguish:

¹⁰The Move Book, “Abilities”, <https://move-book.com/reference/abilities/>.

Legal title:

Who has an off-chain claim?

Protocol authority:

Who can cause the state transition?

Custody:

Who controls the signing material or operational process?

Economic exposure:

Who gains or loses value if the state changes?

These categories often overlap. When they do not, risk appears.

2.6 Authority: Users, Admins, Validators, Governance, and Custodians

2.6.1 Authority Is the Ability to Cause a Transition

Authority is not status. It is not reputation. It is not what the docs say someone should do. Authority is the ability to cause a state transition that the system will accept.

An authority may be:

- A private key.
- A signature threshold.
- A smart contract condition.
- A role assignment.
- A governance vote.
- A validator majority.
- A sequencer.
- A relayer.
- An oracle committee.
- A backend signing service.
- A custodian's internal approval process.
- A legal administrator with control over off-chain reserves.

Authority can be explicit or implicit.

Explicit authority is easy to see:

```
onlyOwner
onlyRole(MINTER_ROLE)
```



```
signer == user  
threshold signatures >= 3 of 5
```

Implicit authority is more dangerous:

```
the frontend constructs the transaction  
the indexer decides which position exists  
the oracle publishes the price  
the backend signs withdrawals  
the relayer decides whether to submit messages  
the sequencer decides ordering  
the deployer can upgrade implementation  
the package maintainer can ship malicious code
```

Both kinds matter.

2.6.2 User Authority

User authority usually comes from keys, signatures, account-validation logic, or possession of a capability.

Common user authorities:

- Transfer owned assets.
- Approve a spender.
- Delegate votes.
- Sign a permit.
- Create an order or intent.
- Deposit or withdraw.
- Borrow, repay, liquidate, stake, or claim.
- Configure account recovery or session keys.

User authority is often assumed to be benign. That is a mistake. A malicious user is still a user. A compromised user key is still able to sign. A tricked user may authorize a harmful transition. A bot may call public functions in sequences no ordinary user would consider.

The security question is:

What can any valid user do, not what should an honest user do?

2.6.3 Admin Authority

Admin authority is the ability to change protocol behavior, parameters, or emergency state.

Examples:

- Upgrade implementation.
- Change fees.
- Add collateral types.
- Set risk parameters.
- Pause or unpause.
- Freeze accounts.
- Set oracle addresses.
- Assign minter or burner roles.
- Change bridge validator sets.
- Recover stuck funds.
- Sweep tokens.

Admin authority is not automatically bad. Many systems need controlled administration for upgrades, parameter tuning, emergency response, and operational recovery. The problem is pretending admin authority is not trust.

OpenZeppelin’s docs make the core point bluntly: access control may determine who can mint tokens, vote, freeze transfers, or perform other critical actions.¹¹ Timelocks add delay to privileged operations so users can inspect and react before maintenance actions execute.¹²

Security review must answer:

- What can admins do?
- Can admins directly take user funds?
- Can admins indirectly take user funds by upgrading, changing parameters, or replacing dependencies?
- Is there a timelock?
- Who can bypass the timelock?
- Can emergency powers be abused?
- Is there a path to recover if admin keys are lost?
- Are admin actions observable before they take effect?

Admin authority should be treated as an attack surface and a trust assumption, not a footnote.

2.6.4 Validator, Miner, Builder, and Sequencer Authority

Block production authorities do not usually own user assets directly, but they can affect the history in which asset transitions occur.

Depending on the system, these actors may influence:

¹¹OpenZeppelin, “Access Control”, <https://docs.openzeppelin.com/contracts/5.x/access-control>.

¹²OpenZeppelin, “Access Control”, <https://docs.openzeppelin.com/contracts/5.x/access-control>.

- Transaction inclusion.
- Transaction ordering.
- Censorship.
- Reorg risk.
- Finality.
- MEV extraction.
- L2 batch publication.
- Data availability.
- Cross-domain timing.

Bitcoin’s whitepaper frames the need for a system where participants agree on a single history of transaction order to address double spending without a trusted mint.¹³ That is an authority problem: who decides which spend came first?

In modern systems, the authority may be distributed across validators, builders, relays, sequencers, or committees. Even when these actors cannot forge signatures, they may still decide whether and when valid transitions become canonical.

Security review must ask:

Can block producers reorder this operation profitably?

Can they censor exits or liquidations?

Can they delay oracle updates?

Can they observe intent before inclusion?

Can they create a reorg that invalidates an assumed state?

Can a sequencer include a transaction on L2 before the data or output is secure on L1?

Authority over ordering is authority over outcomes.

2.6.5 Governance Authority

Governance is privileged execution with a voting interface.

It may control:

- Treasury movement.
- Protocol upgrades.
- Parameter changes.
- Role assignments.
- Emergency powers.
- Validator-set changes.
- Oracle sets.
- Bridge configuration.

¹³Satoshi Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System”, <https://bitcoin.org/bitcoin.pdf>.

- Fee switches.

Governance should not be romanticized as “the community.” A governance system is a concrete mechanism with specific state:

```
token balances
delegations
quorum
proposal threshold
voting delay
voting period
timelock delay
execution targets
cancel paths
emergency vetoes
```

The real authorities are the actors who can cause proposals to pass and execute. That may include token whales, delegates, bribery markets, exchanges, multisigs, guardians, foundations, or attackers using flash liquidity.

Governance is safer only if its authority is well scoped, observable, delayed where needed, and resistant to realistic capture.

2.6.6 Custodial and Operational Authority

Some systems depend on people and organizations holding keys or running processes.

Examples:

- Exchange hot wallets.
- Treasury multisigs.
- MPC signing clusters.
- HSM-backed custodians.
- Backend withdrawal signers.
- Bridge validator operators.
- Oracle publishers.
- Relayers and keepers.
- Deployment engineers.
- CI/CD systems.
- Domain registrars and DNS providers.

This authority often lives outside the smart contract but controls contract-relevant transitions.

For example, a contract may require `onlyWithdrawalSigner`, but the real question is:

Who can make the signing service sign?
 Who can change the signing policy?
 Who can deploy signer code?
 Who can access production secrets?
 Who approves emergency withdrawals?
 What happens if one operator is compromised?
 What happens if the signer is unavailable?

Human and organizational authority belongs in the security model.

2.6.7 Authority Can Be Delegated, Composed, or Hidden

Authority is rarely a single direct edge.

Consider:

```
Vault.owner = Timelock
Timelock.proposer = Governor
Governor voting power = token balances + delegation
Token balances include tokens held by an exchange
Timelock.executor = anyone
Guardian can cancel proposals
Proxy admin can upgrade Governor
Frontend helps users delegate
```

Who controls the vault?

There is no one-line answer. Control is a graph.

Reviewers should draw that graph:

```
Exchange custody
  -> token voting power
  -> governance outcome
  -> timelock proposal
  -> vault upgrade
  -> user asset movement
```

Hidden authority often appears in:

- Upgradeable proxy admins.
- Role admin roles.
- Token allowances.
- Operator approvals.
- Meta-transaction forwarders.
- Signature validators.
- Trusted relayers.

- Oracle owners.
- Mutable registries.
- Off-chain signers.
- Emergency pause modules.
- Dependency contracts that can be replaced.

If authority is hidden, the system may appear safer than it is.

2.7 Trust Boundaries Across On-Chain, Off-Chain, and Human Systems

2.7.1 A Trust Boundary Marks a Change in Verification

A trust boundary is where one part of the system must rely on claims, behavior, or data from another part that it cannot fully verify for itself.

OWASP describes threat modeling as identifying and understanding threats and mitigations in the context of protecting something valuable.¹⁴ NIST’s zero trust architecture guidance makes a related general security point: trust should not be implicit merely because of network location or asset ownership.¹⁵

For blockchain security, translate that into this rule:

Do not trust a component just because it is nearby, official, on-chain,
off-chain, decentralized-looking, or operated by the protocol team.
Name what it can do and what verifies it.

A trust boundary is crossed when:

- A contract accepts an oracle price.
- A bridge accepts a source-chain proof.
- A rollup accepts a sequencer batch.
- A frontend constructs a transaction for a wallet.
- A backend signs a withdrawal.
- A governance proposal calls an arbitrary target.
- A token contract trusts another token’s behavior.
- An indexer supplies state to a UI.
- A user trusts a wallet simulation.
- A protocol trusts a multisig not to abuse an upgrade key.

At each boundary, ask:

¹⁴OWASP, “Threat Modeling”, https://owasp.org/www-community/Threat_Modeling.

¹⁵NIST SP 800-207, “Zero Trust Architecture”, <https://csrc.nist.gov/pubs/sp/800/207/final>.

What is being trusted?
Who can lie?
Who verifies?
What happens if the claim is false?
Can the system fail closed?
Can users observe or exit before damage?

2.7.2 On-Chain Does Not Mean Trustless

On-chain code is transparent and independently executable, but it can still contain trust assumptions.

On-chain trust assumptions include:

- A privileged role will not abuse power.
- An upgrade admin will not deploy malicious code.
- A governance process will not be captured.
- A token contract will behave like expected.
- A library or dependency address is correct.
- A proxy points to the intended implementation.
- A registry returns correct addresses.
- A timelock delay is long enough for users to react.
- A contract's code is verified and corresponds to the deployed bytecode.

Even if every line is on-chain, the system may still trust people, keys, roles, and external state.

On-chain verification reduces some forms of trust. It does not erase trust.

2.7.3 Off-Chain Does Not Mean Insecure

Off-chain components are not automatically bad. They often provide necessary services:

- Wallets.
- Frontends.
- Indexers.
- Oracles.
- Keepers.
- Relayers.
- Sequencers.
- Provers.
- Monitoring systems.
- Signing infrastructure.

- Governance discussion forums.

The issue is whether their failure modes are explicit and controlled.

For each off-chain component, ask:

- Is it trusted for safety, liveness, or only convenience?
- Can users bypass it?
- Can it forge state transitions, or only suggest them?
- Can it censor or delay?
- Can it lie without being detected?
- Can multiple independent providers reduce risk?
- Is there an on-chain verification step?
- What happens if it disappears?

An off-chain indexer that only improves UI performance is one thing. An off-chain signer that can authorize withdrawals is another. A sequencer that can delay users but not steal funds is different from a bridge multisig that can mint wrapped assets.

Do not classify components by location. Classify them by authority and failure impact.

2.7.4 Cross-Chain Systems Create Stacked Trust Boundaries

Bridges, rollups, appchains, and cross-chain applications combine multiple state machines. That creates stacked trust boundaries.

A simple lock-and-mint bridge may depend on:

- Source-chain finality.
- Source bridge contract correctness.
- Event or message commitment.
- Relayers.
- Validator or multisig signatures.
- Destination bridge contract correctness.
- Domain separation.
- Replay protection.
- Mint authority for the wrapped asset.
- Governance over bridge configuration.

The wrapped token on the destination chain may look like a normal token, but its backing depends on the entire path.

Security review should avoid statements like:

This is backed by locked assets on chain A.

Instead say:

This claim is valid if the source-chain lock event is final,
the bridge verification path is sound,
the relayer or validator assumptions hold,
the destination contract enforces replay protection,
and the bridge authority cannot mint outside the intended rules.

That is the trust boundary made visible.

2.7.5 Human Systems Are Part of the Boundary

Human systems include:

- Multisig signer coordination.
- Incident response.
- Governance participation.
- Key backup and recovery.
- Deployment review.
- Release management.
- Legal compliance.
- Custody operations.
- Oracle operations.
- Community monitoring.

These systems can be strong or weak. They can have redundancy, separation of duties, checklists, audits, delays, and rehearsed response plans. Or they can be one engineer with a hot wallet and a production `.env` file.

Blockchain security often fails because the code was treated as the whole system.

Ask:

Which human process can authorize or block asset movement?

Which process can change code or configuration?

Which process notices malicious changes?

Which process can recover from mistakes?

Which process is assumed to be honest, available, and competent?

If the answer is “the team will handle it,” you have found an unanalyzed trust boundary.

2.7.6 Trust Boundaries Should Be Written as Assumptions

A good trust-boundary statement is concrete enough to be challenged.

Weak:

The oracle is trusted.

Better:

The lending market assumes the oracle reports a price within 1 percent of the executable market price and updates at least every 30 minutes. If the oracle is stale or manipulable, accounts may borrow against inflated collateral and leave bad debt.

Weak:

The multisig is secure.

Better:

The protocol assumes at least 3 of 5 multisig signers remain uncompromised and available. The multisig can upgrade the vault implementation after a 48-hour timelock, which gives users a chance to exit only if withdrawals remain available during the delay.

Weak:

The bridge verifies messages.

Better:

The destination contract accepts messages signed by 13 of 19 bridge validators. If 13 validators collude or are compromised, they can mint unbacked wrapped assets unless a separate rate limit or pause mechanism intervenes.

This is the style the rest of the book will use.

2.8 Mapping Who Can Move, Freeze, Mint, Burn, Upgrade, or Censor Assets

2.8.1 Authority Mapping Is the Practical Output

An authority map is a structured answer to:

Who can do what to which asset, through which transition, under which assumptions?

This is not a diagramming exercise for its own sake. It is how reviewers avoid missing the key that can drain the protocol.

A useful authority map includes:

- Asset or state affected.
- Transition or function.
- Required authority.

- Authority holder.
- Delegation path.
- Delay or timelock.
- Revocation path.
- Observability.
- Failure impact.
- Recovery path.

2.8.2 The Core Authority Verbs

For each asset, ask who can perform these actions.

Verb	Meaning	Example risk
Move	Transfer or redirect the asset	Unauthorized transfer, malicious withdrawal
Approve	Delegate spending or operation rights	Infinite allowance, operator abuse
Mint	Create new claims	Inflation, unbacked wrapped assets
Burn	Destroy claims	User balance destruction, forced redemption
Freeze	Prevent movement or use	Censorship, trapped funds
Pause	Stop protocol transitions	Emergency control abuse, liveness failure
Upgrade	Change future rules	Malicious implementation, storage collision
Configure	Change parameters or dependencies	Unsafe collateral factor, bad oracle
Liquidate	Seize or sell collateral	Manipulated liquidation, griefing
Settle	Finalize claims or messages	Premature bridge release, wrong finality
Censor	Exclude or delay transitions	Broken exits, liquidation failure
Observe	Learn private or pending intent	MEV, order-flow leakage
Recover	Override normal flow after failure	Admin theft disguised as recovery

This list should feel a little uncomfortable. Good. It reveals how many ways “owning” an asset can be affected by actors other than the nominal owner.

2.8.3 Authority Matrix

An authority matrix turns the verb list into a review artifact. For a tokenized vault, a first pass might look like this:

Actor or component	Move	Mint/burn	Freeze/pause	Upgrade/configure	Censor/delay	Observe	Recover
User	Deposit, withdraw, transfer shares	Burn shares on with- drawal	No	No	No	Own transac- tion intent	Recover only through wal- let/key process
Approved spender	Move approved tokens	No	No	No	No	May infer user intent from approvals	No
Vault contract	Move assets according to rules	Mint/burn shares	Enforce pause state	No unless upgrade- able path exists	Can reject invalid exits	Emits events and exposes state	May include recovery hooks
Strategist	Move strategy allocation	No	No	Sometimes configure strategy parame- ters	Can delay yield move- ment or with- drawal liquidity	Sees strategy state	May unwind positions
Oracle/reporter	No direct move- ment	No	No	No	Can delay or distort valuation- dependent transi- tions	Observes/report prices	May trigger fallback modes

Actor or component	Move	Mint/burn	Freeze/pause	Upgrade/configure	Censor/delay	Observe	Recover
Guardian	Usually no move- ment	No	Pause selected transi- tions	Sometimes change emer- gency parame- ters	Can delay user exits if pause is broad	Observes incident signals	May trigger recovery mode
Governance/Maylock	May authorize treasury or strategy move- ment	May change mint/burn logic by upgrade	Can assign pausers	Upgrade, configure, grant roles	Can delay or execute proposals	Observes and publishes proposals	Can replace roles or implemen- tations
Token is- suer/custodian	May freeze or redeem underly- ing token	May mint/burn underly- ing token	Can blacklist or freeze	Can change issuer- side policies	Can block move- ment of underly- ing token	Observes compli- ance data	May recover or seize under issuer rules

The matrix does not prove the system is safe. It prevents hidden authority from staying hidden.

2.8.4 Example: ERC-20 Token Authority Map

Consider an ERC-20-like token with minting, burning, pause, and blacklist features.

Asset/state	Transition	Authority	Failure mode
User balance	<code>transfer</code>	Token holder	Holder key compromise, unsafe recipient assumptions
User balance	<code>transferFrom</code>	Approved spender	Infinite allowance, phishing, stale approval
Total supply	<code>mint</code>	Minter role	Inflation, unbacked issuance

Asset/state	Transition	Authority	Failure mode
User balance	<code>burnFrom</code>	Burner role or allowance	Unauthorized destruction
All transfers	<code>pause</code>	Pauser role	Frozen market, griefing, centralized control
Address usability	<code>blacklist</code>	Compliance/admin role	Censorship, mistaken freeze
Implementation	<code>upgradeTo</code>	Proxy admin/timelock	Malicious logic, storage corruption
Roles	<code>grantRole/revokeRole</code>	Role admin	Privilege escalation

Now ask:

- Who holds each role?
- Can roles be changed?
- Are role changes delayed?
- Are role holders multisigs, EOAs, contracts, or governance?
- Can users detect changes before they are harmed?
- Can the token be safely integrated by protocols that assume standard ERC-20 behavior?

The ERC-20 standard defines a basic interface, including delegated spending through approvals and allowances.¹⁶ It does not guarantee that every token has the same security properties.

2.8.5 Example: Vault Authority Map

Consider a yield vault.

Assets and claims:

- Underlying asset.
- Vault shares.
- Strategy positions.
- Pending withdrawals.
- Reward tokens.
- Admin roles.

Authorities:

¹⁶Fabian Vogelsteller and Vitalik Buterin, “ERC-20: Token Standard”, <https://eips.ethereum.org/EIPS/eip-20>.

Action	Possible authority	Security question
Deposit	Any user	Can deposits manipulate share price?
Withdraw	Share holder	Can withdrawals be blocked or underpaid?
Allocate funds	Strategist or vault logic	Can funds be sent to malicious strategy?
Report profit/loss	Strategy, keeper, or accountant	Can reports inflate share price?
Change strategy	Governance/admin	Is there a timelock and exit window?
Pause deposits	Guardian/admin	Can pause trap users or only reduce risk?
Pause withdrawals	Guardian/admin	Is user exit dependent on trusted discretion?
Upgrade vault	Proxy admin/governance	Can implementation steal assets?
Sweep tokens	Admin/recovery role	Can underlying assets be swept?

The user may “own” shares. But the value of those shares depends on accounting, strategy authority, withdrawal liveness, admin controls, oracle assumptions, and upgrade authority.

2.8.6 Example: Bridge Authority Map

Consider a bridge that locks tokens on chain A and mints wrapped tokens on chain B.

Asset/state	Authority	Boundary
Locked source tokens	Source bridge contract	Source-chain execution and finality
Wrapped destination supply	Destination bridge contract	Correct mint/burn accounting
Message validity	Validator set or light client	Signature threshold or proof verification
Message delivery	Relayer network	Liveness and censorship
Replay protection	Destination contract state	Nonce/message status correctness
Validator set updates	Governance/admin	Capture or key compromise
Emergency pause	Guardian/admin	Abuse vs incident response

Asset/state	Authority	Boundary
Upgrade path	Proxy admin/governance	Future rule changes

A wrapped token holder does not merely depend on the token contract. They depend on every authority in the bridge path.

2.8.7 Example: Governance Authority Map

Governance should be mapped like any other privileged system.

State/action	Authority	Questions
Proposal creation	Token threshold, proposer role	Can attackers cheaply create proposals?
Voting power	Token balances, delegation	Can votes be borrowed, bribed, or custodied?
Quorum	Governance parameters	Can low participation pass major changes?
Proposal outcome	Voting mechanism	Is voting private, delayed, snapshot-based, or flash-loan-resistant?
Execution	Timelock/executor	Who can execute after passing?
Cancellation	Guardian, proposer, admin	Can malicious proposals be stopped? Can legitimate proposals be censored?
Target calls	Governor/timelock	Can arbitrary calls seize assets or grant roles?
Governance upgrades	Governance/admin	Can the process change itself?

Governance is not separate from security. Governance is often the root authority over the security model.

2.8.8 Minimal Authority Mapping Template

Use this template when starting any review:

Asset or state:

Representation:

Where is it stored or recorded?

Normal holder:

Who appears to own or control it?

Transitions:

Which functions, instructions, messages, or processes affect it?

Authorities:

Who can call or authorize those transitions?

Delegation:

Can authority be delegated through allowances, operators, roles, signatures, or governance?

Administration:

Who can change the rules, roles, parameters, or implementation?

Ordering/finality:

Does the asset depend on inclusion, ordering, confirmations, settlement, or cross-domain finality?

Off-chain dependencies:

Which oracles, custodians, signers, frontends, relayers, indexers, legal agreements, or human processes matter?

Failure impact:

What can go wrong if each authority is malicious, compromised, unavailable, or mistaken?

User exit:

Can users observe danger and leave before damage occurs?

If the template feels tedious, use it anyway. Most severe bugs hide in the row someone skipped.

2.9 Worked Example: Mapping a Tokenized Vault

Suppose a protocol offers a vault:

Users deposit USDC.

The vault mints shares.

A strategist allocates USDC into yield strategies.

An oracle reports strategy value.

Users can withdraw USDC by burning shares.

Governance can upgrade the vault after a 48-hour timelock.

A guardian can pause deposits and withdrawals immediately.

The marketing version is:

Users own vault shares backed by USDC and strategies.

The security version begins with assets:

Asset	Representation
USDC deposits	Vault token balance
Vault shares	Share token balances
Strategy assets	External strategy accounting
Pending withdrawals	Vault queue state
Strategy value	Oracle-reported value
Governance power	Governance token/delegation state
Emergency authority	Guardian role
Upgrade authority	Timelock/governance/proxy admin

Then authority:

Action	Authority	Risk
Deposit	Any user	Share inflation, nonstandard token transfer
Withdraw	Share holder	Queue griefing, underpayment, stuck funds
Allocate to strategy	Strategist	Loss of funds, malicious strategy
Report value	Oracle/reporter	Inflated share price, bad redemptions
Pause deposits	Guardian	Blocks new exposure
Pause withdrawals	Guardian	Traps users
Upgrade vault	Governance/timelock	Rule replacement
Change oracle	Governance/admin	False valuation
Change strategist	Governance/admin	Malicious allocation

Now trust boundaries:

flowchart LR

```

User[User] --> Wallet[Wallet / frontend]
Wallet --> Vault[Vault contract]
Vault --> Token[USDC token]
```

```

Vault --> Strategy[Yield strategy]
Strategy --> Oracle[Oracle / reporter]
Governance[Governance / timelock] --> Vault
Guardian[Guardian] --> Vault
TokenIssuer[USDC issuer] --> Token

```

- USDC itself has issuer and blacklist assumptions.
- Strategy value depends on external strategy contracts and oracle/reporting.
- Governance depends on voting distribution and timelock execution.
- The guardian is trusted not to freeze withdrawals maliciously.
- Users depend on frontend/wallet correctness to understand approvals.
- The proxy admin path can change the vault's future transition rules.

The combined asset/control map is:

Asset or control surface	Representation	Transition	Authority	Failure impact
User deposit	USDC balance held by vault	deposit , withdraw	User, vault logic, USDC token rules	User funds stuck, frozen, or misaccounted
Vault shares	Share token balances	Mint on deposit, burn on withdrawal	Vault accounting logic	Dilution, unfair redemption, share inflation
Strategy allocation	External strategy position	Allocate, withdraw, report	Strategist/governance logic	Locked funds or illiquid withdrawals
Strategy value	Oracle or report state	Report/update valuation	Oracle/reporter	Overstated assets, unfair share price
Withdrawal queue	Vault queue state	Enqueue, process, cancel	User, vault, keeper/processor	Stuck withdrawals, ordering grief
Emergency pause	Pause flag	Pause/unpause	Guardian/governance	Trapped users or uncontrolled incident
Upgrade path	Proxy implementation/admin	Queue/execute upgrade	Governance/timelock admin	Complete rule replacement

Now invariants:

Shares should represent a proportional claim on vault assets.

Withdrawals should not pay more than the user's fair share.

Deposits should not dilute existing users unfairly.

Strategy accounting should not overstate withdrawable assets.

No privileged role should be able to seize assets without an explicit, documented trust assumption.

Users should be able to exit before delayed malicious upgrades execute, unless the system clearly discloses that they cannot.

This is the difference between describing a product and reviewing a system.

2.10 Practical Discipline: Replace Nouns With Verbs

When you see a noun, ask for the verb.

Noun	Better question
Owner	What can this owner do?
Admin	Which state transitions can this admin authorize?
Guardian	Can the guardian only pause, or can it seize/reconfigure?
Governance	Can it call arbitrary targets? Can it upgrade itself?
Oracle	Which transitions rely on its data?
Relayer	Is it needed for liveness, safety, or both?
Sequencer	Can it censor exits or reorder liquidations?
Custodian	Which assets or keys does it control?
Wallet	What can it cause the user to sign?
Bridge	Which domain's state is being trusted by which other domain?

Nouns make systems sound stable. Verbs reveal power.

2.11 Review Questions

1. Why is an on-chain asset better understood as an enforceable claim over state than as an object inside an address?
2. Give three examples of assets that are not simple token balances.
3. What is the difference between legal ownership, protocol authority, custody, and economic exposure?

4. Why are allowances, operator approvals, upgrade keys, oracle keys, and governance roles forms of authority?
5. Why can the words “owner” and “account owner” mislead reviewers across Ethereum, Solana, and Move-style systems?
6. What is a trust boundary, and why do bridges create stacked trust boundaries?
7. What information belongs in an authority map?
8. Give an example where a user owns an asset economically but cannot move it without relying on another actor.

2.12 Applied Exercises

2.12.1 Exercise 1: Authority Map a Token

Choose any token design with:

- Transfers.
- Allowances.
- Minting.
- Burning.
- Pausing.
- Upgradeability.

Create an authority map with:

- Asset/state affected.
- Transition.
- Required authority.
- Current authority holder.
- Whether authority is delayed, revocable, or observable.
- Worst-case failure if that authority is compromised.

Then answer: which authority is economically equivalent to control over the whole token?

2.12.2 Exercise 2: Trust Boundary and Custody Review

Analyze both scenarios:

1. A protocol uses an oracle price to decide whether accounts can borrow.
2. A user has 100 tokens on a centralized exchange; the exchange displays the balance but controls the on-chain wallet keys.

For each scenario, write:

- The asset or claim.
- The actor with protocol authority.
- The custody or reporting dependency.
- The trust-boundary statement.
- The failure impact if the trusted actor is wrong, compromised, unavailable, or dishonest.

2.12.3 Exercise 3: Bridge Claim Analysis

A wrapped token on chain B claims to represent locked tokens on chain A. The bridge uses a 7-of-10 validator signature threshold and can be upgraded by a 3-of-5 multisig.

Answer:

- What is the asset on chain B?
- What is the claimed backing on chain A?
- Who can mint on chain B?
- What trust boundary connects chain A to chain B?
- Which authority is stronger: the validator set or the upgrade multisig?
- What happens if 7 bridge validators collude?
- What happens if 3 upgrade multisig signers collude?

2.12.4 Exercise 4: Replace Nouns With Verbs

For each phrase, rewrite it as authority over transitions:

1. “The DAO controls the treasury.”
2. “The guardian protects users.”
3. “The protocol uses a trusted oracle.”
4. “The token is backed by reserves.”
5. “The sequencer runs the rollup.”
6. “The admin can configure markets.”

Example answer style:

“The DAO controls the treasury” becomes:

Token holders with enough voting power can pass a proposal that, after the timelock delay
 , causes the treasury contract to transfer assets or call target contracts.

2.13 Key Takeaways

- Assets are enforceable claims over state, not physical objects inside addresses.
- Authority is the ability to cause accepted state transitions that affect assets.
- Ownership, custody, control, legal title, and economic exposure are different categories.
- UTXOs, accounts, contract storage, Solana accounts, and Move resources encode ownership differently.
- Token balances are incomplete without allowances, operators, roles, upgrade paths, and external dependencies.
- Privileged roles, upgrade keys, oracle keys, validator keys, sequencer power, and governance votes are assets because they can affect value.
- Trust boundaries appear wherever a system relies on a claim, actor, or component it cannot fully verify locally.
- On-chain systems can still contain trust assumptions. Off-chain components can be safe if their authority and failure modes are constrained.
- Authority maps turn vague security claims into reviewable facts.
- When in doubt, replace nouns with verbs: what can this actor actually do?

2.14 Further Reading

- ethereum.org, “Ethereum Accounts”. Useful for distinguishing externally owned accounts, contract accounts, account fields, and wallets.
- Satoshi Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System”. See the transaction and double-spend discussion for ownership as signature chain plus agreed history.
- Fabian Vogelsteller and Vitalik Buterin, “ERC-20: Token Standard”. The canonical fungible token interface, including transfers, approvals, and allowances.
- William Entriken et al., “ERC-721: Non-Fungible Token Standard”. The NFT standard, including token ownership, approvals, and operator authority.
- Solana Foundation, “Accounts”. The core reference for Solana account structure, account ownership, and runtime modification rules.
- The Move Book, “Abilities”. Current reference for Move abilities, linear behavior, storage constraints, and resource-like asset modeling.
- OpenZeppelin, “Access Control”. Practical reference for ownership, roles, timelocks, and privileged smart contract actions.
- OWASP, “Threat Modeling”. General security framing for identifying threats, assumptions, mitigations, and validation.
- NIST SP 800-207, “Zero Trust Architecture”. Useful general framing for avoiding implicit trust and focusing on users, assets, and resources.

Chapter 3

Transactions and State Transitions

3.1 Purpose

Chapter 1 gave us the state-machine model. Chapter 2 taught us to identify assets and authority. Chapter 3 studies the object that usually carries authority into the state machine:

A transaction is a signed, metered, ordered request to execute one or more state transitions.

That definition is intentionally more careful than “a transaction sends money.” Some transactions transfer native assets. Some call contracts. Some deploy code. Some carry multiple instructions. Some vote. Some update validator sets. Some submit proofs. Some relay cross-chain messages. Some fail but still consume fees. Some leave no application state change but still affect liveness, ordering, fees, or user expectations.

Transactions are where user intent, cryptographic authorization, resource pricing, chain ordering, execution semantics, and application logic meet. That makes them one of the richest security surfaces in blockchain systems.

The central discipline of this chapter is:

Do not ask only what a transaction says.

Ask what state it can change, under whose authority, at what cost,
in which order, with what failure behavior, and with which observable result.

3.2 Learning Objectives

By the end of this chapter, you should be able to:

- Describe transaction anatomy across UTXO, account, instruction-based, object-based, and Cosmos-style systems.

- Explain how signatures, nonces, sequence numbers, recent blockhashes, object versions, and chain IDs limit replay.
- Explain fees, gas, compute budgets, fee markets, and resource exhaustion as security mechanisms, not just UX concerns.
- Interpret transaction results, receipts, logs, events, return data, status codes, and reverts without confusing them for application truth.
- Distinguish syntactic validity, mempool acceptance, block inclusion, execution success, finality, and protocol safety.
- Identify valid transactions that can still produce unsafe states.

3.3 The Core Model

A transaction has three layers:

Layer	Review question	Typical fields or evidence
Authorization	Who is asking for this transition, and how is authority proven?	Signatures, signers, account keys, witnesses, approvals, roles
Execution request	What operation, message, instruction, script, or call should run?	Calldata, script, instruction data, message type, target program/contract
Resource and ordering envelope	What determines whether, when, and how the request can be included?	Fees, gas, compute budget, nonce, sequence, blockhash, expiration, chain ID

Different chains package these layers differently.

Ethereum documentation describes transactions as cryptographically signed instructions from accounts that update network state.¹ Bitcoin’s developer guide describes transactions as composed of inputs and outputs, where each input spends a previous output and each output waits as a UTXO until later spent.² Solana documentation describes transactions as containing one or more instructions, account signatures, and a recent blockhash; all instructions in a transaction are processed together and failure of one instruction fails the entire transaction.³ Cosmos SDK documentation describes a lifecycle that moves from creation and signing, to mempool checks, to block inclusion, to deterministic execution and committed state changes.⁴

Those systems look different, but they all answer the same questions:

¹ethereum.org, “Transactions”, <https://ethereum.org/developers/docs/transactions/>.

²Bitcoin Developer Guide, “Transactions”, <https://developer.bitcoin.org/devguide/transactions.html>.

³Solana Foundation, “Transactions”, <https://solana.com/docs/core/transactions>.

⁴Cosmos SDK v0.47, “Transaction Lifecycle”, <https://docs.cosmos.network/sdk/v0.47/learn/beginner/tx-lifecycle>.

Who authorized the input?
 What state does it read?
 What state does it write?
 What resources does it consume?
 How is replay prevented?
 What happens if it fails?
 When is the result accepted?

This chapter is about those questions.

The lifecycle also matters. A transaction can be safe at one point and unsafe at another:

flowchart LR

```

  A[Constructed] --> B[Signed]
  B --> C[Submitted]
  C --> D[Propagated]
  D --> E[Pending]
  E --> F[Included]
  F --> G[Executed]
  G --> H[Confirmed]
  H --> I[Finalized / settled]

```

3.4 Transaction Anatomy Across Chain Models

3.4.1 Transaction Anatomy Is a Security Interface

A transaction is not only a data structure. It is the boundary between intent and state transition.

The fields inside a transaction determine:

- Who authorizes it.
- Which state objects or accounts it can touch.
- Which code path runs.
- How much value moves.
- How much resource the sender is willing to pay for.
- Whether the transaction can be replayed.
- Whether it expires.
- Whether it can be replaced.
- Whether it is valid only on a particular chain or domain.
- What data validators, wallets, relayers, builders, and applications can inspect.

This means transaction anatomy is security-critical. A missing field, ambiguous field, incorrectly signed field, or misunderstood field can become an exploit primitive.

3.4.2 UTXO Transactions: Consume Outputs, Create Outputs

Bitcoin-style transactions are built from inputs and outputs.

Each input references a previous output by transaction ID and output index. Each output assigns value to a spending condition. A wallet balance is a derived view over spendable unspent outputs, not a single mutable account balance.⁵

Abstractly:

Transaction:

```
version
inputs:
  previous_txid
  previous_output_index
  unlocking_data / witness
  sequence
outputs:
  amount
  locking_script
locktime
```

The state transition is:

```
remove consumed UTXOs
add newly created UTXOs
```

Validity depends on conditions such as:

- Referenced outputs exist and are unspent.
- Unlocking data satisfies each referenced output's spending condition.
- Input value is sufficient to cover output value.
- No output is spent twice.
- Locktime and sequence constraints are satisfied.
- Consensus and standardness rules are met.

Security consequences:

- There is no account nonce in the Ethereum sense.
- Replay is constrained because spending consumes specific outputs.
- Change outputs are normal outputs and can leak privacy or be mishandled.
- Fee is typically implicit: input value minus output value.
- Transaction malleability historically mattered because downstream transactions may reference transaction IDs.

⁵Bitcoin Developer Guide, "Transactions", <https://developer.bitcoin.org/devguide/transactions.html>.

- Script conditions define spending authority.

UTXO analysis starts with:

Which outputs are consumed?

Which new outputs are created?

Which spending conditions were satisfied?

Which value disappeared as fee?

Which future spending paths now exist?

3.4.3 Account Transactions: Mutate Account and Contract State

Account-based systems represent state differently. Ethereum transactions are initiated by externally owned accounts and can transfer ETH, deploy contracts, or call contract code.⁶

A modern Ethereum transaction may include:

```
from
to
nonce
value
input data / calldata
gas limit
max fee per gas
max priority fee per gas
chain-specific signature data
access list or transaction type fields
signature
```

Ethereum.org lists these core fields, including sender, recipient, signature, nonce, value, input data, gas limit, max priority fee, and max fee.⁷

The state transition may update:

- Sender nonce.
- Native ETH balances.
- Contract storage.
- Account code if a contract is created.
- Logs emitted during execution.
- Gas accounting and fee distribution.

In the simplest ETH transfer:

⁶ethereum.org, “Transactions”, <https://ethereum.org/developers/docs/transactions/>.

⁷ethereum.org, “Transactions”, <https://ethereum.org/developers/docs/transactions/>.

```

balances[sender] -= value + fee
balances[recipient] += value
nonce[sender] += 1
base_fee portion is burned
priority fee goes to block proposer

```

In a contract call:

```

nonce[sender] += 1
call target with calldata and value
execute EVM code
update storage, balances, logs, and gas accounting according to execution

```

Security consequences:

- A single to address can hide complex logic if it is a contract.
- Calldata may be opaque to users and wallets.
- A transaction that sends zero native value may still move tokens through contract logic.
- Nonces serialize transactions from an account.
- Failed execution can still consume fees.
- Logs may tell applications what happened, but logs are not the same as state.

Account-model analysis starts with:

```

Who signed?
Which nonce is consumed?
Which target is called?
What calldata is supplied?
What value is attached?
Which storage, balances, logs, and subcalls can change?
What happens on revert?

```

3.4.4 Instruction-Based Transactions: Multiple Operations, Shared Atomicity

Solana uses transactions containing one or more instructions. The transaction includes signatures from accounts that authorize changes and a recent blockhash. Solana’s docs emphasize that instructions in a transaction are processed together: if any instruction fails, the entire transaction fails and state changes are reverted; fees are still charged on failure.⁸

A Solana transaction contains, at a high level:

```

signatures
message:
  header

```

⁸Solana Foundation, “Transactions”, <https://solana.com/docs/core/transactions>.

```

account addresses
recent blockhash
compiled instructions:
  program id
  account indexes
  instruction data

```

Security consequences:

- Atomicity spans multiple instructions.
- The caller supplies the account list.
- Programs must verify that supplied accounts are the expected accounts.
- Signer and writable flags are part of the authority boundary.
- A recent blockhash limits transaction lifetime.
- Transaction size and compute limits are first-class constraints.
- Cross-program invocations can propagate privileges in ways reviewers must understand.

Solana analysis starts with:

```

Which instructions execute?
Which accounts are passed to each instruction?
Which accounts signed?
Which accounts are writable?
Which program owns each account?
Which account constraints are enforced by the program?
Which instruction fails if assumptions are false?

```

3.4.5 Object-Based Transactions: Inputs, Versions, and Effects

Object-centric systems such as Sui represent many assets and state components as objects. Sui documentation describes every update on Sui as happening through a transaction and distinguishes user-submitted programmable transaction blocks from system transactions submitted by validators.⁹

Sui transaction metadata includes fields such as sender address, gas input object, gas price, gas budget, epoch, transaction type, authenticator, and optional expiration.¹⁰

The object model changes security analysis:

- Objects are explicit inputs and outputs.
- Ownership and object versions matter.
- Parallel execution can be possible when transactions touch disjoint objects.

⁹Sui Foundation, “Transaction Overview”, <https://docs.sui.io/develop/transactions/txn-overview>.

¹⁰Sui Foundation, “Transaction Overview”, <https://docs.sui.io/develop/transactions/txn-overview>.

- Shared objects introduce contention and ordering concerns.
- Gas is paid through a gas object.
- Expiration can bound when a transaction may execute.

Sui-style analysis starts with:

Which objects are inputs?

Which objects are mutated, created, transferred, wrapped, or destroyed?

Which object versions are assumed?

Which objects are owned, shared, or immutable?

What is the gas object?

What are the effects?

3.4.6 Cosmos-Style Transactions: Messages Through an Application Pipeline

Cosmos SDK transactions usually contain one or more messages. The transaction passes through creation, signing, mempool checks, block proposal, execution, and commit.

The Cosmos SDK transaction lifecycle distinguishes:

- Transaction creation and signing.
- Gas and fee settings.
- `CheckTx` before mempool acceptance.
- Stateless and stateful checks.
- An ante handler for signatures, sequence numbers, fee deduction, and related operations.
- `DeliverTx` during block execution.
- Commit of state changes after consensus.¹¹

Security consequences:

- A transaction can pass initial checks and still fail later.
- Local mempool fee policies can differ from deterministic block execution rules.
- Sequence numbers and fees help prevent replay and spam.
- Multiple messages may execute under one transaction.
- Application modules define message handlers that cause state transitions.

Cosmos-style analysis starts with:

Which messages are inside the transaction?

Which module handles each message?

Which ante-handler checks occur?

¹¹Cosmos SDK v0.47, “Transaction Lifecycle”, <https://docs.cosmos.network/sdk/v0.47/learn/beginner/tx-lifecycle>.

Which account sequence is consumed?

Which fees are deducted?

Which checks happen in mempool only, and which happen during block execution?

3.4.7 Cross-Model Comparison

Model	Primary input shape	Replay boundary	Resource boundary	Common security focus
UTXO	Inputs consume outputs; outputs create new claims	Spent outputs	Fee from input-output difference, size/weight	Double spend, script conditions, change, pinning
Ethereum account	Signed account transaction with nonce, gas, calldata	Account nonce and chain/domain separation	Gas limit and fee fields	Calldata intent, nonce ordering, reverts, gas griefing
Solana	Signed transaction with account list, instructions, recent blockhash	Recent blockhash, durable nonce, account/signature checks	Compute units, size limits, fees	Account substitution, signer/writable flags, CPI
Sui/object	Transaction block over objects and gas object	Object versions, sender/authentication epoch/expiration	Gas budget, object limits	Object ownership, shared objects, effects
Cosmos SDK	Signed transaction with messages, fees, sequence	Account sequence, mempool cache, timeout height	Gas wanted, fees, min gas price	Ante handler, module message routing, deterministic deliver

No single anatomy is “the blockchain transaction model.” The right security model comes from the state representation and execution rules of the chain being reviewed.

For chapter-level review, keep the anatomy table compact:

Model	Fields the reviewer must locate	What proves authority?	What shows success?
Bitcoin/UTXO	Inputs, previous outputs, witnesses/scripts, output amounts, locktime, sequence	Satisfaction of each consumed output's spending condition	Transaction remains in canonical history; intended outputs exist and inputs are spent
Ethereum/account	from, to, nonce, value, calldata, gas/fee fields, chain ID, signature	Sender signature, contract authorization checks, approvals or permits	Receipt status, state changes, logs as evidence, sufficient settlement
Solana/instruction	Signatures, account list, writable/signer flags, recent blockhash, instructions	Required signatures and program-enforced account constraints	Transaction confirmation, account state changes, emitted logs if relevant
Object/resource-based	Sender/authenticator, input objects/resources, versions, gas object, effects, expiration	Ownership, capabilities, authenticators, module rules	Object effects, version changes, event evidence, settlement status

This table is deliberately shallow. Part III will handle each execution model in depth. Here the purpose is to learn what must be inspected before trusting a transaction outcome.

3.5 Signatures, Nonces, Sequence Numbers, and Replay Prevention

3.5.1 Signatures Authorize a Specific Message, Not a Vague Intention

A signature does not prove that a user understood what they were doing. It proves that some signing authority produced a valid cryptographic signature over a particular message under a particular scheme.

This distinction matters.

A user may think they are signing:

Swap 100 USDC for at least 0.04 ETH.

The wallet may actually present or submit:

```
approve(spender = attacker, amount = unlimited)
```

The cryptographic signature can be valid while the human intention was violated. Blockchain security must therefore separate:

```
cryptographic authorization
from
informed consent
from
application safety
```

Chapter 4 will go deeper into keys and signatures. For this chapter, remember:

A transaction signature authorizes exactly the bytes and domain that the signature scheme verifies.

Everything not bound into the signed message is a potential ambiguity.

3.5.2 Replay Is Reuse of Authority in the Wrong Context

A replay attack occurs when a valid authorization is reused where it should not be valid.

Replay can happen across:

- Time: the same transaction or message is accepted twice.
- Chain: a transaction valid on one chain is valid on another.
- Contract: a signature intended for one contract is accepted by another.
- Function: a signature intended for one operation authorizes another.
- User/account: a message is valid for an unintended signer or account.
- Bridge domain: a cross-chain message is executed on the wrong destination or more than once.
- Fork or clone: old signatures work after migration or redeployment.

Replay prevention is not one mechanism. It is a family of mechanisms that bind authority to a specific context and consume it when used.

3.5.3 Ethereum Nonces Serialize Account Transactions

Ethereum transactions include a nonce: a sequential counter for transactions from the sender account.¹²

The nonce serves several purposes:

- It makes two otherwise identical transactions distinct if their nonces differ.
- It prevents the same signed transaction from being included twice on the same chain.

¹²ethereum.org, “Transactions”, <https://ethereum.org/developers/docs/transactions/>.

- It orders transactions from one account.
- It allows replacement behavior in pending transaction pools.

But nonces also introduce operational risks:

- A low-fee transaction can block later higher-nonce transactions from the same account.
- A replacement transaction must satisfy node replacement rules.
- A signing system that mismanages nonces can create stuck or conflicting transactions.
- Parallel signing from the same account can produce nonce races.
- Transaction cancellation is usually replacement with a same-nonce transaction, not erasure of history.

Security questions:

Who assigns nonces?

Can two systems sign with the same nonce?

What happens if a transaction is stuck?

Can a malicious or compromised signer block future transactions?

Can an attacker force a protocol bot into nonce contention?

3.5.4 Chain IDs and Domain Separation Prevent Cross-Chain Replay

Ethereum’s EIP-155 introduced simple replay protection by including the chain ID in the transaction signing scheme after the Spurious Dragon fork.¹³ This prevents a transaction signed for one EIP-155-protected chain from being valid on another chain with a different chain ID.

The security principle is broader than Ethereum:

Every authorization should be bound to the domain where it is intended to be valid.

Domains may include:

- Chain ID.
- Contract address.
- Program ID.
- Verifying contract.
- Application name.
- Version.
- Source and destination chain.
- Message nonce.
- User account.
- Expiration.

¹³Vitalik Buterin, “EIP-155: Simple Replay Attack Protection”, <https://eips.ethereum.org/EIPS/eip-155>.

If a signature says only “approve 100 tokens” but not which chain, which contract, which token, which spender, which deadline, and which nonce, it may authorize more than intended.

Domain separation turns vague authority into scoped authority.

3.5.5 UTXO Replay Prevention Comes From Consumption

In a UTXO system, replay prevention is tied to spending specific outputs.

Once an output is spent in the canonical chain, another transaction trying to spend the same output conflicts. A transaction cannot be accepted twice because its inputs would no longer be unspent.

But UTXO systems still have replay concerns:

- Transactions can be replayed across chains after forks if rules and UTXOs remain compatible.
- Signatures may authorize different transaction variants depending on sighash modes.
- Unconfirmed transaction replacement and pinning can affect propagation and settlement.
- Child transactions may depend on parent transaction IDs or witness behavior.
- Time locks and sequence values affect when spending is valid.

The main lesson is:

UTXO replay prevention is based on consuming unique state, not incrementing an account counter.

3.5.6 Solana Recent Blockhashes and Durable Nonces

Solana transactions include a recent blockhash. Solana documentation states that a transaction’s recent blockhash is valid for 150 slots.¹⁴ This creates a time-bounded replay window.

Security consequences:

- Old transactions expire and cannot be included forever.
- Offline signing needs special handling.
- Users and bots must manage expiration and resubmission.
- A transaction can fail because it is too old, even if signatures and instruction data are otherwise correct.

Solana also supports durable nonces for offline signing workflows. Durable nonces solve a real operational problem but introduce their own lifecycle: nonce account creation, nonce advancement, authority, and failure behavior.

Ask:

¹⁴Solana Foundation, “Transactions”, <https://solana.com/docs/core/transactions>.

Is replay bounded by blockhash expiry, durable nonce state, or both?
 Who controls the nonce authority?
 Can old signed transactions remain valid longer than expected?
 Can a nonce account be advanced or consumed by an unexpected party?

3.5.7 Cosmos Sequence Numbers and Timeout Heights

Cosmos SDK documentation describes account sequence numbers as part of replay prevention and notes that fees and sequence counters help distinguish replayed transactions from identical but valid transactions.¹⁵

Cosmos transactions can also use timeout heights, limiting commitment after a certain block height.¹⁶

Security consequences:

- Sequence numbers prevent repeated acceptance of the same account authorization.
- Timeout height limits stale transaction execution.
- Fee checks affect spam resistance and mempool admission.
- A transaction may pass local checks but later fail during delivery.

Sequence numbers are not merely bookkeeping. They are authority consumption.

3.5.8 Object Versions and Expiration

Object-based systems can use object versions, owned object references, epochs, and expiration rules to prevent stale or conflicting transitions.

Sui documentation describes a transaction flow where objects are inputs and outputs, and transaction metadata includes epoch and optional expiration.¹⁷

Security consequences:

- A transaction that uses an old object version may no longer be valid.
- Shared objects require consensus and ordering.
- Owned objects may allow parallel execution when independent.
- Expiration prevents delayed execution after context changes.

Object models make some conflicts explicit:

This transaction is not just from Alice.
 It consumes or mutates these specific object versions.

¹⁵Cosmos SDK v0.47, “Transaction Lifecycle”, <https://docs.cosmos.network/sdk/v0.47/learn/beginner/tx-lifecycle>.

¹⁶Cosmos SDK v0.47, “Transaction Lifecycle”, <https://docs.cosmos.network/sdk/v0.47/learn/beginner/tx-lifecycle>.

¹⁷Sui Foundation, “Transaction Overview”, <https://docs.sui.io/develop/transactions/txn-overview>.

3.5.9 Replay Protection Must Match the Asset

Replay protection is only correct if it protects the asset or authority being consumed.

Examples:

- A transaction nonce protects Ethereum account-level transaction replay.
- A permit nonce protects token approval signature replay.
- A bridge message nonce protects cross-chain message replay.
- A withdrawal nullifier protects double-claiming.
- A UTXO input protects double-spending a specific output.
- A governance proposal ID prevents accidental duplicate execution only if execution status is tracked.
- A rollup output root claim needs finality, proof, and claim-status tracking.

Do not say “we have a nonce” as if that settles the matter.

Ask:

What authority is the nonce consuming?

Is the nonce scoped to the correct signer, chain, contract, and operation?

Can the nonce be skipped, reused, reset, or shared across domains?

Is there an expiration?

Can the same effect occur through another path?

3.6 Fees, Gas, Compute Budgets, and Resource Limits

3.6.1 Resource Pricing Is Part of Consensus Security

Blockchains cannot allow unbounded computation, storage growth, signature verification, or network bandwidth consumption. Public systems need resource limits because any participant can submit inputs.

Fees and metering serve several purposes:

- Prevent unlimited computation.
- Allocate scarce block space.
- Compensate or reward block producers.
- Discourage spam.
- Price storage, execution, and signature verification.
- Create ordering incentives.
- Bound denial-of-service impact.

Fees are not only economics. They are a security control.

If execution were free and unbounded, attackers could force every node to do arbitrary work. A blockchain cannot rely on “users will not submit expensive nonsense.” Permissionless execution means attackers will submit whatever the rules allow if it benefits them or harms others.

3.6.2 Ethereum Gas

Ethereum gas measures the computational and storage work required to execute transactions. Ethereum.org describes gas as the fee required to conduct transactions or execute contracts, with each transaction consuming gas units and fees depending on gas used and price per gas.¹⁸

The EIP-1559 transaction fee market introduced a base fee that is burned and a priority fee paid to validators, with transactions specifying max fee and max priority fee.¹⁹ Ethereum.org’s transaction page describes the `gasLimit`, `maxPriorityFeePerGas`, and `maxFeePerGas` fields.²⁰

Security consequences:

- If the gas limit is too low, execution can fail.
- If execution runs out of gas, state changes revert but gas is consumed according to protocol rules.
- If fees are too low, the transaction may remain pending or be ignored.
- Complex loops over user-controlled state can become denial-of-service vulnerabilities.
- Attackers can grief systems by forcing expensive paths, even when they cannot steal funds.
- Protocols that assume an operation will always fit within block gas limits can fail as state grows.
- Gas costs can change across forks or differ across chains, affecting brittle assumptions.

Gas analysis asks:

Can this transition become too expensive to execute?

Can an attacker increase the cost for other users?

Can a required recovery or withdrawal path run out of gas?

Does failure consume fees without producing the intended state?

Can a low-level call fail because of gas forwarding assumptions?

3.6.3 Solana Compute Budgets and Fees

Solana charges a base fee per signature and optional prioritization fees. Solana’s fee documentation describes a base fee and an optional prioritization fee, with the prioritization fee increasing the likelihood that the current leader schedules the transaction ahead of competitors.²¹

¹⁸ethereum.org, “Gas and Fees”, <https://ethereum.org/developers/docs/gas/>.

¹⁹Vitalik Buterin et al., “EIP-1559: Fee Market Change for ETH 1.0 Chain”, <https://eips.ethereum.org/EIPS/eip-1559>.

²⁰ethereum.org, “Transactions”, <https://ethereum.org/developers/docs/transactions/>.

²¹Solana Foundation, “Fees”, <https://solana.com/docs/core/fees>.

Solana also has compute unit limits. Its documentation lists default and maximum compute-unit limits and transaction size limits.²² The transaction page lists a 1,232-byte size limit, 64-byte Ed25519 signatures, and blockhash expiry.²³

Security consequences:

- Transaction size limits constrain how many accounts and instructions can be included.
- Compute limits constrain program complexity.
- Account locking and writable accounts affect parallelism and contention.
- Prioritization fees affect scheduling.
- Failed transactions can still pay fees.
- Bots can create congestion through failed or competing transactions.

Solana resource analysis asks:

Can the transaction fit within size and account limits?

Can the program fit within compute limits under worst-case state?

Can attackers lock accounts or create contention?

Can they force expensive CPIs?

Can a required transaction fail because too many accounts are needed?

3.6.4 Cosmos Gas and Local Mempool Fee Policy

Cosmos SDK documentation distinguishes user-specified gas, gas prices, fees, local minimum gas prices, mempool checks, and deterministic delivery.²⁴

Important nuance: local mempool admission can depend on each node’s `min-gas-prices`, but deterministic block execution cannot depend on local settings. Cosmos documentation notes that `DeliverTx` does not compare gas prices to local `min-gas-prices` because differing local values would yield nondeterministic results.²⁵

Security consequences:

- A transaction may be rejected by one node’s mempool and accepted by another.
- Fee policy affects propagation and inclusion.
- Gas estimation can undercount worst-case execution.
- Ante-handler logic affects signatures, sequences, and fee deduction.
- Block gas limits can prevent large batches or emergency operations.

²²Solana Foundation, “Fees”, <https://solana.com/docs/core/fees>.

²³Solana Foundation, “Transactions”, <https://solana.com/docs/core/transactions>.

²⁴Cosmos SDK v0.47, “Transaction Lifecycle”, <https://docs.cosmos.network/sdk/v0.47/learn/beginner/tx-lifecycle>.

²⁵Cosmos SDK v0.47, “Transaction Lifecycle”, <https://docs.cosmos.network/sdk/v0.47/learn/beginner/tx-lifecycle>.

The security lesson is general:

Mempool policy may be local.

Consensus execution must be deterministic.

Do not confuse the two.

3.6.5 UTXO Fees and Transaction Pinning

In Bitcoin-style systems, fees are commonly implicit: the difference between total input value and total output value.

`fee = sum(inputs) - sum(outputs)`

Fee policy affects:

- Whether a transaction propagates.
- Whether miners include it.
- Whether it can be replaced.
- Whether child-pays-for-parent can rescue it.
- Whether an attacker can pin a transaction and prevent timely confirmation.

UTXO fee analysis asks:

What fee does this transaction pay?

Can it be replaced?

Can a child transaction increase its effective fee?

Can an attacker create a low-fee dependency?

Can time-sensitive protocols fail if confirmation is delayed?

This will matter later for payment channels, bridges, vault exits, and liquidation systems that depend on timely confirmation.

3.6.6 Resource Limits Create Liveness Risks

Resource limits do not only stop attackers. They can stop honest users.

Examples:

- A withdrawal loop over all users becomes impossible after the user set grows.
- A liquidation path needs too much gas during market stress.
- A bridge proof exceeds transaction size limits.
- A governance proposal includes too many actions to execute atomically.
- A Solana instruction needs more accounts than the transaction can carry.
- A Cosmos module message consumes more gas than expected under extreme state.

- A rollup escape hatch is too expensive during L1 congestion.

The invariant may still be true:

Every user has a claim.

But liveness may fail:

No user can practically execute the transition that realizes the claim.

Security reviews must test not only “does the code work on an example” but:

Does the required transition remain executable under worst-case state, market stress, fee spikes, and adversarial input?

3.6.7 Fees Shape Adversarial Strategy

Attackers are cost-sensitive. Fees determine which attacks are profitable or feasible.

Examples:

- A spam attack may be uneconomic if fees are high enough.
- A sandwich attack may be profitable only if expected extraction exceeds gas and priority fees.
- A governance griefing attack may depend on cheap proposal creation.
- A liquidation race may be won by paying higher priority fees.
- A bridge relayer may stop relaying if fees exceed compensation.
- A keeper network may fail if executing maintenance tasks is unprofitable.

Security analysis should include:

What does the attacker pay?

What can the attacker gain?

Who bears failed transaction costs?

Who is reimbursed?

Who has incentives to include, relay, execute, or ignore the transaction?

Fees are part of the threat model.

3.7 Execution Results: Receipts, Logs, Events, and Reverts

3.7.1 Inclusion Is Not Execution Success

A transaction can exist in several states:

- Constructed but unsigned.

- Signed but not broadcast.
- Broadcast but not seen by most nodes.
- Accepted into some mempools.
- Included in a block.
- Executed successfully.
- Executed and reverted or failed.
- Reorged out.
- Finalized or settled.
- Reflected correctly by an RPC, indexer, or UI.

These are different facts.

Do not say “the transaction went through” unless you specify which fact you mean.

Claim	What it means	What it does not prove
Broadcast	Some endpoint accepted the transaction for propagation	Inclusion, execution, or finality
Included	A block contains the transaction	Successful execution or durable settlement
Receipt success	Top-level execution did not revert	Economic safety or user intent
Event emitted	Code emitted a log	State changed as the event name suggests
State updated	Canonical state reflects the change	The change was safe or final enough for downstream use
Finalized/settled	Reversion is outside the accepted risk model	Off-chain interpretation, indexer correctness, or user intent

For example:

The transaction was broadcast.
 The transaction was included.
 The transaction succeeded with status 1.
 The transaction emitted event X.
 The transaction updated storage slot Y.
 The transaction is finalized.
 The transaction was observed by this indexer.

Precision matters because attackers exploit ambiguity.

3.7.2 Receipts Summarize Execution Outcomes

Ethereum transaction receipts include information such as block hash, block number, gas used, logs, contract address for contract creation, transaction index, type, and either a status or a pre-Byzantium state root. Ethereum JSON-RPC documentation specifies that `status` is 1 for success or 0 for failure, and receipts include logs and gas used fields.²⁶

Receipts are useful because they summarize execution without requiring every observer to replay the transaction immediately.

But receipts are not a substitute for understanding state:

- A successful status means execution did not revert at the top level.
- It does not mean the business operation was economically safe.
- Logs may be emitted by contracts, but they do not prove that an off-chain interpretation is correct.
- A receipt can disappear from canonical history if its block is reorged out before finality.
- Indexers can misinterpret receipts or miss reorgs.

Receipt analysis asks:

Was the transaction included?

Did it succeed or fail?

How much gas was used?

Which logs were emitted?

Which contract emitted each log?

Which state change do the logs claim to represent?

Is the block final enough for this application?

3.7.3 Logs and Events Are Communication, Not Authority

Solidity documentation describes events as an abstraction over the EVM logging functionality. Event arguments are stored in transaction logs associated with the emitting contract address; applications can subscribe through client RPC interfaces.²⁷

Events are extremely useful for:

- Indexers.
- UIs.
- Analytics.
- Off-chain automation.
- Bridges and message watchers.

²⁶ethereum.org, “JSON-RPC API”, <https://ethereum.org/developers/docs/apis/json-rpc/>.

²⁷Solidity Documentation, “Events”, <https://docs.soliditylang.org/en/latest/contracts.html#events>.

- Monitoring.
- Human debugging.

But events are not state accessible to contracts that emitted them, and contracts cannot generally use their own past logs as authoritative state. Solidity documentation also notes that event data must be interpreted with knowledge of the event type, and anonymous events can complicate interpretation.²⁸

Security consequences:

- An event can be emitted without the expected state change if the code is wrong.
- A state change can occur without an expected event if the code omits it.
- An indexer can treat an event as truth and become inconsistent with state.
- A bridge that watches events must account for finality and reorgs.
- Anonymous or misleading events can confuse tooling.
- Off-chain systems must verify the emitting contract and event semantics.

Rule:

Events are evidence.

State is authority.

Sometimes a protocol intentionally uses logs as part of a cross-domain commitment path. Then the security model must say exactly which log, from which contract, in which finalized block, under which proof or watcher assumption, is accepted.

Evidence type	Produced by	Useful for	Security warning
State	Protocol/client execution	Authority for future validity	Must be read from the correct canonical state
Receipt	Client execution result	Execution status, gas, logs, block context	Can be reorged out or misread
Event/log	Contract or program execution	Off-chain discovery and monitoring	Communicates; does not define authority by itself
Indexer view	Off-chain derived database	Search, UI, analytics, automation	Can lag, omit reorgs, or interpret incorrectly
Simulation	Local execution against assumed state	Preflight and UX warning	Not a promise about eventual inclusion or ordering

²⁸Solidity Documentation, “Events”, <https://docs.soliditylang.org/en/latest/contracts.html#events>.

3.7.4 Return Data and Revert Data Matter

Execution can produce return data or revert data.

In EVM systems:

- A call may return bytes.
- A call may revert with error data.
- A low-level call may report success/failure without automatically bubbling the revert.
- A token may return `false`, return no value, revert, or behave nonstandardly.
- A wrapper library may interpret return behavior differently.

Solidity documentation states that `revert` and failed `require` calls revert changes in the current call and provide error data.²⁹

Security consequences:

- Ignoring return values can make a failed operation appear successful.
- Assuming all tokens return booleans can break integrations.
- Catching reverts can create partial-success semantics.
- Low-level calls can hide failure if the result is not checked.
- A transaction can succeed while an internal operation failed and was ignored.

Ask:

Which call's success matters?

Is the return value checked?

Are errors bubbled, swallowed, or transformed?

Can partial progress remain after a failure?

Does the chain's atomicity rule cover this whole operation or only part of it?

3.7.5 Atomicity Is Model-Specific

People often say blockchain transactions are atomic. That is sometimes true, but only at the correct boundary.

Examples:

- In Ethereum, if a top-level transaction reverts, state changes from that execution are reverted, but gas is still consumed. Internal calls can fail and be handled without reverting the top-level transaction.

²⁹Solidity Documentation, “Error handling: Assert, Require, Revert and Exceptions”, <https://docs.soliditylang.org/en/latest/control-structures.html#error-handling-assert-require-revert-and-exceptions>.

- In Solana, if any instruction in a transaction fails, the entire transaction fails and state changes are reverted, while fees are still charged.³⁰
- In Cosmos SDK, failed state changes during delivery are reverted, but fee and ante-handler behavior must be understood in context.³¹
- In Bitcoin, a transaction either becomes part of canonical history or does not, but protocols built from multiple transactions do not get automatic atomicity across those transactions.
- Cross-chain operations are almost never atomic in the same sense as a single-chain transaction.

Atomicity analysis asks:

What exactly reverts?

What does not revert?

Are fees charged?

Are logs preserved?

Are nonce or sequence effects preserved?

Can internal failures be caught?

Does a multi-step protocol assume atomicity across multiple transactions or chains?

3.7.6 Simulation Is Not Execution

Wallets, bots, keepers, searchers, and protocols often simulate transactions before submitting them.

Simulation is useful, but it is not the same as execution because:

- The state may change before inclusion.
- The transaction may be ordered differently.
- The block context may differ.
- The account nonce may change.
- Gas prices may change.
- Oracle prices may update.
- Another transaction may consume the same opportunity.
- A different node or RPC may simulate against a different state view.
- The transaction may land on another fork or not land at all.

Simulation answers:

What would happen if this transaction ran now under this assumed state and context?

It does not answer:

What will happen when this transaction is actually included?

This gap is central to MEV, liquidations, wallet UX, and transaction risk.

³⁰Solana Foundation, “Transactions”, <https://solana.com/docs/core/transactions>.

³¹Cosmos SDK v0.47, “Transaction Lifecycle”, <https://docs.cosmos.network/sdk/v0.47/learn/beginner/tx-lifecycle>.

3.8 When a Valid Transaction Still Produces an Unsafe State

3.8.1 Validity Is Layered

A transaction can pass many validity layers:

```
well-encoded
properly signed
nonce or sequence correct
fees sufficient
accepted by a node
included in a block
executed without revert
receipt status success
finalized
```

And still be unsafe.

Why? Because these layers prove that the transaction satisfied explicit machine rules. They do not prove that the protocol encoded the right economic, authorization, or liveness properties.

Examples:

- A lending market accepts a borrow against a manipulated oracle price.
- A vault accepts a deposit that mints zero shares because of rounding.
- A bridge accepts a message before the source event is final.
- A governance proposal executes malicious calls after satisfying quorum.
- A user signs an unlimited approval through a malicious frontend.
- A liquidation succeeds because the transaction was ordered after a temporary price update.
- A keeper transaction succeeds but pays an attacker-controlled address because configuration was stale.

The transaction is not invalid. The protocol design is insufficient.

3.8.2 Successful Execution Can Encode a Failed Assumption

A successful transaction may mean:

```
The code path completed.
```

It does not necessarily mean:

```
The oracle price was fair.
The user understood the signature.
The admin action was legitimate.
The bridge source state was final.
```


The governance outcome was socially acceptable.
 The vault shares were fairly priced.
 The recipient can use the received asset.
 The protocol remains solvent.

A receipt status can tell you whether the execution reverted. It cannot tell you whether the transition preserved intent.

Security review must therefore recover the implied assumptions behind a transaction:

This transition is safe if:

```
price is fresh and manipulation-resistant
signer intended this operation
nonce is unique and domain-scoped
output is final enough
role holder is trusted or constrained
resource limits leave the exit path live
```

If the “if” clause is false, successful execution can be the exploit path.

3.8.3 Unsafe State From Correct Ordering Rules

Ordering is part of transaction semantics.

Two valid transactions can produce different outcomes depending on order:

```
tx_A: update oracle price
tx_B: borrow against collateral
```

If tx_B executes before tx_A, the borrower sees the old price. If tx_B executes after tx_A, the borrower sees the new price. Both orderings may be valid under chain rules. Only one may be safe under the application’s assumptions.

Ordering risk appears in:

- Front-running.
- Back-running.
- Sandwiching.
- Liquidations.
- Oracle updates.
- NFT mints.
- Governance proposal execution.
- Bridge message delivery.
- Withdrawals from queues.
- Auctions.

Do not ask only:

Can this transaction execute?

Ask:

What can execute before it?

What can execute after it?

Who can observe it while pending?

Who can choose or influence ordering?

What state assumptions become false if ordering changes?

3.8.4 Unsafe State From Partial Mental Models

Many transaction bugs come from developers assuming the transaction has only one effect.

Example:

```
transfer token from user to vault  
mint shares to user
```

The developer thinks there are two effects. A real transaction might include:

- Token hook calls.
- Fee-on-transfer behavior.
- ERC-777-style callbacks.
- Reentrant calls.
- Oracle updates.
- Share price rounding.
- Event emission.
- Internal accounting.
- External strategy deposit.
- Gas griefing.
- Failure behavior from a nonstandard token.

Security review expands the mental model until every relevant effect is visible.

3.8.5 Unsafe State From Stale Context

A transaction is constructed at one time and executed later.

Between construction and execution:

- Prices can move.
- Nonces can change.
- Allowances can change.

- Blocks can advance.
- Governance proposals can queue.
- Liquidation status can change.
- A pool can be manipulated.
- A target contract can be upgraded.
- A bridge source event can be reorged.
- A Solana recent blockhash can expire.
- A Sui epoch can advance beyond expiration.

A transaction that was safe when signed may be unsafe when executed.

Defenses include:

- Slippage bounds.
- Deadlines.
- Nonces.
- Domain separation.
- Expected state checks.
- Minimum output amounts.
- Freshness checks.
- Finality delays.
- Commit-reveal mechanisms.
- Explicit target and version binding.

Every transaction that depends on mutable context should say:

`Execute only if these assumptions are still true.`

3.8.6 Unsafe State From Successful Failure

Sometimes attackers want a transaction to fail.

Examples:

- Griefing by causing another user's transaction to revert.
- Filling blocks or queues with failing transactions.
- Forcing a keeper to pay fees repeatedly.
- Causing a liquidation to fail so bad debt grows.
- Making a withdrawal batch fail because one recipient reverts.
- Locking accounts or objects so competitors cannot execute.
- Triggering expensive failure paths.

Failure is an outcome. It can be profitable.

Security review should ask:

Who pays when this fails?

Who benefits when it fails?

Can one failing operation block a batch?

Can failure leave state partially updated?

Can repeated failure exhaust resources?

3.8.7 Transaction Safety Checklist

For any important transaction path, review:

Authorization:

Who signed or authorized?

Is the signature scoped to the correct chain, contract, function, account, nonce, and deadline?

State:

Which state is read?

Which state is written?

Which assumed state can change before inclusion?

Ordering:

Can the transaction be observed, copied, frontrun, backrun, delayed, or censored?

Resources:

What gas, fees, compute units, size limits, or account limits apply?

Can the transition remain executable under worst-case state?

Failure:

What happens if execution reverts or partially fails?

Who pays?

Are nonce, fee, log, or queue effects preserved?

Results:

Which receipt, event, log, or state update proves success?

Is the application relying on logs instead of state?

Finality:

When is the result safe to depend on?

What happens if the block or source event is reorged?

Intent:

What did the user or protocol intend?

Which invariant should the transaction preserve?

If this checklist feels long, that is because transactions are where many separate assumptions collide.

3.9 Worked Example: A Swap Transaction

Consider a user signing a transaction through a decentralized exchange frontend:

```
swapExactTokensForTokens(  
  tokenIn = USDC,  
  tokenOut = WETH,  
  amountIn = 10,000 USDC,  
  minAmountOut = 3.8 WETH,  
  path = [USDC, WETH],  
  recipient = user,  
  deadline = now + 10 minutes  
)
```

The frontend says:

Swap 10,000 USDC for at least 3.8 WETH.

A transaction review asks:

Authorization:

- Did the user sign the swap transaction?
- Did the router already have allowance?
- Was there a separate approval transaction?

State:

- Which pool reserves determine the price?
- Is the token standard or nonstandard?
- Does either token take fees on transfer?

Ordering:

- Can the transaction be sandwiched?
- Can a searcher move the pool price before and after the swap?

Resources:

- Is gas limit sufficient?
- Can the transaction be stuck because fee settings are too low?

Failure:

- If output is below minAmountOut, does it revert?
- If it reverts, what fees are consumed?
- Is the approval still outstanding?

Results:

Did the transaction status succeed?
 Did balances change as expected?
 Are events consistent with state?

Finality:

Is the block final enough for the UI or downstream protocol to rely on?

Intent:

Did the user intend this exact router, path, recipient, amount, and deadline?

The most important security control in this transaction is not that the user signed it. The signature only proves authorization. The safety controls are the bound on `minAmountOut`, the deadline, the target contract, the token assumptions, and the user's ability to understand what they are signing.

The same transaction as a trace:

Step	Evidence	Security question
User signs	Signature over router call	Did the signed payload bind the intended router, path, recipient, amount, deadline, and chain?
Transaction waits	Mempool/private route observation	Who can see it, copy it, delay it, or order around it?
Router executes	Pool reserves and token behavior	Are price, slippage, transfer fees, and callbacks within assumptions?
Receipt appears	Status, gas used, emitted logs	Did execution succeed, and do logs match actual state changes?
Balances change	Token balances and pool reserves	Was the user paid at least <code>minAmountOut</code> , and did allowances remain risky?
Block settles	Confirmation/finality signal	Is the result durable enough for the UI or downstream protocol?

Now consider a malicious variant:

```
approve(attacker, type(uint256).max)
```

This transaction may:

- Be well-encoded.
- Be signed by the user.
- Pay enough gas.
- Be included in a valid block.
- Execute successfully.
- Emit an Approval event.

It is still unsafe for the user. The failure is not signature validation. The failure is that the user-side authority was tricked into authorizing a harmful state transition.

3.10 Practical Discipline: Transaction Review Notes

When reviewing a transaction path, write notes in this format:

Transaction:

 Name or entry point.

Intent:

 What the user/protocol thinks this does.

Signed authority:

 Who signs, or which account/role authorizes.

Replay scope:

 Nonce, sequence, object version, blockhash, chain ID, domain, deadline.

State touched:

 Reads and writes.

External assumptions:

 Oracles, token behavior, bridge finality, keeper behavior, frontend correctness.

Resource assumptions:

 Gas, compute, fees, size, account count, block limits.

Ordering assumptions:

 What must not happen before/after.

Failure behavior:

 What reverts, what persists, who pays.

Observable result:

 Receipt status, logs, state changes, confirmations/finality.

Invariant:

What must remain true afterward.

This format forces transaction analysis to connect mechanics to security properties.

3.11 Review Questions

1. What is the difference between a transaction, an instruction, a message, and a state transition?
2. In a UTXO model, what state is consumed and what state is created?
3. Why can a zero-ETH Ethereum transaction still move valuable assets?
4. Why do Solana account lists and signer/writable flags matter for security?
5. What does a transaction signature prove, and what does it not prove?
6. How do nonces, sequence numbers, blockhashes, chain IDs, and deadlines constrain replay?
7. What is the difference between inclusion, execution success, and settlement?
8. Why can events, receipts, and simulations mislead off-chain systems if treated as authority?

3.12 Applied Exercises

3.12.1 Exercise 1: Transaction Anatomy and Evidence

Compare three transaction models:

- Bitcoin transaction spending a UTXO.
- Ethereum transaction calling an ERC-20 transfer.
- Solana transaction with two instructions.

For each, identify:

- Authorization.
- Replay-prevention mechanism.
- State consumed or written.
- Resource limits or fees.
- Failure behavior.
- Evidence an observer should check to confirm success.

Then choose one real or hypothetical transaction receipt/result and identify:

- Signer or authority.
- Replay-prevention mechanism.

- State touched.
- Logs or events.
- Revert or failure behavior.
- Settlement assumption.
- One way the transaction could be valid but unsafe.

3.12.2 Exercise 2: Replay Scope

A protocol accepts off-chain signatures authorizing withdrawals:

```
withdraw(user, amount, signature)
```

The signed message includes only:

```
user  
amount
```

Answer:

- Which replay attacks are possible?
- Which fields should be added to the signed message?
- Which state should be updated when the signature is consumed?
- Should the signature expire?
- Should the signature be valid across contract upgrades?

3.12.3 Exercise 3: Event vs State

A bridge watcher sees this event:

```
Deposit(address indexed user, uint256 amount, uint256 nonce)
```

The watcher mints wrapped tokens on another chain.

Answer:

- What must the watcher verify besides the event data?
- Why is the emitting contract address important?
- What finality assumption is required?
- What replay-protection state should exist on the destination chain?
- What happens if the source block is reorged?

3.12.4 Exercise 4: Resource Limits and Valid-But-Unsafe Outcomes

A protocol has a function:

```
processAllWithdrawals()
```

It loops over every pending withdrawal and pays each recipient.

Answer:

- What resource limit can break this function?
- How can an attacker make the problem worse?
- Is this a safety failure, liveness failure, or both?
- What design patterns would reduce the risk?

Then explain why each transaction may be valid and still unsafe:

1. A user signs an unlimited token approval to a malicious spender.
2. A lending market accepts a borrow after a one-block oracle manipulation.
3. A governance proposal executes after satisfying quorum, but transfers the treasury to the proposer.
4. A bridge releases funds after seeing a source-chain event with too few confirmations.
5. A liquidation transaction succeeds because it was ordered immediately after a temporary price update.

3.13 Key Takeaways

- Transactions are signed, metered, ordered requests to execute state transitions.
- Transaction anatomy is a security interface: every field determines authority, replay scope, resource use, or execution context.
- UTXO, account, instruction-based, object-based, and Cosmos-style systems package transactions differently.
- Signatures prove authorization over specific bytes and domains. They do not prove user understanding or protocol safety.
- Replay prevention must be scoped to the authority and asset being consumed.
- Fees, gas, compute budgets, size limits, and block limits are security mechanisms.
- Inclusion, execution success, receipt status, event emission, and finality are different facts.
- Logs and events are evidence for observers, not a substitute for state.
- Simulation is useful but not execution.
- A transaction can be valid, successful, finalized, and still unsafe if the protocol encoded the wrong assumptions.

3.14 Further Reading

- ethereum.org, “Transactions”. Practical overview of Ethereum transaction fields, signing, call-data, gas fields, and lifecycle.

- ethereum.org, “Gas and Fees”. Technical overview of Ethereum gas, gas limits, base fee, priority fee, and transaction costs.
- Vitalik Buterin, “EIP-155: Simple Replay Attack Protection”. The Ethereum replay-protection change that binds transaction signing to chain ID.
- Vitalik Buterin et al., “EIP-1559: Fee Market Change for ETH 1.0 Chain”. The fee-market change introducing base fee mechanics and dynamic fee transactions.
- Bitcoin Developer Guide, “Transactions”. Detailed explanation of Bitcoin transaction inputs, outputs, scripts, and validation.
- Solana Foundation, “Transactions”. Reference for Solana transaction structure, signatures, atomicity, blockhash expiry, and limits.
- Solana Foundation, “Fees”. Reference for base fees, prioritization fees, and compute-budget-related limits.
- Cosmos SDK v0.47, “Transaction Lifecycle”. Versioned reference for transaction creation, mempool checks, ante handling, delivery, and commit.
- Sui Foundation, “Transaction Overview”. Overview of transaction metadata, object inputs and outputs, gas input, expiration, and effects.
- ethereum.org, “JSON-RPC API”. Useful for transaction receipts, logs, status, and execution client interaction.
- Solidity Documentation, “Events”. Reference for events, logs, topics, and event interpretation.
- Solidity Documentation, “Error handling: Assert, Require, Revert and Exceptions”. Reference for `require`, `revert`, exceptions, and state rollback.

Chapter 4

Keys, Signatures, and Authorization

4.1 Purpose

The previous chapters built the state-machine frame, identified assets and authority, and explained how transactions carry inputs into execution. This chapter studies the cryptographic mechanism most often used to prove authority:

A digital signature proves that a signing authority authorized a specific message under a specific verification rule.

That is all it proves.

It does not prove the signer understood the message. It does not prove the transaction was safe. It does not prove the UI was honest. It does not prove the key was not stolen. It does not prove the signature is valid in only one domain unless the signed message binds that domain. It does not prove the same authority should be allowed to perform every action associated with an address.

Digital signatures are necessary for blockchain authorization, but they are not a complete authorization model.

This chapter's central discipline is:

Never ask only "is the signature valid?"

Ask:

- valid under which public key?
- over which exact bytes?
- for which domain?
- for which operation?
- at which time?
- with which replay protection?
- causing which state transition?

In blockchain systems, private keys are not merely credentials. They are portable authority. A compromised key can move assets, approve spenders, vote, upgrade contracts, operate validators, sign oracle updates, relay bridge messages, and authorize withdrawals. A misunderstood signature can create the same practical result as a stolen key.

4.2 Learning Objectives

By the end of this chapter, you should be able to:

- Explain private keys, public keys, addresses, seed phrases, derivation paths, and wallet accounts without conflating them.
- Compare ECDSA, EdDSA, BLS, and Schnorr at the level needed for blockchain security review.
- Explain why nonce generation, malleability rules, subgroup checks, rogue-key resistance, and domain separation matter.
- Distinguish transaction signing from message signing.
- Explain why EIP-712-style typed data improves but does not solve signing safety.
- Compare on-chain multisig, threshold signatures, MPC signing, and hardware signing.
- Identify authorization failures involving replay, malleability, confused signing, stale approvals, key compromise, and wrong signer assumptions.
- Write an authorization model for a protocol action.

4.3 The Core Model

A signature system has three basic operations:

keygen:

```
private_key, public_key
```

sign:

```
signature = Sign(private_key, message)
```

verify:

```
Verify(public_key, message, signature) -> valid or invalid
```

NIST FIPS 186-5 describes digital signatures as mechanisms that verify the identity of a signatory and the integrity of signed data; signature generation uses a private key, while verification uses the corresponding public key.¹

In blockchain systems, this becomes:

¹NIST, “FIPS 186-5: Digital Signature Standard”, <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf>.

private key:

authority material

public key:

verification material

address:

usually a shorter identifier derived from, associated with, or assigned to a public key
,
account, script, object, or contract

signature:

evidence that the signing authority approved specific bytes

authorization:

the protocol decision to allow a state transition because verification and policy
checks pass

The last line is the important one.

A minimal authorization-flow map should make the signed payload explicit:

message:

action
parameters
signer
verifying contract or program
chain ID or domain
nonce or sequence number
deadline or expiry
intended recipient or beneficiary

verification:

recover or verify signer
check domain
check nonce unused
check deadline
check signer has authority for this action in current state
apply transition only if all checks pass

For review work, draw the flow before trusting the signature:

user or signer

-> signed payload
-> verifier
-> domain and replay checks
-> authority policy

-> state transition

Signature verification is usually only one part of authorization:

```
require(signature_valid)
require(nonce_unused)
require(deadline_not_expired)
require(chain_id == expected)
require(verifying_contract == this)
require(action_allowed_for_signer)
require(current_state_still_matches_signed_assumptions)
```

If any of these checks is missing, a valid signature can authorize the wrong transition.

4.4 Private Keys, Public Keys, and Address Derivation

4.4.1 Private Keys Are Authority Material

A private key is a secret value used to generate signatures. Whoever can use the private key can usually exercise the authority attached to the corresponding account, address, validator, role, or signer policy.

For an externally owned Ethereum account, ethereum.org states that the private key is used to sign transactions and grants custody over funds associated with the account.² More generally:

If the protocol accepts signatures from key K as authority for action A,
then control of K is control of A.

This statement is deliberately broader than “control of funds.” A key may control:

- A user account.
- A token allowance.
- A governance vote.
- A validator duty.
- A bridge validator role.
- An oracle publisher role.
- A deployment account.
- A proxy admin.
- A backend withdrawal signer.
- A session key.
- A recovery guardian.
- A threshold-signing share.

²ethereum.org, “Ethereum Accounts”, <https://ethereum.org/developers/docs/accounts/>.

Key security must be evaluated according to what the key can authorize, not what the UI labels it.

4.4.2 Public Keys Verify; Addresses Identify

A public key is derived from a private key and used to verify signatures. An address is usually a compact identifier derived from a public key, script, account, object, or contract.

These are not the same object.

Ethereum:

private key -> public key -> Keccak-256(public key) -> last 20 bytes -> address

Ethereum.org describes public address derivation as taking the last 20 bytes of the Keccak-256 hash of the public key and prefixing it with 0x.³

Bitcoin-style systems may use addresses derived from hashes of public keys or scripts. Solana account addresses are 32-byte public keys for ordinary keypair accounts, while program-derived addresses are derived addresses without private keys and are controlled by program logic. Some systems use account addresses that are not direct hashes of public keys at all.

Security consequences:

- An address is not always a public key.
- A contract address may have no private key.
- A program-derived address may be controlled by program rules, not by an externally held private key.
- A smart account may validate signatures using arbitrary logic.
- The same public key may correspond to different address formats on different chains.
- A signature over bytes does not automatically prove control of every address format that can be derived from the same key material.

Do not say “this address signed.” Addresses do not sign. Keys or account-validation logic sign or validate.

4.4.3 Key Generation Depends on Entropy

Private keys must be generated with sufficient unpredictability.

Bad key generation breaks everything downstream:

- Weak random number generators.
- Predictable seeds.

³ethereum.org, “Ethereum Accounts”, <https://ethereum.org/developers/docs/accounts/>.

- Browser entropy failures.
- Brain wallets from human-chosen phrases.
- Reused test keys.
- Deterministic derivation from public data.
- Leaked seed generation logs.
- Insecure hardware or firmware.

The math may be sound and the protocol may be correct, but if the private key is guessable, the account is gone.

Security questions:

Who generated the key?

What entropy source was used?

Was generation offline or online?

Was the seed ever displayed, transmitted, logged, or backed up insecurely?

Can the same seed generate keys on multiple chains?

Can a compromised derivation path expose more authority than intended?

Key generation is part of the security model, not a wallet UX detail.

4.4.4 Seed Phrases Are Root Authority

Many wallets derive many keys from one root secret. BIP-39 defines a mnemonic sentence as a way to encode entropy and derive a binary seed from that mnemonic using PBKDF2 with HMAC-SHA512.⁴

The security implication is severe:

Compromise of the seed phrase compromises every descendant key derived from it, unless another mechanism prevents use of those keys.

A seed phrase can control:

- Multiple accounts.
- Multiple chains.
- Multiple address types.
- Hidden accounts at different derivation paths.
- Legacy and modern wallet accounts.
- Testnet and mainnet accounts.

Users often think a seed phrase backs up “this wallet app.” In reality, it may back up a tree of cryptographic authority.

Seed phrase review questions:

⁴Marek Palatinus et al., “BIP-39: Mnemonic code for generating deterministic keys”, <https://github.com/bitcoin/bips/blob/master/bip-0039.mediawiki>.

- Is the mnemonic generated with enough entropy?
- Is it shown only on a trusted display?
- Is there an optional passphrase?
- Are backups resistant to theft, loss, fire, coercion, and inheritance failure?
- Are users told which accounts and chains the seed controls?
- Can imported seeds be scanned by malicious software?
- Are old unused accounts still holding allowances or roles?

A seed phrase is not a password. A password can often be reset by an authority. A seed phrase often is the authority.

4.4.5 Hierarchical Deterministic Wallets Create Authority Trees

BIP-32 defines hierarchical deterministic wallets, where a tree of keypairs is derived from a single seed. It supports extended private keys and extended public keys, with hardened and non-hardened child derivation.⁵

This is powerful because:

- One backup can recover many keys.
- Public child keys can be derived from an extended public key for non-hardened paths.
- Services can generate receiving addresses without holding spending keys.
- Accounts can be separated by purpose, coin type, account index, change, and address index.

It is also dangerous.

An extended private key can derive descendant private keys. An extended public key can reveal descendant public keys for non-hardened paths. In some derivation designs, exposure of a child private key together with a parent extended public key can compromise the parent private key; BIP-32 discusses hardened derivation partly to reduce parent-key leakage risk.⁶

Security questions:

Which root seed controls this account?

Which derivation path produced this key?

Is the path hardened where it needs to be?

Has an xpub been exposed?

Can the xpub reveal addresses, balances, privacy, or future receiving addresses?

Can one compromised child key endanger other branches?

Derivation paths are not metadata. They are part of the authority map.

⁵Pieter Wuille, “BIP-32: Hierarchical Deterministic Wallets”, <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>.

⁶Pieter Wuille, “BIP-32: Hierarchical Deterministic Wallets”, <https://github.com/bitcoin/bips/blob/master/bip-0032.mediawiki>.

4.4.6 Wallets Are Interfaces to Authority

Ethereum.org makes a useful distinction: an account is not a wallet; a wallet is an interface or application that lets a user interact with an account.⁷

This distinction matters because wallets mediate:

- Key generation.
- Key storage.
- Address derivation.
- Transaction construction.
- Message display.
- Signature requests.
- Chain switching.
- dApp permissions.
- Transaction simulation.
- Hardware-signer communication.
- Recovery flows.

The wallet may be honest, compromised, malicious, confused, outdated, or connected to a malicious dApp.

Security review must separate:

Account authority:

What the key or smart account can authorize.

Wallet behavior:

What the interface asks the user to sign and how it explains the result.

A mathematically valid signature can still be a user-side security failure if the wallet presented the action incorrectly.

4.4.7 Key Rotation and Revocation Are Hard

In many web systems, credentials can be reset. In blockchain systems, key rotation depends on the asset model.

Easy cases:

- A smart contract role can be transferred to a new key.
- A multisig owner can be replaced.
- A smart account can update validation logic.

⁷ethereum.org, “Ethereum Accounts”, <https://ethereum.org/developers/docs/accounts/>.

- A validator can update withdrawal credentials or rotate operational keys if the protocol supports it.

Hard cases:

- A lost EOA key controlling funds cannot be rotated by itself.
- A compromised seed with old token approvals may remain dangerous.
- An immutable contract owner set to a lost key may be unrecoverable.
- A bridge validator set may require governance to rotate.
- A hardware wallet compromise may require sweeping all assets and revoking approvals.

Key lifecycle questions:

How is the key generated?

How is it stored?

How is it backed up?

How is it used?

How is use monitored?

How is it rotated?

How is authority revoked?

What happens if it is lost?

What happens if it is compromised?

If there is no rotation path, then key compromise is not an incident. It is final authority loss.

4.5 ECDSA, EdDSA, BLS, Schnorr, and Signature Assumptions

4.5.1 Digital Signatures Provide Authenticity and Integrity

A digital signature is meant to provide:

- Data integrity: the signed message was not modified.
- Origin authentication: the signer possessed the private key corresponding to the public key.
- Non-repudiation in some legal or operational contexts.

NIST FIPS 186-5 describes digital signatures as detecting unauthorized modification and authenticating the signatory, while also warning that security depends on protecting private keys and that conformance to a standard does not guarantee system security.⁸

For blockchain review, translate this into:

The signature proves key possession for a message.

The protocol must decide what that key is allowed to do with that message.

⁸NIST, “FIPS 186-5: Digital Signature Standard”, <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf>.

4.5.2 ECDSA

ECDSA is widely used in Bitcoin and Ethereum account transactions. FIPS 186-5 specifies ECDSA and approves deterministic ECDSA as specified in RFC 6979.⁹

ECDSA security depends on:

- The elliptic curve group, such as secp256k1 in Bitcoin and Ethereum.
- Private key secrecy.
- Correct public key validation.
- Correct message hashing.
- Correct per-signature nonce generation.
- Correct signature verification.
- Correct malleability handling.
- Side-channel resistance.

The per-signature nonce is especially critical. If the same ECDSA nonce is reused across two different messages, or if nonces are biased enough, the private key can be recovered.

Security review questions:

Who signs?

Which curve?

Which hash?

How is the per-message nonce generated?

Is deterministic ECDSA used correctly?

Are signatures canonicalized?

Can signatures be malleated?

Is the recovered signer checked against the expected authority?

4.5.3 ECDSA Malleability and Low-s Rules

ECDSA signatures are pairs usually represented as (r, s) plus sometimes a recovery identifier. For many curves, $(r, n - s)$ can also verify for the same message and public key unless verification rules reject one form.

This is signature malleability: a valid signature can be transformed into a different valid signature for the same message.

Bitcoin and Ethereum have both had to care about this. Ethereum’s EIP-2 made high- s transaction signatures invalid while leaving the `ecrecover` precompile behavior unchanged for compatibility with existing signatures.¹⁰

⁹NIST, “FIPS 186-5: Digital Signature Standard”, <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf>.

¹⁰Ethereum Improvement Proposals, “EIP-2: Homestead Hard-fork Changes”, <https://eips.ethereum.org/EIPS/eip-2>.

Security consequences:

- Do not use the raw signature bytes as the unique identity of an authorization unless malleability is handled.
- Prefer consumed message hashes, nonces, or structured authorization IDs for replay protection.
- If using `ecrecover`, enforce valid `v`, `r`, `s` ranges and low-`s` where appropriate.
- Do not assume two byte-distinct signatures imply two different authorizations.

The safer question is:

What message or authorization is consumed?

Not:

Have these exact signature bytes been seen before?

4.5.4 EdDSA

EdDSA is a digital signature scheme over Edwards curves. RFC 8032 specifies Ed25519 and Ed448.¹¹ FIPS 186-5 also specifies EdDSA.¹²

EdDSA is commonly associated with:

- Deterministic signing.
- Compact keys and signatures.
- Fast verification.
- Ed25519 usage in ecosystems such as Solana.

But EdDSA is not “safe by default” in every integration.

Security concerns include:

- Public key validation.
- Canonical encoding.
- Cofactor and subgroup handling.
- Domain separation variants.
- Prehash vs pure modes.
- Incorrect library APIs.
- Signing-oracle misuse.
- Cross-protocol key reuse.

¹¹Josefsson and Liusvaara, “RFC 8032: Edwards-Curve Digital Signature Algorithm”, <https://datatracker.ietf.org/doc/html/rfc8032>.

¹²NIST, “FIPS 186-5: Digital Signature Standard”, <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.186-5.pdf>.

Security review questions:

Which EdDSA variant is used?

Is the message prehashed?

Is domain separation included?

Are public keys and signatures canonical?

Does the library verify according to the ecosystem’s expected rules?

Can the same key sign messages in another protocol with different assumptions?

Do not assume “Ed25519” names one complete security model. The verification rules and message domain matter.

4.5.5 BLS

BLS signatures are pairing-based signatures with aggregation properties. The CFRG BLS draft describes BLS as deterministic, non-malleable, aggregation-friendly, and useful where storage or bandwidth are scarce; it also explains that aggregate signatures can authenticate multiple messages and public keys.¹³

Ethereum proof-of-stake uses BLS keys for validators; the Ethereum consensus specs define BLS signature helpers such as `Sign`, `Verify`, `Aggregate`, `FastAggregateVerify`, `AggregateVerify`, and `KeyValidate` for beacon-chain operations.¹⁴

BLS is powerful because many signatures can be compressed into one aggregate signature. That is valuable for consensus protocols where many validators sign votes.

But aggregation introduces specific risks:

- Rogue-key attacks.
- Missing proof-of-possession checks.
- Public key validation failures.
- Subgroup membership failures.
- Wrong domain separation.
- Aggregating signatures over the wrong message set.
- Treating aggregate success as proof that all expected signers signed.

The BLS draft explicitly defines a rogue-key attack as one involving a specially crafted public key used to forge an aggregate signature, and specifies methods such as proof-of-possession schemes to secure against it.¹⁵

Security review questions:

¹³Boneh et al., “BLS Signatures” Internet-Draft, <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-bls-signature>.

¹⁴Ethereum Consensus Specs, “Beacon Chain: BLS Signatures”, <https://ethereum.github.io/consensus-specs/specs/phase0/beacon-chain/#bls-signatures>.

¹⁵Boneh et al., “BLS Signatures” Internet-Draft, <https://datatracker.ietf.org/doc/html/draft-irtf-cfrg-bls-signature>.

Which BLS scheme variant is used: basic, message augmentation, or proof-of-possession?

Are public keys validated?

Are subgroup checks performed?

Is the aggregate signer set explicit?

Are duplicate public keys handled?

Is the signed domain correct?

Can an aggregate signature be replayed across fork, domain, chain, or duty?

BLS aggregation reduces bandwidth. It does not reduce the need to know exactly who signed exactly what.

4.5.6 Schnorr Signatures

Schnorr signatures are important in Bitcoin Taproot and in threshold protocols. BIP-340 specifies Schnorr signatures for secp256k1, using 32-byte public keys and 64-byte signatures, and includes design choices such as tagged hashing for domain separation.¹⁶

Schnorr has properties that make it attractive:

- Simpler structure than ECDSA.
- Natural linearity useful for multisignatures and threshold signatures.
- Efficient verification.
- Compatibility with constructions such as MuSig and FROST.

But linearity is a tool, not a free lunch. Threshold and multisignature protocols must handle nonce commitments, rogue keys, participant binding, aborts, and signer coordination.

RFC 9591 specifies FROST, a two-round Schnorr threshold protocol, and states that threshold signatures require cooperation among a threshold number of participants, with security assuming an adversary corrupts fewer than the threshold.¹⁷

Security review questions:

Which Schnorr variant is used?

How are nonces generated and committed?

Are public keys bound to participants?

Is the signer set committed to the challenge?

Can a coordinator equivocate?

Can nonce reuse reveal a key or share?

Can an aborting signer grief liveness?

Schnorr makes powerful threshold designs possible. Those designs still have protocol risk.

¹⁶Pieter Wuille et al., “BIP-340: Schnorr Signatures for secp256k1”, <https://github.com/bitcoin/bips/blob/master/bip-0340.mediawiki>.

¹⁷Connolly et al., “RFC 9591: The Flexible Round-Optimized Schnorr Threshold Protocol”, <https://www.rfc-editor.org/rfc/rfc9591.html>.

4.5.7 Signature Assumptions Are Broader Than Algorithms

When a protocol says “secured by signatures,” ask which assumptions are included.

Assumption	Failure
Private keys remain secret	Attacker signs valid messages
Signing devices are honest	Device signs unintended payload
Randomness/nonces are safe	Private key or share recovery
Message encoding is canonical	Signature authorizes ambiguous message
Domain separation is correct	Signature replay across contexts
Public keys are validated	Invalid-curve or subgroup attacks
Signature malleability is handled	Replay or duplicate authorization confusion
Signer set is explicit	Aggregate/multisig result misinterpreted
Verification code is correct	Forged or wrong signatures accepted
Users understand the prompt	Confused signing and phishing

The signature algorithm is one row in the security model, not the whole model.

4.6 Message Signing, Transaction Signing, and Domain Separation

4.6.1 Transaction Signing Authorizes a Chain-Level Request

A transaction signature usually authorizes a specific transaction envelope.

For an Ethereum transaction, the signed data includes transaction fields such as nonce, recipient, value, calldata, gas parameters, and chain-specific replay-protection fields. EIP-155 binds transaction signing to a chain ID to reduce cross-chain replay.¹⁸

Transaction signing answers:

Did this account authorize this transaction envelope on this chain?

It does not answer:

Was the target contract honest?

Was calldata understandable?

Was the oracle safe?

Was the transaction ordered safely?

Did the user understand the effect?

Transaction signatures are necessary authority, not sufficient safety.

¹⁸Vitalik Buterin, “EIP-155: Simple Replay Attack Protection”, <https://eips.ethereum.org/EIPS/eip-155>.

4.6.2 Message Signing Authorizes Off-Chain or Contract-Interpreted Meaning

Message signing is broader and riskier. A user may sign bytes that are not a chain transaction:

- Login challenge.
- Permit.
- Limit order.
- Intent.
- Bridge withdrawal.
- Governance delegation.
- Session-key grant.
- Marketplace listing.
- Account-abstraction operation.
- Contract-wallet authorization.
- Terms acceptance.

The message becomes dangerous when a contract or backend treats it as authority.

Example:

```
sign("withdraw 100")
```

Questions:

- Withdraw from which protocol?
- Which chain?
- Which contract?
- Which token?
- Which recipient?
- Which nonce?
- Which deadline?
- Can it be replayed after upgrade?
- Can it be submitted by anyone?
- Is it valid if the user's state changed?

If the message does not bind these facts, the signature may authorize more than intended.

4.6.3 EIP-191 and the Problem of Raw Bytes

EIP-191 defines a signed data standard for Ethereum, including an initial 0x19 byte and version-specific data to prevent signed data from being valid RLP-encoded Ethereum transactions.¹⁹

¹⁹Ethereum Improvement Proposals, “EIP-191: Signed Data Standard”, <https://eips.ethereum.org/EIPS/eip-191>.

This is a domain-separation improvement over signing arbitrary raw bytes.

But it does not solve human comprehension. A user may still be asked to sign a hex blob or vague text, and a contract may still interpret the signature in a dangerous way.

Security rule:

A message-signing prefix can prevent one class of replay.

It does not prove the user understood the message or that the protocol scoped it correctly.

4.6.4 EIP-712 Typed Data

EIP-712 defines typed structured data hashing and signing for Ethereum. It includes a domain separator and a structured message hash, and its specification explicitly states that it does not include replay protection by itself.²⁰

EIP-712 is valuable because it can bind a signature to:

- Domain name.
- Version.
- Chain ID.
- Verifying contract.
- Structured fields.
- Human-readable typed data.

But the protocol must still include nonces, deadlines, expected state, and correct verification logic.

Good EIP-712 signing model:

Domain:

```
name
version
chainId
verifyingContract
```

Message:

```
action type
signer
recipient
token
amount
nonce
deadline
expected state if needed
```

²⁰Ethereum Improvement Proposals, “EIP-712: Typed structured data hashing and signing”, <https://eips.ethereum.org/EIPS/eip-712>.

Bad EIP-712 signing model:

Domain looks official.

Message omits nonce or deadline.

Contract accepts it forever.

Typed data makes dangerous signing easier to inspect. It does not make dangerous authorization impossible.

4.6.5 Permits and Signed Approvals

ERC-2612 adds permit, allowing ERC-20 approvals to be made by signature rather than by an on-chain approval transaction.²¹

This is useful:

- Better UX.
- Gasless approvals.
- Composable flows.
- Signer does not need native asset for gas.

It is also dangerous:

- A signature can grant spending authority.
- A phishing site can trick a user into signing a permit.
- A permit can be submitted by someone else.
- Replay protection depends on the token's nonce logic.
- Deadlines must be checked.
- The spender and value must be clear to the user.

Permit-style review checklist:

Is owner bound?

Is spender bound?

Is token contract bound?

Is chain ID bound?

Is amount bound?

Is nonce consumed?

Is deadline checked?

Can signature malleability bypass nonce tracking?

Can the spender immediately transfer after permit?

Does the UI clearly show this is an approval?

A permit is not “just a signature.” It is delegated spending authority.

²¹Ethereum Improvement Proposals, “ERC-2612: Permit Extension for EIP-20 Signed Approvals”, <https://eips.ethereum.org/EIPS/eip-2612>.

4.6.6 Contract Signatures and ERC-1271

EOAs verify signatures through public-key cryptography. Contract accounts can define their own signature-validation logic.

ERC-1271 standardizes a method for contracts to validate whether a signature is valid for given data.²²

This enables:

- Smart contract wallets.
- Multisig wallets.
- Session-key schemes.
- Social recovery.
- Account abstraction.
- Policies richer than one ECDSA key.

It also creates review questions:

- Is signature validity time-dependent?
- Can the contract’s validation logic change after signing?
- Does the signature remain valid after owner rotation?
- Is the validation result replayable across contracts?
- Does the verifier call `isValidSignature` on the correct contract?
- Can a malicious contract return valid for arbitrary messages?
- Is a signature from a contract wallet valid only under current state, or historical state?

For EOAs, signature validity is mostly cryptographic. For contract accounts, signature validity is code and state.

4.6.7 Domain Separation Checklist

Any signature that authorizes a protocol action should bind:

Protocol:

name and version

Domain:

chain ID, network, rollup, appchain, or source/destination domain

Verifier:

contract address, program ID, module, or backend verifier identity

²²Ethereum Improvement Proposals, “ERC-1271: Standard Signature Validation Method for Contracts”, <https://eips.ethereum.org/EIPS/eip-1271>.

Action:

function or operation type

Actor:

signer, owner, delegate, relayer, or account

Asset:

token, object, vault share, bridge message, or position

Parameters:

amount, recipient, market, order terms, or proposal data

Replay guard:

nonce, sequence, nullifier, order ID, or consumed-message ID

Time bound:

deadline, expiry, valid-after, valid-before, block, slot, or epoch

State expectation:

optional but important for price, chain state, order fill, or account configuration

The question is not whether the signature verifies. The question is whether it verifies only for the intended transition.

4.7 Multisig, Threshold Signatures, MPC, and Hardware Signing

4.7.1 Single-Key Control Is a Concentrated Failure Mode

A single key is simple:

one private key -> one signature -> one authority

It is also fragile:

- Key theft means full compromise.
- Key loss means loss of authority.
- Coercion of one person may be enough.
- Malware on one device may be enough.
- Operational mistakes are hard to stop.

High-value systems therefore distribute signing authority.

The main approaches are:

- On-chain multisig.
- Native script multisig.
- Threshold signatures.
- MPC signing.
- Hardware or HSM-backed signing.
- Hybrid policies combining several of these.

Each improves some risks and introduces others.

4.7.2 On-Chain Multisig

An on-chain multisig is a contract that enforces a policy such as:

```
execute transaction if at least m of n owners approve
```

Safe describes itself as smart accounts that allow users and organizations to manage digital assets, with multisig-style owner and threshold policies.²³

Benefits:

- Clear on-chain policy.
- Signer set and threshold may be observable.
- Can support batching, modules, guards, spending limits, and timelocks.
- Can rotate owners.
- Can use contract-level validation such as ERC-1271.

Risks:

- Contract bugs.
- Malicious or overpowered modules.
- Owner compromise.
- Threshold too low.
- Signer unavailability.
- Social engineering of signers.
- Blind signing of batched calls.
- Upgradeable multisig infrastructure.
- Chain-specific execution risk.

On-chain multisig review questions:

²³Safe, “What is Safe?”, <https://docs.safe.global/home/what-is-safe>.

Who are the owners?
What is the threshold?
Can owners be added or removed?
Who can change the threshold?
Are modules enabled?
Can modules bypass owner threshold?
Is there a timelock?
Can transactions be simulated and decoded?
Can signers verify exactly what they approve?

A multisig is not automatically safe. A 2-of-3 multisig where all keys sit on the same laptop is a one-device wallet with extra ceremony.

4.7.3 Native Multisig and Script Policies

Some chains have native multisig or script-level policies.

Bitcoin scripts can encode spending conditions such as multisig and timelocks. The authority is enforced by the script condition on a UTXO rather than by a separate contract account.

Benefits:

- Policy can be native to the spend condition.
- No general-purpose contract needed.
- Can combine keys and time locks.

Risks:

- Complex scripts can be hard to recover from.
- Fee and transaction-size constraints matter.
- Key loss can make funds unspendable.
- Policy privacy varies by construction.
- Coordinating signatures can be operationally hard.

The review question is still:

Which spending paths exist, and who can satisfy each one?

4.7.4 Threshold Signatures

Threshold signatures distribute one signing key among multiple participants. A threshold number cooperate to produce one signature that verifies under one public key.

FROST, for example, lets a threshold number of participants cooperate to compute a Schnorr signature, improving distribution of trust and redundancy.²⁴

Benefits:

- On-chain output may look like one normal signature.
- Lower fees and better privacy than some on-chain multisig schemes.
- No single participant holds the full signing key.
- Can support institutional signing without exposing a single key.

Risks:

- Distributed key generation complexity.
- Signing-round coordination.
- Nonce misuse or reuse.
- Aborting signers causing liveness failure.
- Coordinator equivocation.
- Share backup and recovery complexity.
- Difficult incident response if shares are suspected compromised.
- Harder on-chain transparency: observers may not know the real signing policy.

Threshold signature review questions:

How was the key generated?

Was there a trusted dealer or distributed key generation?

How many shares exist?

What threshold is required?

Who holds shares?

How are signing nonces generated and committed?

Can one participant block signing?

Can compromised shares be rotated?

Can policy changes be audited?

Does on-chain state reveal the real trust model?

Threshold signatures hide complexity from the chain. Reviewers must not let the complexity hide from the threat model.

4.7.5 MPC Signing

Multiparty computation signing is a broader operational pattern where multiple parties jointly compute a signature without reconstructing the private key in one place.

²⁴Connolly et al., “RFC 9591: The Flexible Round-Optimized Schnorr Threshold Protocol”, <https://www.rfc-editor.org/rfc/rfc9591.html>.

MPC is often used by custodians, exchanges, funds, and institutional wallets.

Benefits:

- No single machine holds the full key.
- Signing can enforce approval workflows.
- Shares can be distributed across devices, people, cloud environments, or vendors.
- Policies can include risk engines, limits, and human approvals.

Risks:

- Complex implementation.
- Vendor trust.
- Coordinator or policy-engine compromise.
- Share refresh failures.
- Backup and disaster recovery failures.
- Insider collusion.
- Signing the wrong transaction because policy evaluation is wrong.
- Poor visibility to users and auditors.

MPC should be evaluated as a system:

```
Cryptographic protocol
+ signing policy
+ key-share storage
+ operator workflow
+ audit logging
+ recovery process
+ software supply chain
+ incident response
```

Calling something “MPC” is not a security proof.

4.7.6 Hardware Wallets and HSMs

Hardware signing devices and hardware security modules try to keep private keys inside a protected boundary.

NIST FIPS 140-3 defines security requirements for cryptographic modules across areas such as physical security, roles, authentication, software/firmware security, sensitive security parameter management, and mitigation of other attacks.²⁵

Hardware signing can reduce:

²⁵NIST, “FIPS 140-3: Security Requirements for Cryptographic Modules”, <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.140-3.pdf>.

- Private key exposure to a general-purpose computer.
- Malware extraction of raw keys.
- Some remote compromise risks.
- Some accidental key leakage.

It does not automatically prevent:

- Blind signing.
- Malicious transaction construction.
- Supply-chain compromise.
- Firmware compromise.
- Address poisoning.
- Wrong-chain signing.
- User approval of a malicious payload.
- Coercion.
- Bad backup practices.

Hardware signing review questions:

Does the device display the actual action being authorized?

Can it parse the transaction type or only show hashes?

Can users verify chain, contract, token, amount, recipient, and calldata?

How are firmware updates authenticated?

How is the seed backed up?

Can the host alter what is shown?

Can the device be used in a multisig or policy-controlled flow?

Hardware signing protects keys. It may not protect intent.

4.7.7 Choosing the Right Signing Architecture

Architecture	Strong against	Weak against
Single EOA	Simplicity, low overhead	Theft, loss, phishing, no policy
On-chain multisig	Single-key compromise, transparent policy	Contract/module bugs, signer coordination, blind approvals
Script multisig	Native policy, no general contract	Policy inflexibility, fee/size, recovery complexity
Threshold signature	Single-device key exposure, on-chain privacy	Protocol complexity, hidden policy, liveness griefing
MPC custody	Institutional workflow, distributed key use	Vendor/process risk, opaque failures, policy compromise

Architecture	Strong against	Weak against
Hardware wallet	Key extraction from host	Blind signing, supply chain, backup loss
Smart account	Rich policies, recovery, session keys	Contract bugs, upgrade risk, validation complexity

The right question is:

Which failure modes are unacceptable for this authority,
and which signing architecture actually reduces them?

4.8 Authorization Failures: Replay, Malleability, Confused Signing, and Key Compromise

4.8.1 Signature Validity Is Not Authorization Completeness

Authorization is a policy decision. Signature verification is one input.

Bad authorization:

```
require(ecrecover(hash, sig) == owner)
execute()
```

Better authorization:

```
require(ecrecover(domain_hash(action), sig) == owner)
require(nonce_unused[owner][nonce])
require(block.timestamp <= deadline)
require(action.target == expected_target)
require(action.chain_id == block.chainid)
require(action.verifying_contract == address(this))
require(action.amount <= allowed_limit)
nonce_unused[owner][nonce] = false
execute(action)
```

A valid signature over insufficient data is insufficient authorization.

4.8.2 Replay Failures

Replay occurs when an authorization can be reused outside its intended context.

Common replay failures:

- Same signature used twice because no nonce is consumed.

- Signature valid on multiple chains because no chain ID is bound.
- Signature valid for multiple contracts because no verifying contract is bound.
- Signature valid for multiple actions because function/type is omitted.
- Signature valid after expiry because no deadline exists.
- Signature valid after upgrade because version is omitted.
- Bridge message valid on multiple destinations because destination is omitted.
- Order signature partially fills repeatedly beyond intended amount.

Replay defense requires both:

scope:

`signature is valid only in the intended domain`

consumption:

`accepted authorization is marked used or bounded by fill accounting`

Nonce without domain is incomplete. Domain without nonce may still replay.

4.8.3 Malleability Failures

Malleability matters when a protocol treats signature bytes as identity.

Unsafe:

`used[signature] = true`

Safer:

`used[signer][nonce] = true`

or:

`used[authorization_hash] = true`

where `authorization_hash` is a canonical structured hash of the intended action.

Malleability review questions:

Can another valid signature exist for the same message?

Does verification enforce canonical signatures?

Is replay protection based on signature bytes or message identity?

Can different encodings produce the same semantic action?

4.8.4 Confused Signing

Confused signing happens when the signer authorizes one thing while believing they are authorizing another.

Examples:

- User signs `permit` while believing they are logging in.
- User signs unlimited token approval while believing they are swapping.
- User signs a Seaport-style order without understanding consideration items.
- Multisig signers approve a batch without decoding every call.
- Hardware wallet shows only a hash.
- Frontend displays token symbol from an untrusted token contract.
- Wallet simulation omits `delegatecall` or post-signature effects.
- Governance delegates vote for a proposal without inspecting executable calldata.

Confused signing is not solved by stronger cryptography. It is a UI, protocol-design, and domain-separation problem.

Defenses:

- Typed structured signing.
- Clear domain separators.
- Human-readable transaction decoding.
- Simulation with explicit limitations.
- Spending limits.
- Session-key scopes.
- Timelocks.
- Multisig review processes.
- Hardware display of critical fields.
- Warnings for approvals, permits, `delegatecalls`, upgrades, and unknown contracts.

The review question:

Could a reasonable signer believe this signature is harmless while it actually grants valuable authority?

4.8.5 Wrong Signer and Wrong Authority

Many bugs come from verifying a valid signature from the wrong authority.

Examples:

- Recover signer but do not check signer equals `owner`.
- Check token owner, but not approved operator.
- Check message signer, but action affects another account.
- Check guardian signature for owner-only action.
- Accept a signature from any multisig owner instead of threshold result.
- Treat relay signature as user authorization.

- Treat backend signature as proof that user consented.
- Accept a contract wallet signature by checking ECDSA instead of ERC-1271.

Signature verification answers:

`This key signed this message.`

Authorization must answer:

`Is this key allowed to authorize this transition for this asset in this state?`

4.8.6 Stale Authority

An authorization may be valid when signed but unsafe when submitted.

Examples:

- A permit signed before a user revoked trust.
- A limit order signed before market conditions changed.
- A delegation signature signed before governance parameters changed.
- A withdrawal signature signed before an account was frozen.
- A session key valid after a user changed devices.
- A contract-wallet signature that remains accepted after owner rotation.

Defenses:

- Short deadlines.
- Nonce invalidation.
- Versioned domains.
- State-bound signatures.
- Revocation registries.
- Cancel functions.
- Fill accounting.
- Expected state checks.

Question:

`If this signature is submitted later, what assumptions may no longer hold?`

4.8.7 Key Compromise

Key compromise means the attacker can generate valid signatures.

Once this happens, the chain cannot usually distinguish attacker signatures from legitimate signatures. The signatures are valid.

Response depends on the authority:

- User account: sweep assets, revoke approvals, move NFTs, rotate smart-account owners if possible.
- Admin key: pause protocol, rotate roles, inspect pending privileged actions.
- Multisig signer: remove signer if threshold still safe.
- Threshold share: refresh shares or rotate key if protocol supports it.
- Validator key: stop duties, prevent slashable messages, rotate if supported.
- Oracle key: remove publisher, invalidate bad reports, pause dependent markets.
- Bridge key: pause bridge, rotate validator set, audit minted supply.
- Deployment key: verify deployments and revoke CI/CD secrets.

Key-compromise analysis:

What can the key sign?

What already-signed messages are pending?

What approvals or delegations remain active?

Can authority be revoked?

Can funds be moved before attacker moves them?

Can users observe compromise before damage?

Can protocol state distinguish pre-compromise and post-compromise signatures?

If the system has no key-compromise plan, the key is a single point of final failure.

4.8.8 Authorization Review Checklist

For any signature-gated action, write:

Action:

What state transition is being authorized?

Signer:

Which key, account, contract, role, or threshold policy authorizes it?

Message:

What exact bytes are signed?

Domain:

Chain, contract, version, function, module, source/destination, and operation type.

Replay:

Nonce, sequence, nullifier, order ID, fill accounting, or deadline.

Verification:

ECDSA, EdDSA, BLS, Schnorr, ERC-1271, multisig, threshold, or custom validation.

Signer-set policy:

One key, m-of-n, threshold, validator set, oracle committee, or governance rule.

State dependency:

Which state must still be true when the signature is submitted?

Failure mode:

What happens if the key is stolen, lost, stale, malicious, or unavailable?

User comprehension:

Can the signer understand the authority being granted?

This checklist is deliberately broader than cryptography because authorization is broader than cryptography.

4.9 Worked Example: Permit as Delegated Authority

Consider an ERC-20 token implementing ERC-2612 permit:

```
permit(owner, spender, value, deadline, v, r, s)
```

A user signs a typed message authorizing:

```
owner = Alice
spender = Router
value = 10,000 USDC
deadline = Friday 17:00 UTC
nonce = 42
token = USDC
chain = Ethereum mainnet
verifying contract = USDC token contract
```

If verification succeeds, the token updates:

```
allowance[Alice][Router] = 10,000 USDC
nonce[Alice] += 1
```

Now analyze it as authorization:

Asset:

Alice's token balance.

Authority granted:

Router can transfer up to 10,000 USDC from Alice using transferFrom.

Signer:

Alice's account.

Replay protection:

Alice's permit nonce plus deadline.

Domain:

token contract, chain ID, domain separator, typed Permit structure.

State transition:

allowance changes; no tokens move yet.

The most common user misunderstanding:

Alice thinks she signed a swap.

Actually she signed an approval.

The approval may be safe if:

- Router is the intended contract.
- Amount is bounded.
- Deadline is short.
- Token contract consumes nonce correctly.
- UI displays spender, amount, token, deadline, and chain.
- The spender's later transfer is constrained by the swap logic Alice intended.

It may be unsafe if:

- Spender is malicious.
- Amount is unlimited.
- Deadline is far in the future.
- Signature can be replayed.
- UI hides that this is approval authority.
- The signed domain is misleading.

The signature is not the end of the analysis. It is the beginning of the authority trail.

4.10 Practical Discipline: Translate Signatures Into Authority

When you see a signature, translate it into this sentence:

This signature allows [actor/verifier] to cause [state transition]
over [asset/state] in [domain], until [expiry],
provided [replay/state checks] pass.

Examples:

This transaction signature allows Ethereum validators to include a transaction from Alice's EOA that calls Router.swap on mainnet with nonce 17 and specified gas fields.

This permit signature allows the token contract to set Router's allowance over Alice's USDC to 10,000 until the deadline, if Alice's permit nonce is unused.

This BLS signature allows the consensus protocol to count validator V's attestation for this slot, fork, domain, and beacon block root.

If you cannot fill in that sentence, the authorization model is incomplete.

4.11 Review Questions

1. What does a digital signature prove, and what does it not prove?
2. Why is signature verification only one part of authorization?
3. What fields should a signed payload bind to reduce replay and confused-signing risk?
4. Why do transaction signing and message signing create different review questions?
5. How can a valid signature authorize an unsafe state transition?
6. What is the difference between an on-chain multisig, threshold signature scheme, MPC signing process, and hardware signer?
7. Why should private keys, validator keys, oracle keys, bridge keys, and admin keys be treated as authority-bearing assets?
8. Why does typed data improve signing safety without proving that a user understood the authorization?

4.12 Applied Exercises

4.12.1 Exercise 1: Signed-Payload Anatomy

Given a signed approval or permit-like message, write an authorization-flow map with:

- signer;
- verifying domain;
- chain ID or domain identifier;
- verifying contract or program;
- nonce or sequence number;
- deadline or expiry;
- action being authorized;
- state transition that becomes possible if the signature is accepted.

Expected output: one table and one paragraph explaining whether the signed payload is scoped narrowly enough.

4.12.2 Exercise 2: Replay and Domain-Separation Review

A protocol accepts signatures authorizing withdrawals. The message contains:

```
withdraw(user, token, amount)
```

It omits chain ID, verifying contract, nonce, and deadline.

Identify at least four replay or confused-signing risks. Then rewrite the payload fields so the authorization is scoped to one protocol, one domain, one action, and one time window.

Expected output: a risk list and a revised message schema.

4.12.3 Exercise 3: Compare Signing Arrangements

Compare the following for a protocol treasury:

- one EOA;
- on-chain multisig;
- threshold signature or MPC policy;
- hardware-backed signing process.

For each, list the authority it grants, the failure mode, the observability of misuse, and the recovery path.

Expected output: one comparison table.

4.12.4 Exercise 4: Authorization Finding Draft

Counterexample:

```
A permit signature is valid forever.
```

```
The protocol does not include a nonce in the signed message.
```

```
The same signature can be submitted multiple times after the user's balance changes.
```

Write a concise finding with:

- root cause;
- affected authority;
- exploit path;
- impact;
- recommended mitigation;

- regression test.

Expected output: a short review note, not a full report.

4.13 Key Takeaways

- Private keys are authority material. Evaluate them by what transitions they can authorize.
- Public keys verify signatures; addresses identify accounts, scripts, contracts, programs, or derived public-key material.
- Seed phrases and HD wallet roots can control many accounts across many chains.
- A wallet is an interface to authority, not the same thing as the account.
- Signature algorithms rely on assumptions about private-key secrecy, randomness, encoding, domain separation, public-key validation, and implementation correctness.
- ECDSA nonce failures can reveal private keys; ECDSA malleability can break naive replay protection.
- EdDSA, BLS, and Schnorr have different strengths and different integration risks.
- BLS aggregation and threshold signatures are powerful but require explicit signer-set, proof-of-possession, nonce, and domain handling.
- Message signing is often more dangerous than transaction signing because the meaning is defined by the verifier.
- EIP-712 improves structured signing but does not remove the need for nonces, deadlines, and correct domain separation.
- Multisig, threshold signatures, MPC, and hardware wallets reduce some risks while introducing others.
- Authorization means a valid signature plus a correct policy decision about what transition that signer may cause.

4.14 Further Reading

- NIST, “FIPS 186-5: Digital Signature Standard”. Primary reference for digital signature concepts, ECDSA, EdDSA, and key-protection warnings.
- ethereum.org, “Ethereum Accounts”. Useful distinction between EOAs, contract accounts, private keys, addresses, and wallets.
- Pieter Wuille, “BIP-32: Hierarchical Deterministic Wallets”. HD wallet derivation, extended keys, hardened derivation, and wallet tree structure.
- Marek Palatinus et al., “BIP-39: Mnemonic code for generating deterministic keys”. Mnemonic-to-seed construction and seed phrase framing.
- Josefsson and Liusvaara, “RFC 8032: Edwards-Curve Digital Signature Algorithm”. EdDSA standard reference.

- Pieter Wuille et al., “BIP-340: Schnorr Signatures for secp256k1”. Bitcoin Schnorr signature standard.
- Boneh et al., “BLS Signatures”. CFRG Internet-Draft for BLS signatures and aggregation considerations; check current status before treating it as a finalized standard.
- Ethereum Consensus Specs, “Beacon Chain: BLS Signatures”. Ethereum consensus-layer context for BLS usage.
- Connolly et al., “RFC 9591: FROST”. Threshold Schnorr signing protocol and security considerations.
- Ethereum Improvement Proposals, “EIP-191: Signed Data Standard”. Ethereum signed-data prefixing and versioning.
- Ethereum Improvement Proposals, “EIP-712: Typed structured data hashing and signing”. Structured data signing and domain separators.
- Ethereum Improvement Proposals, “EIP-155: Simple Replay Attack Protection”. Chain ID replay protection for Ethereum transactions.
- Ethereum Improvement Proposals, “EIP-2: Homestead Hard-fork Changes”. Includes Ethereum’s high-s transaction signature rule.
- Ethereum Improvement Proposals, “ERC-2612: Permit Extension for EIP-20 Signed Approvals”. Permit-based signed token approvals.
- Ethereum Improvement Proposals, “ERC-1271: Standard Signature Validation Method for Contracts”. Signature validation for contract accounts.
- Safe, “What is Safe?”. Practical reference for smart-account and multisig-style custody.
- NIST, “SP 800-57 Part 1 Revision 5: Recommendation for Key Management”. General key-management lifecycle and cryptographic-key guidance.
- NIST, “FIPS 140-3: Security Requirements for Cryptographic Modules”. Hardware and software cryptographic-module security requirements.

Chapter 5

Cryptographic Commitments and Proofs

5.1 Purpose

Chapter 1 treated blockchains as replicated state machines. Chapter 2 identified assets and authority. Chapter 3 showed how transactions carry inputs into execution. Chapter 4 studied keys, signatures, and authorization. Chapter 5 studies the cryptographic mechanism that lets participants make compact claims about large bodies of data:

A cryptographic commitment lets a system bind itself to data without always revealing or transmitting all of that data.

This is one of the quiet foundations of blockchain security.

A block hash commits to a block. A transaction hash commits to a transaction. A Merkle root commits to a set or sequence of leaves. An Ethereum `state_root` commits to the post-block world state. A `receipts_root` commits to receipts and logs. A bridge proof may claim that some event, storage slot, or message was included in a source chain. A rollup output root may commit to an L2 state. A data-availability commitment may claim that data exists and can be sampled or reconstructed. A ZK verifier may check a proof against public inputs that are themselves commitments.

These commitments are powerful because they let a verifier check a small proof against a small root instead of downloading and trusting everything.

But they are also dangerous when misunderstood.

A proof does not mean “this is true” in the abstract. It means:

Given this commitment root,
and this verification algorithm,

and these cryptographic assumptions,
this claim is consistent with the data committed by that root.

That sentence should feel precise. Precision is the point.

5.2 Learning Objectives

By the end of this chapter, you should be able to:

- Explain preimage resistance, second-preimage resistance, and collision resistance in security-review terms.
- Distinguish hashes, commitments, digests, identifiers, fingerprints, and proofs.
- Explain why canonical encoding and domain separation are necessary for safe commitments.
- Describe Merkle trees, Merkle Patricia tries, state roots, transaction roots, receipt roots, and inclusion proofs.
- Explain what a proof actually proves and what it does not prove.
- Identify proof-verification bugs involving wrong roots, stale roots, malformed proofs, replay, ambiguous encodings, and false trust assumptions.
- Write a trust-minimized claim in the form: root, proof, verification algorithm, domain, finality, and security assumption.

5.3 The Core Model

A commitment system has three pieces:

Data:

The thing being committed to.

Commitment:

A compact value derived from the data.

Opening or proof:

Evidence that a specific claim is consistent with the commitment.

For a simple hash commitment:

```
commitment = H(encoded_data)
```

```
proof = encoded_data
```

verify:

```
H(encoded_data) == commitment
```


For a Merkle inclusion proof:

```
commitment = merkle_root(all_leaves)
```

```
claim = leaf i has value v
```

```
proof = sibling hashes along the path from leaf i to root
```

```
verify:
```

```
  recompute root from v, i, and proof
```

```
  check recomputed_root == commitment
```

For a blockchain state proof:

```
commitment = state_root in a block header
```

```
claim = account or storage key has value v
```

```
proof = trie nodes needed to verify that key/value path
```

```
verify:
```

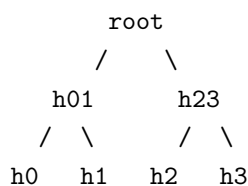
```
  recompute path under the trie rules
```

```
  check resulting root == state_root
```

```
  check state_root is from an acceptable block
```

The proof never floats in empty space. It is always relative to a root and a verification rule.

A minimal Merkle proof diagram should show the claim, the siblings, and the root:



```
claim:
```

```
  h2 is included
```

```
proof:
```

```
  sibling h3
```

```
  sibling h01
```

```
verify:
```

```
  h23 = H(h2 || h3)
```

```
  root' = H(h01 || h23)
```

```
  accept if root' == root
```

The security question is therefore:

Which root is trusted?

Why is that root trusted?

What exact claim is proved against it?

Which algorithm verifies the proof?

Which assumptions would make the proof invalid, stale, or irrelevant?

5.4 Hash Functions, Preimages, and Collision Resistance

5.4.1 A Cryptographic Hash Is Not Just a Checksum

A cryptographic hash function maps arbitrary-length input to a fixed-length output.

`digest = H(message)`

NIST’s Secure Hash Standard specifies SHA-family algorithms that generate message digests, and NIST SP 800-107 Rev. 1, used here as legacy terminology guidance, describes a hash function as mapping a bit string of arbitrary length to a fixed-length bit string.^{1 2}

For blockchain security, the important practical properties are:

- Determinism: the same input gives the same output.
- Preimage resistance: given a digest, it should be infeasible to find an input that hashes to it.
- Second-preimage resistance: given one input, it should be infeasible to find a different input with the same digest.
- Collision resistance: it should be infeasible to find any two different inputs with the same digest.
- Efficient verification: anyone can recompute and compare.

These properties let a small digest act as a compact fingerprint for much larger data.

But a cryptographic hash is not encryption. It does not hide low-entropy data. It does not authenticate who produced the data. It does not prove that the data is meaningful, safe, fresh, or final.

It proves only a relationship:

`This input hashes to this output.`

Everything else comes from context.

¹NIST, “FIPS 180-4: Secure Hash Standard”, <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.180-4.pdf>.

²NIST, “SP 800-107 Revision 1: Recommendation for Applications Using Approved Hash Algorithms”, legacy guidance whose supplementary material has moved to NIST hash-function resources, <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-107r1.pdf>.

5.4.2 Preimage Resistance

Preimage resistance means that given a hash output, an attacker should not be able to find an input that produces it.

Given $y = H(x)$,

it should be infeasible to find any x' such that $H(x') = y$.

NIST SP 800-107 Rev. 1 describes preimage resistance as the property that, given a randomly chosen hash value, finding an input that maps to it is computationally infeasible. For an L-bit hash function, expected preimage strength is generally L bits.³

In blockchain systems, preimage resistance appears in:

- Hashlocks.
- Commit-reveal schemes.
- Address derivation.
- Transaction and block identifiers.
- Content-addressed storage.
- Proof-of-work puzzles.
- Merkle roots and trie node references.

Security review question:

Is the committed input unpredictable enough that attackers cannot brute-force it?

This question matters because preimage resistance of the hash function is not enough if the input space is tiny.

Example:

```
commitment = H("yes")
```

This hides nothing. An attacker can hash "yes" and "no" and compare. A hash commitment to a low-entropy value needs a secret salt or nonce if hiding matters.

Better:

```
commitment = H("vote-v1" || voter || proposal_id || vote || random_256_bit_salt)
```

The hash function may be strong, but the protocol must give it enough entropy and context.

5.4.3 Second-Preimage Resistance

Second-preimage resistance means that after seeing a specific input, an attacker should not be able to find a different input with the same digest.

³NIST, "SP 800-107 Revision 1: Recommendation for Applications Using Approved Hash Algorithms", legacy guidance whose supplementary material has moved to NIST hash-function resources, <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-107r1.pdf>.

Given x ,
 it should be infeasible to find $x' \neq x$ such that $H(x') == H(x)$.

This property matters when a commitment has already been made to a known object.

Examples:

- Replacing a transaction with a different transaction that has the same transaction ID.
- Replacing a block body while preserving a block commitment.
- Replacing a contract artifact while preserving an expected source hash.
- Replacing a leaf value while preserving a Merkle root.

If second-preimage resistance fails, a commitment no longer binds the committer to the original data.

Security review question:

Can the prover substitute a different object while preserving the committed digest?

Usually the answer should be “computationally infeasible.” But that answer depends on the correct hash function, correct encoding, correct domain separation, and no exploitable ambiguity in what is being hashed.

5.4.4 Collision Resistance

Collision resistance means that an attacker should not be able to find any two different inputs with the same digest.

Find $x \neq x'$ such that $H(x) == H(x')$.

NIST SP 800-107 Rev. 1 states that for an L -bit hash function, the expected collision-resistance strength is $L/2$ bits because of birthday-bound attacks; for SHA-256, this corresponds to an expected 128-bit collision strength.⁴

Collision resistance matters when attackers can choose both inputs.

Examples:

- A malicious builder creates two different payloads with the same commitment.
- A contract accepts one document during review and a different document during execution.
- Two typed messages collide because encoding is ambiguous.
- A Merkle tree allows a leaf to be interpreted as an internal node.

Collision attacks are not the only hash risk, but they are central to binding commitments. If an attacker can produce two valid openings for one commitment, the commitment is not binding.

⁴NIST, “SP 800-107 Revision 1: Recommendation for Applications Using Approved Hash Algorithms”, legacy guidance whose supplementary material has moved to NIST hash-function resources, <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-107r1.pdf>.

5.4.5 Hash Security Depends on the Application

The same hash function may be used for different security goals.

Use	Needed property	Failure mode
Hashlock	Preimage resistance	Attacker claims without knowing secret
File fingerprint	Second-preimage resistance	Attacker swaps file for another with same hash
Digital signature prehash	Collision and second-preimage resistance	Attacker signs one message, substitutes another
Merkle root	Collision and second-preimage resistance	Attacker proves false membership
Commit-reveal	Binding and possibly hiding	Attacker changes value or brute-forces committed value
Content addressing	Second-preimage resistance	Wrong content served under expected address
Proof-of-work	Preimage-style search difficulty under target	Attacker finds cheap block headers below target

This is why “we hash it” is not a security argument. You must say which property the protocol needs and why the construction gives it.

5.4.6 Hashes Are Identifiers, Not Authority

Hashes often act as identifiers:

- Transaction hash.
- Block hash.
- Code hash.
- Storage root.
- Content hash.
- Proposal hash.
- Message hash.

But an identifier does not by itself authorize anything. A transaction hash identifies a transaction. It does not prove that the transaction is final, successful, economically safe, or submitted by the correct user unless the verifier checks the surrounding evidence.

Bad:

The transaction hash proves the transfer happened.

Better:

The transaction hash identifies a transaction. To prove the transfer happened, verify that the transaction is included in the canonical chain, executed successfully, emitted or caused the expected state change, and is final enough for the relying system.

Hashes are pointers into evidence. They are not the evidence by themselves.

5.4.7 Hash Functions Do Not Fix Bad Encoding

Hash functions process bytes. They do not understand your data model.

If two different meanings serialize to the same bytes, the hash cannot save you.

Example of ambiguous concatenation:

```
H("ab" || "c") == H("a" || "bc")
```

Both hash the same bytes:

```
"abc"
```

Safe encodings include:

- Length prefixes.
- Type tags.
- Version tags.
- Fixed-width fields where appropriate.
- Canonical serialization.
- Domain separators.
- Explicit chain and contract identifiers.

The security property is not:

```
H(data) is collision resistant.
```

It is:

```
H(canonical_encode(domain, type, version, fields)) binds to exactly one meaning.
```

Many proof and signature bugs are encoding bugs wearing cryptographic clothing.

5.5 Commitments, Binding Assumptions, and Domain Separation

5.5.1 A Commitment Binds a Claim to Data

A commitment is a compact value that binds the committer to data.

In cryptography, commitment schemes are usually discussed through two properties:

- Binding: after committing, the committer cannot open the commitment to a different value.
- Hiding: before opening, the verifier cannot learn the committed value.

Blockchain commitments often care primarily about binding and verifiability. Hiding may or may not be required.

Examples:

Commitment	Usually hiding?	Usually binding?	Opening/proof
$H(\text{transaction})$	No	Yes, assuming hash security and canonical encoding	Transaction bytes
Block hash	No	Yes	Block header/body validation
Merkle root	No	Yes	Merkle path
State root	No	Yes	Trie proof and state-transition validation
Commit-reveal vote hash	Should be	Should be	Vote plus salt
KZG commitment	Not necessarily	Yes under algebraic assumptions	Evaluation proof

The important lesson:

Commitment does not automatically mean secrecy.

A Merkle root commits to leaves. It usually does not hide them if the leaves are public or guessable. A hash of a small value is easy to brute-force. A public transaction hash does not hide the transaction once the transaction is known.

5.5.2 Binding Depends on the Whole Construction

Binding is not supplied by the hash function alone. It depends on:

- The cryptographic primitive.
- The exact bytes committed.
- Canonical serialization.
- Domain separation.
- Tree construction rules.
- Verification rules.
- The root source.
- Finality assumptions.

Suppose a bridge accepts:

```
message_hash = H(amount || recipient || nonce)
```

Questions:

- Is `amount` fixed-width?
- Is `recipient` fixed-width?
- Is `nonce` fixed-width?
- Is the source chain included?
- Is the destination chain included?
- Is the bridge contract included?
- Is the message type included?
- Is the token address included?
- Is the version included?

If not, the hash may be binding bytes but not binding the intended meaning.

The better construction is closer to:

```
message_hash = H(
    "bridge-message-v1" ||
    source_chain_id ||
    source_bridge_address ||
    destination_chain_id ||
    destination_bridge_address ||
    token_address ||
    recipient ||
    amount ||
    nonce
)
```

This is not merely cleaner. It prevents cross-protocol and cross-domain confusion.

5.5.3 Domain Separation Prevents Context Confusion

Domain separation means that commitments for one purpose cannot be validly interpreted as commitments for another purpose.

At minimum, domain separation asks:

What protocol is this for?

What chain or domain is this for?

What contract, program, or module is this for?

What version is this for?

What type of object is this?

What operation is this authorizing or proving?

Without domain separation, the same hash or signature may be reusable in another context.

Examples:

- A message signed for chain A is accepted on chain B.
- A withdrawal proof for token X is accepted for token Y.
- A vote commitment is interpreted as an order commitment.
- A Merkle leaf in one tree type is interpreted as an internal node in another.
- A bridge message for one destination contract is replayed into another.
- A testnet signature works on mainnet.

Domain separation is not optional ceremony. It is how a protocol says:

This commitment means this one thing here, and not something else somewhere else.

5.5.4 Canonical Encoding Is Part of the Commitment

If two honest implementations encode the same object differently, they will compute different commitments. If two different objects encode to the same bytes, the commitment is ambiguous.

Canonical encoding requires:

- One field order.
- One integer representation.
- One byte-order convention.
- One string normalization policy, or no strings where bytes are expected.
- One rule for optional fields.
- One rule for empty values.
- One rule for lists, maps, and sorting.
- One versioning scheme.

Ethereum uses Recursive Length Prefix encoding in its execution-layer trie construction, and the Ethereum consensus specs define Simple Serialize for consensus-layer objects and merkleization.^{5 6}

Security consequences:

- A proof verifier must decode exactly according to the committed format.
- A bridge verifier must not accept “equivalent” but noncanonical encodings.
- A signature verifier must bind the same bytes the user reviewed.
- A client implementation must not disagree with other clients about serialization.

If serialization is ambiguous, the cryptography is standing on wet ground.

5.5.5 Hiding Requires Entropy

A hash commitment hides only if the committed value is hard to guess.

Unsafe:

```
commitment = H(bid_amount)
```

If bid amounts are predictable, every bidder can brute-force likely values.

Better:

```
commitment = H("sealed-bid-v1" || auction_id || bidder || bid_amount || salt)
```

The salt must be unpredictable and large enough. A four-digit salt is not enough. A 256-bit random salt is usually the kind of magnitude expected for high-value commitments.

Hiding analysis asks:

Can an attacker enumerate possible committed values?

Is there a random salt?

Is the salt secret until reveal?

Is the salt unique?

Is the commitment bound to this auction, bidder, chain, and contract?

Can revealing in one context leak or authorize in another?

Commit-reveal protocols fail surprisingly often because the commit phase binds too little or hides too little.

5.5.6 Commitments Can Become Stale

A commitment may be correct and still unsafe to rely on.

Examples:

⁵ethereum.org, “Merkle Patricia Trie”, <https://ethereum.org/developers/docs/data-structures-and-encoding/patricia-merkle-trie/>.

⁶Ethereum Consensus Specs, “SimpleSerialize (SSZ)”, <https://ethereum.github.io/consensus-specs/ssz/simple-serialize/>.

- A state root from an old block proves an old balance, not the current balance.
- A transaction inclusion proof from an unfinalized block may disappear in a reorg.
- A bridge proof may be valid for a source-chain event that is not final enough.
- A rollup output root may be disputed during a challenge window.
- A price commitment may be stale even if correctly signed.
- A proof from a pre-upgrade contract may not mean the same thing after upgrade.

Every commitment has a time and domain.

Security review asks:

At what block, slot, epoch, version, or state root is this claim true?

Is that root final enough?

Is the relying protocol allowed to use old data?

Does the commitment expire?

Can a fresher contradictory commitment exist?

Freshness is not a cryptographic property. It is a protocol property.

5.6 Merkle Trees, Patricia Tries, State Roots, and Inclusion Proofs

5.6.1 Merkle Trees Compress Many Claims Into One Root

A Merkle tree hashes leaves into parent nodes until one root remains.

Simple binary example:

```
L0 = H("leaf" || data0)
```

```
L1 = H("leaf" || data1)
```

```
L2 = H("leaf" || data2)
```

```
L3 = H("leaf" || data3)
```

```
P0 = H("node" || L0 || L1)
```

```
P1 = H("node" || L2 || L3)
```

```
root = H("node" || P0 || P1)
```

The root commits to all leaves and their positions. A proof for data2 needs:

```
data2
```

```
L3
```

```
P0
```

```
position information: right child of P1, P1 right child of root
```

The verifier recomputes:

```

L2 = H("leaf" || data2)
P1 = H("node" || L2 || L3)
root' = H("node" || P0 || P1)
accept if root' == root

```

RFC 6962, the Certificate Transparency specification, uses Merkle hash trees to let clients verify that data is included in a log and to check consistency between tree sizes.⁷ Bitcoin uses Merkle trees so a lightweight client can verify that a transaction is included in a block without downloading the full block.⁸

For security analysis, the important feature is:

A small proof can connect one leaf to one trusted root.

5.6.2 Inclusion Proofs Prove Membership Relative to a Root

An inclusion proof proves:

This leaf is included in the committed data structure represented by this root.

It does not prove:

- The root is canonical.
- The root is final.
- The leaf is semantically valid.
- The leaf has not been superseded.
- The transaction succeeded.
- The event means what the verifier thinks it means.
- The same claim has not already been consumed elsewhere.

Those facts require additional checks.

For a Bitcoin SPV-style payment check, a Merkle branch can show transaction inclusion in a block. But the verifier still needs the block header chain and enough confirmation confidence. The Bitcoin whitepaper's simplified payment verification section relies on block headers and proof-of-work chain evidence, not a Merkle proof alone.⁹

For a bridge, an event proof can show that an event log was included in a source-chain receipt trie. But the bridge still needs:

- Correct source chain.
- Correct source contract.

⁷IETF, "RFC 6962: Certificate Transparency", <https://datatracker.ietf.org/doc/html/rfc6962>.

⁸Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", <https://bitcoin.org/bitcoin.pdf>.

⁹Satoshi Nakamoto, "Bitcoin: A Peer-to-Peer Electronic Cash System", <https://bitcoin.org/bitcoin.pdf>.

- Correct event signature and fields.
- Correct block header.
- Correct finality threshold.
- Correct replay protection.
- Correct destination-chain message handling.

The Merkle proof is one piece.

5.6.3 Tree Shape and Leaf Meaning Matter

Merkle verification must match the exact tree construction.

Important questions:

- Are leaves ordered?
- Is the tree binary, sparse, Patricia, or another structure?
- Are internal nodes domain-separated from leaves?
- Are leaf values hashed before insertion?
- Are duplicate leaves allowed?
- What happens with an odd number of leaves?
- Does the proof include positions, or does the verifier sort pairs?
- Does membership include an index?
- Are absence proofs supported?
- Is the proof for a set, sequence, map, or key/value store?

A proof for “leaf value X exists somewhere” is weaker than a proof for “leaf value X exists at index i.” A proof for “key k maps to value v” is different from a proof for “value v appears somewhere.”

Security failure example:

Verifier accepts sorted pair hashing.

Protocol intended ordered leaves.

Attacker reorders siblings and proves a different position.

Another:

Verifier hashes raw leaf bytes.

Internal node bytes can be shaped like leaf bytes.

Attacker exploits leaf/internal ambiguity.

Domain tags such as "leaf" and "node" are a simple defense in many constructions. More generally, the committed encoding must make each node type unambiguous.

5.6.4 Patricia Tries Commit to Key/Value State

A Merkle tree is good for proving that a leaf appears in a sequence or set. A blockchain state often needs a map:

```
key -> value
```

Ethereum uses a modified Merkle Patricia Trie for execution-layer state. Ethereum.org describes Ethereum’s state as encoded into a special Merkle-tree-like structure, where a root value can prove facts about the data, and explains that Ethereum’s structure is a modified Merkle-Patricia trie designed for efficient state retrieval.¹⁰

At a high level:

```
state root
  commits to account trie

account trie entry
  commits to account fields, including storage root

storage root
  commits to contract storage trie
```

Ethereum blocks contain commitments such as:

- **stateRoot**: root of the final state after applying transactions.
- **transactionsRoot**: root of the transaction trie.
- **receiptsRoot**: root of the receipts trie.

Ethereum.org’s block documentation lists these roots in block and execution-payload structures, and notes that clients re-execute transactions to ensure the new state matches the block’s state root; the Yellow Paper formalizes the block/state-transition framing and trie-root fields.^{11 12}

This gives us the state-machine connection:

```
parent_state_root
+ ordered transactions
+ execution rules
= new_state_root
```

If a block claims a **stateRoot**, honest validators re-execute the transactions and check that the computed root matches. If they disagree about execution rules or trie construction, consensus can split.

¹⁰ethereum.org, “Merkle Patricia Trie”, <https://ethereum.org/developers/docs/data-structures-and-encoding/patricia-merkle-trie/>.

¹¹ethereum.org, “Blocks”, <https://ethereum.org/developers/docs/blocks/>.

¹²Gavin Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger”, <https://ethereum.github.io/yellowpaper/paper.pdf>.

5.6.5 State Proofs and Storage Proofs

A state proof proves that an account or storage value is included in a committed state root.

EIP-1186 specifies the `eth_getProof` RPC method for retrieving Merkle proofs for an account and storage slots.¹³ The method returns account fields and proof nodes that allow a verifier to check claims against Ethereum’s state trie.

A simplified storage-proof flow:

Inputs:

```
trusted block header
stateRoot from header
account address
storage slot key
claimed storage value
account proof
storage proof
```

Verify account proof:

```
account address -> account object under stateRoot
```

Extract storageRoot from account object

Verify storage proof:

```
storage slot key -> claimed value under storageRoot
```

Accept if:

```
both trie proofs verify
block header is trusted/final enough
chain/domain is correct
claimed value has the expected meaning
```

The proof does not prove that the storage value is current forever. It proves the value at that state root.

This distinction matters for:

- Bridges.
- Cross-chain governance.
- Historical queries.
- On-chain light clients.
- Rollup withdrawals.
- Oracle attestations.

¹³Ethereum Improvement Proposals, “EIP-1186: RPC-Method to get Merkle Proofs - `eth_getProof`”, <https://eips.ethereum.org/EIPS/eip-1186>.

- Proof-of-reserves systems.

State proofs are often the difference between trusting an RPC response and verifying a claim against a committed root.

5.6.6 Receipt Proofs and Event Proofs

Events live in transaction logs, and logs are represented in receipts. Ethereum block headers include a `receiptsRoot`, committing to the receipts trie.¹⁴

An event proof usually needs to show:

```
block header commits to receiptsRoot
receiptsRoot commits to transaction receipt
receipt contains log
log was emitted by expected contract
log topics and data match expected event
block is canonical and final enough
event has not already been consumed
```

Common mistakes:

- Verifying the event data but not the emitting contract.
- Verifying the event but not the source chain.
- Verifying inclusion but not finality.
- Trusting a relayer’s JSON instead of verifying the receipt proof.
- Forgetting replay protection on the destination chain.
- Treating event existence as proof that the corresponding application invariant is true.

An event is a statement emitted by code. It is not automatically the state itself. Chapter 3’s rule still holds:

```
Events are evidence.
State is authority.
```

5.6.7 SSZ Merkleization and Consensus Objects

Ethereum’s consensus layer uses Simple Serialize (SSZ), which defines serialization, merkleization, and generalized indices for consensus objects.¹⁵

This matters because proof systems are not interchangeable.

¹⁴ethereum.org, “Blocks”, <https://ethereum.org/developers/docs/blocks/>.

¹⁵Ethereum Consensus Specs, “SimpleSerialize (SSZ)”, <https://ethereum.github.io/consensus-specs/ssz/simple-serialize/>.

An Ethereum execution-layer Merkle Patricia proof and an Ethereum consensus-layer SSZ proof have different:

- Serialization.
- Tree structure.
- Key/indexing model.
- Hashing rules.
- Object schemas.
- Verification algorithms.

Security review question:

Which commitment scheme is this proof for?

If the answer is vague, proof verification is already in trouble.

5.7 Proof Verification, Proof Forgery Risk, and Trust-Minimized Claims

5.7.1 A Proof Is Only as Good as Its Root

Every proof is relative to a root.

Bad verification:

```
verify(proof, claim) == true
```

Better verification:

```
verify(root, proof, claim, domain, rules) == true
```

Best verification also asks:

Where did root come from?

Is root canonical?

Is root final enough?

Is root from the expected chain, block, contract, epoch, version, and fork?

Is the verifying code using the exact committed encoding?

Has this claim already been consumed?

A valid proof against an untrusted root is meaningless. An invalid root can commit to any attacker's preferred universe.

5.7.2 Proof Verification Is a State Transition

When a smart contract, bridge, rollup, wallet, or backend verifies a proof and acts on it, proof verification becomes part of the state machine.

Example:

```
claimWithdrawal(proof, message):
    require(verifyProof(trustedRoot, proof, message))
    require(!consumed[message.id])
    consumed[message.id] = true
    transfer(message.amount, message.recipient)
```

The proof verifier is not passive. It gates value movement.

Security review must inspect:

- The root registry.
- The proof parser.
- The encoding rules.
- The hash function.
- The domain separator.
- The finality rule.
- The consumed-message state.
- The action triggered after verification.
- The failure behavior if verification reverts or runs out of gas.

Proof verification bugs are often bridge bugs, withdrawal bugs, governance bugs, and oracle bugs.

5.7.3 Common Proof Verification Failures

Failure	Description	Result
Wrong root	Proof verifies against attacker-supplied or stale root	False claims accepted
Wrong domain	Proof from chain A accepted for chain B	Cross-domain replay
Wrong contract	Event from fake contract accepted	Forged bridge or oracle event
Wrong encoding	Verifier hashes bytes differently from producer	False rejection or false acceptance
Missing position	Leaf value proved without index/key	Reordered or duplicate claim accepted

Failure	Description	Result
Missing nonmembership semantics	Absence proof interpreted incorrectly	Claims allowed despite existing state
Missing replay protection	Same proof consumed twice	Double mint or double withdrawal
Insufficient finality	Proof from reorged block accepted	Unbacked release
Malformed proof parser	Invalid nodes accepted or decoder confused	Proof forgery or denial of service
Gas/resource failure	Valid proofs too costly to verify	Liveness failure
Semantic mismatch	Proof is valid but proves the wrong thing	Unsafe state transition

The last row is especially common. A proof can be mathematically valid and still not support the action the protocol takes.

5.7.4 Proof of Inclusion Is Not Proof of Validity

Suppose a bridge sees a source-chain event:

```
Deposit(user, amount, nonce)
```

An inclusion proof can show that this event was in a receipt committed by a source-chain block.

It does not automatically prove:

- The deposit contract was not upgradeable to malicious code.
- The deposited token was genuine.
- The deposit was not later reverted by a reorg.
- The source chain had sufficient finality.
- The event fields were interpreted correctly.
- The message was intended for this destination.
- The user has not already claimed on the destination chain.

The proof answers an inclusion question. The protocol's security requires additional validity, finality, and replay checks.

This distinction is one of the most important in cross-chain security:

Inclusion answers "was this committed?"

Validity answers "should this committed thing be trusted for this action?"

5.7.5 Trust-Minimized Does Not Mean Trust-Free

Trust-minimized systems reduce the number or strength of assumptions a user must accept. They do not eliminate assumptions.

A storage proof can remove the need to trust an RPC provider's answer about a storage slot. But the verifier still trusts:

- The block header root.
- The consensus/finality mechanism that accepted that block.
- The hash and trie assumptions.
- The proof verification implementation.
- The meaning of the storage slot.
- The freshness requirement for the relying action.

A light client can reduce reliance on a full node. It still trusts a sync committee, validator quorum, proof-of-work chain, or other consensus evidence depending on design.

A ZK proof can reduce the need to re-execute computation. It still trusts the statement, circuit, verifier, public inputs, setup assumptions if any, and integration logic.

Serious phrasing:

`This system verifies storage membership against finalized Ethereum headers,
so it does not trust an RPC provider for the storage value.`

`It still assumes Ethereum finality, correct header verification,
correct MPT verification, and correct interpretation of the storage slot.`

That is how to write trust-minimized claims without fantasy.

5.7.6 Proofs Need Freshness and Expiration

Proofs often age badly.

Examples:

- A proof that an account had a balance at block N does not prove it has that balance at block $N + 100$.
- A proof that a message was unconsumed at block N does not prove it is unconsumed now.
- A proof that a validator set was active in epoch E may not apply in epoch $E + 1$.
- A proof that an oracle price was fresh at one block may be stale by execution time.
- A proof generated before an upgrade may not correspond to post-upgrade semantics.

Defenses:

- Include block number, slot, epoch, or timestamp in the verified claim.

- Enforce maximum age.
- Enforce finality minimums.
- Store consumed nullifiers or message IDs.
- Bind proofs to destination chain, contract, and version.
- Require expected state checks at execution time.
- Make stale proof failure explicit and safe.

Freshness belongs in the verifier, not in off-chain hope.

5.7.7 Proof Verification Can Be a Denial-of-Service Surface

Proof verification may be expensive.

Attackers may submit:

- Very large proofs.
- Deep trie paths.
- Malformed encodings.
- Duplicate nodes.
- Proofs that almost verify but fail late.
- Proofs that trigger worst-case parser behavior.
- Proofs for stale roots that waste computation.

On-chain proof verification must consider:

- Gas or compute limits.
- Maximum proof size.
- Maximum path length.
- Canonical encoding checks.
- Early rejection.
- Memory allocation.
- Hash cost.
- Reentrancy or callbacks after proof acceptance.
- Whether failed proofs cost the attacker enough.

A verifier that is cryptographically correct but economically unusable can still fail security goals through liveness.

5.7.8 The Verification Checklist

For any proof-based protocol, write the verification checklist explicitly:

Claim:

What exact fact is being proved?

Root:

Which commitment root is used?

Root source:

Where does the root come from?

Finality:

Why is the root canonical and final enough?

Domain:

Which chain, contract, program, module, version, and operation is this proof for?

Encoding:

Which canonical serialization and hashing rules apply?

Proof type:

Inclusion, absence, consistency, state, receipt, storage, light-client, ZK, or other?

Verifier:

Which algorithm and implementation checks it?

Freshness:

How old can the root or proof be?

Replay:

What state marks the claim consumed?

Action:

What state transition happens after verification?

Failure:

What happens if proof verification fails or is too expensive?

If a protocol cannot answer these, the proof system is not yet a security system. It is a cryptographic decoration.

5.8 Worked Example: Verifying a Cross-Chain Deposit Claim

Suppose a destination-chain bridge wants to mint wrapped tokens after a user deposits tokens into a source-chain bridge.

Naive design:

Relayer submits:

```
user
amount
source_tx_hash
```

Destination bridge:

```
if relayer is trusted:
    mint(user, amount)
```

This trusts the relayer for safety.

More trust-minimized design:

Relayer submits:

```
source block header
receipt proof
deposit event fields
transaction index / receipt index
log index
```

Destination bridge verifies:

```
source block header is accepted and final enough
receiptsRoot is in the block header
receipt proof verifies against receiptsRoot
receipt contains Deposit event
event was emitted by expected source bridge
event includes destination chain and destination bridge
event includes token, recipient, amount, nonce
nonce has not been consumed
```

Then:

```
consumed[source_chain_id][source_bridge][nonce] = true
mint(recipient, amount)
```

Now compare the trust assumptions.

Naive design:

The relayer tells the truth.

Proof-based design:

The relayer can provide data but cannot forge inclusion under a trusted finalized source block root.

The destination bridge still trusts the source-chain finality rule, source bridge contract, proof verifier, token mapping, and replay protection.

This is better, but not magic.

Possible bugs:

- The verifier accepts an unfinalized source block.
- The proof checks receipt inclusion but not emitting contract.
- The event omits destination chain ID.
- The nonce is global instead of scoped by source chain and source bridge.
- The token mapping can be changed by an admin without delay.
- The same event can be replayed to another destination bridge.
- The source bridge is upgradeable and can emit fake deposits.
- The proof parser accepts malformed trie nodes.
- The verifier runs out of gas on valid proofs during congestion.

The cryptographic proof is only one part of the security model. The transition after verification is where value moves.

5.9 Practical Discipline: Root, Proof, Claim

When someone says “we have a proof,” write three lines:

Root:

What compact commitment is trusted?

Proof:

What evidence connects the claim to the root?

Claim:

What exact statement follows if verification succeeds?

Then write three more:

Root trust:

Why is the root accepted?

Domain:

Where is the proof valid?

Action:

What state transition occurs because of the proof?

This prevents the most common proof-system mistake: verifying a true fact and then doing an unjustified thing with it.

5.10 Review Questions

1. What does a cryptographic commitment bind, and what does it not prove by itself?
2. Why does a proof need a trusted root and a precise verification algorithm?
3. What does a Merkle inclusion proof prove relative to a root?
4. Why does an inclusion proof not prove finality, semantic correctness, or non-replay by itself?
5. What is domain separation, and why does it matter for commitments and proofs?
6. How can stale roots, wrong roots, or wrong verification rules create valid-looking but unsafe claims?
7. Why are proof verification functions also state-transition surfaces?
8. What must a verifier check before accepting a cross-domain proof?

5.11 Applied Exercises

5.11.1 Exercise 1: Manual Merkle Verification

Given four leaves and a claimed proof for one leaf, draw the tree and recompute the root.

Expected output:

- tree diagram;
- proof path;
- recomputation steps;
- final accept/reject decision.

5.11.2 Exercise 2: Root and Domain Review

A bridge contract accepts a proof that a deposit event appeared on a source chain. The proof verifies against a block root, but the contract does not check source chain ID, finality depth, message nonce, or whether the message was already consumed.

Identify the failure modes.

Expected output: one table with columns for missing check, false assumption, possible impact, and mitigation.

5.11.3 Exercise 3: Proof-Verification Pseudocode

Write pseudocode for a verifier with this shape:

```
verify(root, claim, proof, domain, nonce) -> accept or reject
```

It must check:

- root source;
- domain;
- proof algorithm;
- claim encoding;
- replay or message consumption;
- finality or settlement assumption.

Expected output: pseudocode and a short explanation of each rejection condition.

5.11.4 Exercise 4: Trust-Minimized Claim Statement

Rewrite this weak statement:

The bridge is trustless because it verifies proofs.

Write a precise security claim that names the trusted root, proof algorithm, finality assumption, replay protection, and verifier authority.

Expected output: one improved security claim and one residual-risk paragraph.

5.12 Key Takeaways

- Hashes are compact fingerprints of bytes, not proofs of meaning, authority, freshness, or finality.
- Preimage, second-preimage, and collision resistance are different properties used for different security goals.
- Commitments bind data only if the primitive, encoding, domain separation, and verification rules are correct.
- Hash commitments do not hide low-entropy values unless they include sufficient secret randomness.
- Merkle proofs prove membership relative to a specific root and tree construction.
- State roots, transaction roots, and receipt roots are commitments to different parts of blockchain data.
- Ethereum Merkle Patricia proofs and SSZ proofs are different proof systems with different verification rules.
- A valid proof against an untrusted, stale, or wrong-domain root is useless or dangerous.
- Proof verification is itself a state transition when it gates minting, withdrawal, governance, or settlement.
- Trust-minimized does not mean trust-free. It means the remaining assumptions are narrower and should be named precisely.

5.13 Further Reading

- NIST, “FIPS 180-4: Secure Hash Standard”. Specification of SHA-family hash algorithms and their message-digest role.
- NIST, “SP 800-107 Revision 1: Recommendation for Applications Using Approved Hash Algorithms”. Useful legacy definitions of collision, preimage, and second-preimage resistance and security-strength guidance; NIST notes that supplementary material has moved to its hash-functions webpage and that remaining requirements are to move into CMVP implementation guidance before withdrawal.
- Satoshi Nakamoto, “Bitcoin: A Peer-to-Peer Electronic Cash System”. See the simplified payment verification discussion for Merkle branches and block headers.
- IETF, “RFC 6962: Certificate Transparency”. A clear standard reference for Merkle hash trees, inclusion proofs, and consistency proofs.
- Gavin Wood, “Ethereum: A Secure Decentralised Generalised Transaction Ledger”. Formal Ethereum reference for state, transactions, receipts, and trie roots.
- ethereum.org, “Merkle Patricia Trie”. Practical explanation of Ethereum’s modified Merkle-Patricia trie and state proofs.
- ethereum.org, “Blocks”. Useful overview of block fields including `state_root`, `transactionsRoot`, and `receiptsRoot`.
- Ethereum Improvement Proposals, “EIP-1186: RPC-Method to get Merkle Proofs - `eth_getProof`”. Specification for retrieving account and storage proofs.
- Ethereum Consensus Specs, “SimpleSerialize (SSZ)”. Consensus-layer serialization and merkleization reference.

Chapter 6

Threat Modeling Blockchain Systems

6.1 Purpose

The first five chapters gave us the primitives:

- State machines.
- Assets and authority.
- Transactions and state transitions.
- Commitments and proofs.
- Keys, signatures, and authorization.

This chapter turns those primitives into a working security-review method.

A threat model is a structured explanation of what the system is, what can go wrong, why those failures matter, and what will be done about them.

That sounds simple. In practice, most weak threat models fail in one of three ways:

- They are too vague: “attackers may steal funds.”
- They are too generic: “check access control, reentrancy, and arithmetic.”
- They are too decorative: a diagram exists, but it does not change what anyone reviews, tests, monitors, or fixes.

A useful blockchain threat model must be more concrete. It should tell a reviewer:

These are the assets.

These are the state transitions that affect them.

These actors can influence those transitions.

These assumptions must hold.

These assumptions are most fragile.

These threats deserve priority.

These invariants should be written.

These tests, proofs, mitigations, and monitoring controls should exist.

OWASP describes threat modeling as identifying, communicating, and understanding threats and mitigations in the context of protecting something valuable.¹ The Threat Modeling Manifesto frames threat modeling around four practical questions: what are we working on, what can go wrong, what will we do about it, and did we do a good enough job.²

For blockchain systems, we will adapt those questions:

What state machine are we reviewing?

Which assets and authorities matter?

Which adversarial transitions can break safety, liveness, solvency, or authorization?

Which assumptions are we relying on?

Which mitigations, invariants, tests, and monitoring controls make those assumptions defensible?

Threat modeling is not separate from auditing. It is how you decide where serious review begins. OWASP’s smart-contract security verification material makes this operational: security architecture review should identify threats, assess risks, and define mitigations for smart-contract systems.³

6.2 Learning Objectives

By the end of this chapter, you should be able to:

- Turn a whitepaper, docs, codebase, deployment map, or protocol description into a security model.
- Identify actors, assets, entry points, privileged roles, trust boundaries, and external dependencies.
- Separate users, attackers, rational participants, Byzantine actors, insiders, and trusted operators.
- Write explicit trust assumptions instead of vague claims about decentralization or security.
- Map attack surfaces across chain, contract, protocol, economic, infrastructure, and human layers.
- Use threat modeling to produce review priorities, invariants, tests, mitigations, and monitoring requirements.
- Recognize threat-modeling anti-patterns that waste time or create false confidence.

¹OWASP, “Threat Modeling”, https://owasp.org/www-community/Threat_Modeling.

²Threat Modeling Manifesto, <https://www.threatmodelingmanifesto.org/>.

³OWASP Smart Contract Security, “SCSVS-ARCH-3: Threat Modeling”, <https://scs.owasp.org/SCSVS/controls/SCSVS-ARCH-3/>.

6.3 The Core Model

A blockchain threat model connects seven things:

System:

What are we reviewing?

Assets:

What can be lost, corrupted, frozen, diluted, censored, or misrepresented?

Actors:

Who can interact with the system, operate it, influence it, or attack it?

Entry points:

Which transactions, messages, functions, instructions, proofs, admin actions, APIs, bots, and human processes can change state or influence decisions?

Assumptions:

What must be true outside the local code for the security claim to hold?

Threats:

What can go wrong if actors use entry points adversarially or assumptions fail?

Response:

What will we build, test, monitor, document, or explicitly accept?

In compact form:

Threat = actor + capability + entry point + vulnerable assumption + impact

Example:

Actor:

MEV searcher with flash liquidity

Capability:

Can manipulate a low-liquidity AMM pool for one block

Entry point:

borrow(collateral, amount)

Vulnerable assumption:

Oracle spot price reflects fair collateral value

Impact:

Attacker borrows against inflated collateral and leaves bad debt

That is a useful threat. It points to code, state, economic conditions, tests, and mitigations.

Compare:

Threat:

Oracle attack.

That is a label. It is not yet a model.

6.4 Turning a Protocol Description into a Security Model

6.4.1 Start With Scope

Threat modeling begins by defining what is inside the review.

Scope may include:

- Smart contracts or programs.
- Protocol docs and whitepaper.
- Token contracts.
- Governance contracts.
- Upgrade mechanisms.
- Deployed addresses.
- Admin multisigs.
- Frontend and wallet interactions.
- Backend signers.
- Keepers and bots.
- Oracles.
- Bridges and cross-chain messages.
- Rollup or appchain assumptions.
- Monitoring and incident response.
- Human operational processes.

Scope should also state what is outside the review.

Bad:

Review the protocol.

Better:

Review the EVM vault, strategy adapters, share accounting, oracle integration, upgrade path, and emergency controls. Exclude the frontend implementation and the external lending market used by one strategy, but document both as trust dependencies.

If the scope is unclear, the threat model will be unclear. If the threat model is unclear, review effort will drift toward whatever is easiest to inspect instead of whatever is most dangerous.

6.4.2 Collect the Actual System, Not the Marketing System

The system is not only the whitepaper.

Inputs to a real threat model include:

- Whitepaper.
- Protocol docs.
- Architecture diagrams.
- Smart contract code.
- Program code.
- ABI/IDL/interface definitions.
- Deployment addresses.
- Proxy admin addresses.
- Governance contracts.
- Multisig owners and thresholds.
- Timelock configurations.
- Oracle configuration.
- Token lists and accepted collateral.
- Backend services.
- Keeper and relayer design.
- Frontend transaction construction.
- Monitoring alerts.
- Incident response plan.
- Historical governance proposals.
- Previous audit reports.
- Known incidents and postmortems.

The docs describe the intended system. The deployment describes the actual system. The code describes executable rules. Operations describe who can change or rescue the system. Threat modeling needs all four.

If the docs say:

Governance controls upgrades through a 48-hour timelock.

Verify:

Which governance contract?

Which timelock?

Which proxy admin?

Which deployed implementation?
Who can cancel?
Who can execute?
Can anyone bypass the timelock?
Can emergency powers pause withdrawals?

Threat modeling starts polite. It does not stay naive.

6.4.3 Extract Security Claims

A protocol description usually contains security claims, even when it does not call them that.

Examples:

Depositors can always withdraw their fair share.
The oracle prevents price manipulation.
The bridge is trust-minimized.
Governance is decentralized.
The protocol is non-custodial.
The vault is delta-neutral.
Liquidations protect solvency.
The sequencer cannot steal funds.
The admin can only pause in emergencies.
The stablecoin is fully backed.

Convert each claim into reviewable form.

Weak:

The protocol is non-custodial.

Reviewable:

No privileged actor can transfer user funds, change withdrawal accounting, upgrade to logic that transfers user funds, freeze withdrawals indefinitely, or alter token mappings in a way that captures user claims without a documented trust assumption and user exit window.

Weak:

The bridge is trust-minimized.

Reviewable:

Destination-chain minting requires a proof against finalized source-chain state, source and destination domains are bound into the message, each message nonce is consumed exactly once, and no multisig or upgrade path can mint unbacked assets without an explicit trust assumption.

Every serious security claim should become either:

- An invariant.
- A trust assumption.
- A test.
- A proof obligation.
- A monitoring requirement.
- A risk acceptance.

If a claim becomes none of those, it is probably marketing.

6.4.4 Translate Features Into State Transitions

Feature lists hide attack surfaces.

Example feature list:

Users can deposit, withdraw, borrow, repay, and claim rewards.

Admins can configure markets.

Keepers handle liquidations.

Governance controls upgrades.

State-machine translation:

Feature	State transitions
Deposit	Transfer collateral in, mint shares or credit balance, update indexes
Withdraw	Burn shares or debit balance, transfer collateral out, update queues
Borrow	Read price, check collateral, mint or transfer debt asset, update debt
Repay	Transfer debt asset in, reduce debt, update interest indexes
Claim rewards	Calculate accrued rewards, mark claimed, transfer reward asset
Configure markets	Change collateral factors, caps, oracle, interest model
Liquidate	Read price, check health, seize collateral, repay debt, pay incentive
Upgrade	Change implementation or future transition rules

Now the reviewer can ask:

Which transition touches assets?
 Which transition reads external data?
 Which transition can become insolvent?
 Which transition can be called by anyone?
 Which transition depends on ordering?
 Which transition can be paused?
 Which transition cannot be paused?

Threat modeling should turn nouns into transitions.

6.4.5 Build a System Context Diagram

You do not need a beautiful diagram. You need an accurate one.

A minimal blockchain system context diagram should show:

Users / wallets

- > frontend
- > RPC provider
- > chain
- > protocol contracts/programs

Protocol contracts/programs

- > tokens
- > oracles
- > external protocols
- > governance
- > admin multisig
- > upgrade proxy
- > keepers/relayers
- > indexers/analytics
- > monitoring/alerts

For cross-domain systems:

source chain

- > source bridge contract
- > event/state proof
- > relayer or validator set
- > destination bridge contract
- > wrapped token

For each arrow, ask:

What crosses this boundary?

Who can modify it?

Who verifies it?

What happens if it is delayed, missing, stale, reordered, or false?

A simple attack tree turns a high-level threat into review paths:

```
Goal: drain or disable the protocol
-> abuse accounting transition
-> manipulate oracle input
-> compromise privileged authority
-> exploit external call/control flow
-> block withdrawals or exits
-> exploit upgrade or governance path
```

Use the tree to decide where review time goes first. A threat model that does not change review priorities is only decoration.

The Threat Modeling Manifesto explicitly warns against looking for a perfect representation and recommends multiple representations because different views reveal different problems.⁴ For blockchain protocols, useful views include:

- State-transition view.
- Asset-flow view.
- Authority graph.
- Trust-boundary view.
- Dependency graph.
- Transaction-sequence view.
- Upgrade/governance control graph.

No single diagram is sacred. The question is whether the representation exposes risk.

6.4.6 Threat Modeling Is Iterative

OWASP emphasizes that threat modeling should be refined through the lifecycle and updated after new features, incidents, or architecture changes.⁵

Blockchain systems especially require iteration because:

- Governance can change parameters.
- New collateral can be added.
- Oracles can be replaced.
- Token behavior can differ across integrations.
- Upgrade paths can change implementation.
- TVL can grow and change attacker incentives.
- Liquidity can move.

⁴Threat Modeling Manifesto, <https://www.threatmodelingmanifesto.org/>.

⁵OWASP, “Threat Modeling”, https://owasp.org/www-community/Threat_Modeling.

- MEV conditions can change.
- Bridge or rollup dependencies can change.
- New chains and deployments can be added.

A threat model written before deployment may be obsolete after the first governance proposal.

Threat model maintenance triggers:

- New asset listing.
- New strategy.
- New oracle.
- New chain deployment.
- Upgrade.
- Governance change.
- Admin role rotation.
- Incident or near miss.
- Large TVL increase.
- Market regime change.
- External dependency change.
- New integration by another protocol.

If the threat model is not updated when the system changes, it becomes a museum piece.

6.5 Mapping Actors, Assets, Entry Points, and Privileged Roles

6.5.1 Actor Inventory

Actors are entities that can interact with, operate, influence, or depend on the system.

In blockchain systems, actors often include:

- End users.
- Wallets.
- Traders.
- LPs.
- Borrowers.
- Liquidators.
- Arbitrageurs.
- MEV searchers.
- Validators, miners, builders, and sequencers.
- Governance token holders.

- Delegates.
- Admin multisigs.
- Guardians.
- Oracles and oracle operators.
- Keepers.
- Relayers.
- Bridge validators.
- Provers.
- Frontend maintainers.
- RPC providers.
- Indexers.
- Backend operators.
- Custodians.
- Exchanges.
- Token issuers.
- Auditors.
- Attackers.
- Insiders.

For each actor, record:

Actor:

Who or what is this?

Role:

What are they supposed to do?

Authority:

What transitions can they cause?

Capabilities:

What resources, timing, information, or access do they have?

Incentives:

What do they gain or lose?

Failure mode:

What happens if they are malicious, compromised, unavailable, irrational, negligent, or mistaken?

This is intentionally broader than “who can call functions.” A frontend maintainer may not call a privileged function, but can construct malicious transactions. An RPC provider may not move funds, but can distort a user’s view. A keeper may not own assets, but can be necessary for liveness.

6.5.2 Asset Inventory

Chapter 2 defined assets as enforceable claims over state. Threat modeling requires a structured asset inventory.

Include:

- Native assets.
- Tokens.
- Shares.
- LP positions.
- Debt positions.
- Collateral.
- Pending withdrawals.
- Rewards.
- Bridge messages.
- Governance power.
- Upgrade authority.
- Oracle authority.
- Validator or sequencer keys.
- Backend signing keys.
- Domain names and frontend distribution.
- Deployment secrets.
- Monitoring credentials.
- Off-chain reserves.
- Legal claims.

For each asset:

Asset:

What has value?

Representation:

Where is it recorded?

Normal transition:

How should it move or change?

Privileged transition:

Who can move, freeze, mint, burn, configure, or upgrade around it?

Invariant:

What must remain true?

Impact:

What happens if it is stolen, frozen, diluted, mispriced, duplicated, or made unrecoverable?

Do not restrict the asset inventory to user funds. Privileged keys are often the most valuable assets in the system.

6.5.3 Entry Point Inventory

Entry points are ways state can be changed or security-relevant decisions can be influenced.

On-chain entry points:

- Public functions.
- Program instructions.
- Module entry functions.
- Governance proposal execution.
- Upgrade functions.
- Admin setters.
- Oracle updates.
- Bridge message handlers.
- Permit or signature-based functions.
- Callback hooks.
- Token receiver functions.
- Fallback and receive functions.
- Batch and multicall paths.
- Liquidation functions.
- Emergency controls.

Off-chain entry points:

- Frontend transaction construction.
- RPC request/response.
- Indexer ingestion.
- Backend APIs.
- Keeper bots.
- Relay endpoints.
- Signing services.
- CI/CD pipelines.
- Package publishing.
- Governance forums.

- Multisig signing workflows.
- DNS and domain management.

For each entry point:

Entry point:

Function, instruction, API, workflow, or process.

Caller:

Who can invoke it?

Authority:

What authorization is required?

State read:

What state does it depend on?

State written:

What state can it change?

External assumptions:

What oracle, token, RPC, relayer, signer, or human process does it trust?

Failure behavior:

What happens on revert, partial failure, timeout, or non-response?

Abuse:

What can an adversarial caller do with valid inputs?

This inventory becomes the review map. It tells you where to spend attention.

6.5.4 Privileged Role Inventory

Privileged roles are authorities that can change rules, parameters, dependencies, or emergency state.

Examples:

- Owner.
- Proxy admin.
- Upgrade executor.
- Timelock proposer.
- Timelock canceller.
- Governance executor.

- Pauser.
- Guardian.
- Minter.
- Burner.
- Blacklist manager.
- Oracle admin.
- Risk parameter admin.
- Strategy manager.
- Fee recipient setter.
- Bridge validator-set admin.
- Sequencer admin.
- Backend signer.
- Deployment key.

For each privileged role:

Role:

Name and contract/program/module.

Holder:

EOA, multisig, contract, governance, DAO, backend service, or operator.

Capability:

Exact transitions authorized.

Delay:

Timelock, voting delay, challenge window, or no delay.

Bypass:

Emergency path, guardian override, admin override, or upgrade path.

Rotation:

How can it be changed?

Loss:

What happens if lost?

Compromise:

What happens if compromised?

Observability:

Can users see pending action before harm?

Role names lie by omission. A “guardian” may only pause deposits, or it may pause withdrawals

and trap funds. A “risk admin” may only lower caps, or it may raise collateral factors to unsafe values. A “governor” may only adjust fees, or it may call arbitrary targets.

Write the verbs.

6.5.5 Data and Control Flows

Traditional threat modeling often uses data-flow diagrams and trust boundaries. Microsoft STRIDE guidance frames threats around questions such as whether authentication data can be changed, whether data can be read, or whether access can be denied, and categorizes threats as spoofing, tampering, repudiation, information disclosure, denial of service, and elevation of privilege.⁶

Blockchain systems need both data-flow and control-flow views.

Data flows:

- Oracle price -> lending market.
- Event log -> bridge relayer -> destination chain.
- RPC response -> frontend -> wallet.
- Governance proposal JSON -> forum -> UI -> voters.
- Indexer state -> dashboard -> user decision.
- Transaction simulation -> wallet display.
- Keeper observation -> liquidation transaction.

Control flows:

- User transaction -> contract call -> external token hook -> callback.
- Governance vote -> timelock -> proxy admin -> implementation change.
- Admin setter -> oracle address -> price-dependent borrow path.
- Sequencer ordering -> liquidation execution -> solvency.
- Multisig approval -> upgrade -> storage layout migration.
- Relayer submission -> proof verification -> mint.

For each flow:

What crosses the boundary?

Who can alter it?

Who verifies it?

What state transition depends on it?

What if it is stale, false, delayed, missing, or reordered?

⁶Microsoft, “Microsoft Threat Modeling Tool threats”, <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>.

6.5.6 The Minimum Map

Before reviewing a serious protocol, you should be able to fill this table.

Category	Minimum questions
Assets	What can be stolen, frozen, diluted, mispriced, or made unrecoverable?
Actors	Who can interact, operate, influence, or attack?
Entry points	What can change state or influence a state-changing decision?
Privileged roles	Who can change parameters, dependencies, code, or emergency state?
Trust boundaries	What does the system rely on but not verify locally?
External dependencies	Which tokens, oracles, chains, bridges, relayers, RPCs, indexers, and humans matter?
Invariants	What must remain true across transitions?
Liveness paths	What required transitions must remain possible?
Monitoring	What must be observed to detect assumption failure?

If a team cannot answer these questions, the team is not ready for a deep audit. It is still discovering the system.

6.6 Separating Users, Attackers, Rational Participants, and Trusted Operators

6.6.1 Do Not Use One Generic Attacker

“The attacker” is often too broad to be useful.

Different adversaries have different powers:

- A normal user can call public entry points.
- A bot can call them faster and more often.
- A searcher can simulate pending transactions and pay for ordering.
- A validator or sequencer can influence inclusion and ordering.
- A governance whale can pass proposals.
- An admin can call privileged functions.

- A compromised frontend can construct malicious transactions.
- A malicious token can behave unexpectedly during integration.
- A bridge validator set can sign false messages.
- A custodian can delay or block withdrawals.
- An insider can access keys, deployments, secrets, or configs.
- A state-level actor may have long time horizons, supply-chain reach, and legal or coercive power.

NIST SP 800-30 recommends identifying threat sources and characterizing adversarial sources by factors such as capability, intent, and targeting.⁷

For blockchain systems, also add:

- Capital.
- Liquidity.
- Ordering access.
- Governance power.
- Cross-domain reach.
- Operational access.
- Ability to wait.
- Ability to take reputational or legal risk.

Threat modeling improves when each actor has a realistic capability set.

6.6.2 Users Can Be Adversarial Without Being “Hackers”

In permissionless systems, any user may call public functions with valid inputs. That does not make them a hacker. It makes them a participant.

Examples:

- A trader submits a sandwich.
- A liquidator triggers liquidation at the worst moment for a borrower.
- A borrower maximizes leverage.
- A depositor withdraws just before losses are realized.
- A governance voter sells votes.
- A user creates thousands of addresses.
- A keeper ignores unprofitable upkeep.

These may be rational behaviors inside the rules.

Security model:

⁷NIST, “SP 800-30 Revision 1: Guide for Conducting Risk Assessments”, threat-source identification and characterization, <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf>.

Users are not trusted to follow product intent.
They are trusted only to satisfy validity rules and pay costs.

This is why business logic must be robust against adversarial but valid use.

6.6.3 Rational Participants Are Not Byzantine Actors

A rational actor pursues payoff. A Byzantine actor may behave arbitrarily, including actions that appear irrational, malicious, inconsistent, or self-harming.

This distinction matters.

Rational searcher:

Will reorder or backrun if profit exceeds cost.
Probably will not burn money without benefit.

Byzantine validator:

May censor, equivocate, or violate protocol rules if capable,
even if immediate profit is unclear.

Compromised key:

Can sign any message the key is authorized to sign.
The original owner's incentives no longer matter.

Negligent operator:

May fail to update, rotate, monitor, or respond.
The failure may not be malicious.

The mitigation differs:

- Rational attacks require economic bounds.
- Byzantine attacks require validation, quorum assumptions, proofs, or slashing.
- Compromise requires key management and blast-radius reduction.
- Negligence requires process controls, automation, monitoring, and recovery paths.

Do not prescribe one defense for different fault models.

6.6.4 Trusted Operators Are Threat Sources Too

Trusted operators are actors the system relies on:

- Admin multisig signers.
- Governance delegates.

- Oracle operators.
- Bridge validators.
- Sequencer operators.
- Relayers.
- Keepers.
- Backend signers.
- Frontend maintainers.
- DevOps engineers.
- Custodians.
- Token issuers.

They may fail in several ways:

- Malicious action.
- Key compromise.
- Downtime.
- Misconfiguration.
- Slow response.
- Collusion.
- Legal compulsion.
- Insider abuse.
- Incorrect deployment.
- Poor monitoring.

Treating trusted operators as outside the threat model is a classic mistake.

Better:

The oracle committee is trusted to report prices within acceptable deviation and availability bounds. The protocol fails safely if one publisher is wrong, but may become unsafe if a quorum signs a bad price or if updates stop for more than 30 minutes during high volatility.

This names both safety and liveness assumptions.

6.6.5 Insiders Have Different Access

Insiders may have access that outsiders do not:

- Unreleased code.
- Deployment keys.
- Admin interfaces.

- CI/CD secrets.
- Monitoring credentials.
- Backend databases.
- Signing services.
- Governance proposal drafts.
- Oracle update infrastructure.
- Private relay endpoints.
- Multisig coordination channels.
- Incident response plans.

Insider threat modeling asks:

What can one insider do?

What requires two insiders?

What is logged?

What is reviewed?

What can be done without on-chain visibility?

What can be done before users can react?

This is not paranoia. It is basic authority mapping.

6.6.6 Actor Table Template

Use this table.

Actor	Normal role	Capabilities	Incentives	Failure mode	Controls
User	Deposit/withdrawal	Public calls, signatures	Utility/profit	Adversarial sequencing, phishing	Input checks, slippage, limits
Searcher	Extract MEV	Simulation, bundles, priority fees	Profit	Ordering attack	MEV-aware design, private flow assumptions
Admin multisig	Emergency/upgrade	Privileged calls	Protocol maintenance	Collusion, compromise, wrong signing	Timelock, threshold, monitoring
Oracle	Price data	Signed updates	Correct reporting	Stale/manipulated price	Deviation checks, circuit breakers

Actor	Normal role	Capabilities	Incentives	Failure mode	Controls
Keeper	Maintenance	Calls upkeep functions	Fees/rewards	No liveness if unprofitable	Incentives, fallback, permissionless calls

Fill it with actual actors. Generic roles are placeholders, not conclusions.

6.7 Identifying Trust Assumptions and External Dependencies

6.7.1 Assumptions Are Security Debt

Every system has assumptions. The problem is not having assumptions. The problem is hiding them.

Examples:

The oracle price is fresh.
 The bridge source event is final.
 The token does not reenter.
 The admin will not upgrade maliciously.
 The sequencer will not censor exits indefinitely.
 The relayer will submit messages.
 The keeper reward is enough.
 The frontend points to the correct contracts.
 The RPC provider reports current chain state.
 The stablecoin issuer honors redemption.

Each assumption is a place where the system is not self-contained.

Write assumptions in a falsifiable form:

The protocol assumes Chainlink ETH/USD updates at least every 1 hour and that the returned answer is not stale, negative, or outside configured deviation bounds.

Not:

We use a secure oracle.

6.7.2 Dependency Inventory

Common blockchain dependencies:

Dependency	Common assumption	Failure
Base chain	Finality, liveness, valid execution	Reorg, halt, censorship
Token contract	Standard behavior	Fee-on-transfer, blacklist, rebase, hook
Oracle	Fresh and manipulation-resistant data	Bad price, stale price, missing update
AMM/liquidity	Price reflects executable market	Low-liquidity manipulation
Governance	Non-captured decision process	Malicious proposal, bribery, low quorum
Admin multisig	Honest and available signers	Collusion, compromise, lost keys
Bridge	Correct message verification	Forged messages, replay, finality mismatch
Sequencer	Inclusion/liveness assumptions	Censorship, downtime, ordering extraction
RPC	Correct view of chain	Incorrect responses, censorship, lag
Indexer	Correct interpretation	Desync, missed reorg, event misread
Frontend	Correct transaction construction	Malicious approvals, wrong addresses
Backend signer	Correct policy enforcement	Unauthorized withdrawals
Keeper	Timely maintenance	Stuck positions, bad debt
CI/CD	Integrity of deployment	Malicious release, wrong address
Custodian	Asset safekeeping	Insolvency, freeze, withdrawal halt

Each row can become a threat scenario.

6.7.3 Safety vs Liveness Assumptions

Separate assumptions that preserve safety from assumptions that preserve liveness.

Safety assumptions:

- Oracle price is not manipulable enough to cause undercollateralized borrowing.
- Bridge validator quorum does not sign false messages.

- Upgrade admin cannot bypass timelock.
- Token mint authority cannot create unbacked supply.
- Contract verifier rejects malformed proofs.

Liveness assumptions:

- At least one keeper will liquidate unhealthy positions.
- Relayers will submit bridge messages.
- Users can withdraw during an exit window.
- Sequencer downtime fallback works.
- Governance can execute emergency fixes.
- Proof generation service remains available.

Some assumptions affect both:

- Sequencer censorship can block exits and affect liquidation ordering.
- Oracle downtime can freeze borrowing and liquidations.
- Admin key loss can prevent emergency response.
- Bridge relayer compromise can become safety risk if relayers are trusted for message truth.

For each assumption, write:

Type:

Safety, liveness, or both.

Failure impact:

What bad state or stuck state results?

Detection:

How do we know it failed?

Response:

What can the system do?

6.7.4 External Contracts Are Dependencies, Not Background

If your protocol calls another contract, integrates a token, deposits into a strategy, reads an oracle, or relies on a registry, that external system is inside the threat model as a dependency.

External contract risks:

- Upgradeable dependency.
- Reentrancy or callbacks.

- Nonstandard token behavior.
- Incorrect decimals.
- Pausable or blacklistable token.
- Fee-on-transfer or rebasing token.
- Governance-controlled parameter changes.
- Oracle update delays.
- Liquidity migration.
- Contract selfdestruct or deprecated behavior.
- Address substitution.

Dependency review questions:

Can the dependency change behavior?

Who controls it?

Can it call back?

Can it return false, no data, malformed data, or stale data?

Can it pause or freeze?

Can it become illiquid?

Can it censor or reject this protocol?

Can it break accounting assumptions?

Do not integrate an external contract as if it were a math function.

6.7.5 Human Dependencies Are Real Dependencies

Human processes are often the hidden critical path:

- Someone notices an oracle failure.
- Someone signs a pause transaction.
- Someone rotates a compromised key.
- Someone updates the frontend.
- Someone submits the governance proposal.
- Someone pays relayer gas.
- Someone checks deployment addresses.
- Someone verifies a multisig transaction.

Threat model them.

Questions:

Who is responsible?

What signal triggers action?

How fast must action happen?

What if the person is unavailable?

What if signers disagree?

What if the runbook is wrong?

What if the attacker acts faster?

What if the emergency control itself is compromised?

If the system's safety depends on a human responding in five minutes at 3 AM, write that assumption plainly.

6.7.6 Assumption Register

Use an assumption register.

ID	Assumption	Type	Dependency	Failure impact	Detection	Mitigation
A1	Oracle updates within 30 minutes	Safety/liveness	Oracle network	Bad debt or market freeze	Price age alert	Staleness checks, pause
A2	3-of-5 admin multisig not compromised	Safety	Signers	Malicious upgrade	Timelock monitor	Higher threshold, delay
A3	Relayer submits messages within 1 hour	Liveness	Relayer network	Stuck with-drawals	Queue age alert	Permissionless relay
A4	Token has no transfer fee	Safety	Collateral token	Accounting drift	Balance delta check	Measure actual received

An assumption register is valuable because it gives reviewers a target. Each assumption can be challenged, tested, monitored, or accepted explicitly.

6.8 Mapping Attack Surfaces Across Chain, Contract, Infrastructure, and Human Layers

6.8.1 Blockchain Attack Surface Is Layered

Blockchain security fails across layers:

Chain layer:

Consensus, finality, reorgs, censorship, block production.

Transaction/order layer:

Mempools, MEV, fees, ordering, inclusion, replacement.

Execution layer:

VM semantics, gas/compute limits, account model, callbacks.

Contract/protocol layer:

Authorization, accounting, external calls, upgrades, invariants.

Economic layer:

Liquidity, oracle manipulation, incentives, governance capture.

Cross-domain layer:

Bridges, rollups, relayers, proof systems, finality mismatch.

Infrastructure layer:

RPCs, indexers, frontends, backends, CI/CD, dependencies.

Human layer:

Multisig signing, incident response, custody, social engineering.

Many real exploits cross layers.

Example:

Low-liquidity AMM price

- > oracle manipulation
- > lending borrow
- > bad debt
- > governance emergency pause
- > users unable to withdraw

If your threat model only covers Solidity code, you miss the exploit path.

6.8.2 STRIDE as a Prompt, Not a Cage

STRIDE is useful as a prompt:

- Spoofing.
- Tampering.
- Repudiation.
- Information disclosure.
- Denial of service.
- Elevation of privilege.

Microsoft describes STRIDE as a way to categorize threats and ask pointed questions about authentication, data modification, data exposure, service denial, and privilege escalation.⁸

Blockchain adaptation:

STRIDE category	Blockchain examples
Spoofing	Forged signer, fake token, fake frontend, fake bridge message, fake oracle
Tampering	Storage corruption, malicious upgrade, manipulated price, altered calldata
Repudiation	Unclear signer intent, missing audit logs, unverifiable admin process
Information disclosure	Leaked private order flow, exposed xpub, leaked signing policy, metadata deanonymization
Denial of service	Stuck withdrawals, gas griefing, account locking, sequencer censorship, proof too large
Elevation of privilege	Missing access control, role-admin abuse, governance capture, proxy admin takeover

STRIDE should not replace blockchain-specific reasoning. It should help you ask what can go wrong from several angles.

6.8.3 Contract and Program Attack Surface

Contract/program surfaces include:

- Public functions and instructions.
- External calls.
- Delegatecall or equivalent dynamic dispatch.
- Token transfers.
- Callbacks and hooks.

⁸Microsoft, “Microsoft Threat Modeling Tool threats”, <https://learn.microsoft.com/en-us/azure/security/develop/threat-modeling-tool-threats>.

- Signature verification.
- Access control.
- Upgradeability.
- Initialization.
- Storage layout.
- Math and rounding.
- Oracle reads.
- Event emission.
- Batch/multicall.
- Fallback logic.
- Account constraints.
- Resource limits.

OWASP’s Smart Contract Top 10 includes categories such as access control, business logic, oracle manipulation, flash-loan-facilitated attacks, input validation, unchecked external calls, arithmetic errors, reentrancy, and upgradeability risk.⁹

Use such lists as prompts, not as a finish line. A protocol can avoid every named category and still fail economically because its invariant is wrong.

6.8.4 Chain and Ordering Attack Surface

Chain-layer and ordering surfaces include:

- Reorgs.
- Delayed finality.
- Chain halts.
- Censorship.
- Mempool visibility.
- Transaction replacement.
- Priority fees.
- Builder/searcher behavior.
- Sequencer downtime.
- Forced inclusion.
- Cross-domain settlement delay.

Threat questions:

Can a profitable actor change ordering?

Can a transaction be sandwiched?

⁹OWASP, “Smart Contract Top 10”, <https://owasp.org/www-project-smart-contract-top-10/>.

Can a liquidation be delayed?
Can a bridge event be reorged out?
Can exits be censored?
Can a user rely on state before settlement?
Can a governance proposal be front-run or blocked?

If the protocol depends on timely or private execution, ordering is part of the threat model.

6.8.5 Economic Attack Surface

Economic security is not optional in blockchain systems because assets and incentives are native to the execution environment.

Economic surfaces include:

- Liquidity depth.
- Oracle reference markets.
- Collateral factors.
- Liquidation incentives.
- Auction design.
- Reward emissions.
- Governance token distribution.
- Vote borrowing and bribery.
- Peg mechanisms.
- Withdrawal queues.
- Restaking correlation.
- Insurance funds.
- Bad debt handling.

Threat questions:

How much capital does the attack require?
Can flash liquidity reduce that capital requirement?
Can the attacker manipulate a price only briefly?
Can the attacker profit atomically?
Who absorbs losses?
Can a rational actor grief for indirect profit?
Can incentives fail during market stress?

Economic attack surface cannot be found by scanning code alone.

6.8.6 Infrastructure Attack Surface

Infrastructure surfaces include:

- Frontend hosting.
- DNS.
- Wallet connectors.
- RPC providers.
- Indexers.
- Backend APIs.
- Signing services.
- CI/CD.
- Package dependencies.
- Container images.
- Monitoring tools.
- Admin dashboards.
- Cloud accounts.
- Secrets management.

OWASP’s Web3 attack-vector material explicitly goes beyond smart contracts and includes risks such as multisig hijacking, supply-chain attacks, private key compromise, UI/UX spoofing and approval phishing, centralized infrastructure breaches, DNS and routing hijacking, wallet compromise, insider threats, and malicious open-source contributions.¹⁰

This matters because many “protocol” losses happen outside protocol code.

Threat questions:

Can a compromised frontend construct malicious approvals?

Can DNS hijack redirect users?

Can an indexer desync make users take unsafe actions?

Can backend signers authorize withdrawals?

Can CI/CD deploy wrong contracts?

Can a package update exfiltrate keys?

Can RPC censorship hide pending danger?

If an off-chain component can cause users or operators to authorize harmful transitions, it is in scope.

6.8.7 Human and Governance Attack Surface

Human surfaces include:

- Multisig review.
- Key backups.

¹⁰OWASP Smart Contract Security, “Alternate Top 15 - Web3 Attack Vectors Beyond Smart Contracts”, <https://scs.owasp.org/sctop10/Web3-Attack-Vectors-Top15/>.

- Deployment ceremonies.
- Governance discussion.
- Vote delegation.
- Emergency response.
- Legal and compliance controls.
- Operator incentives.
- Public communications.
- Support channels.
- Social engineering.

Governance surfaces include:

- Proposal thresholds.
- Quorum.
- Voting power distribution.
- Delegation.
- Timelock.
- Execution payloads.
- Cancellation rights.
- Emergency veto.
- Treasury control.
- Governance self-upgrade.

Threat questions:

Can governance call arbitrary targets?

Can voting power be borrowed?

Can delegates be bribed?

Can a proposal hide malicious calldata?

Can signers verify what they approve?

Can an attacker exploit public incident response?

Can a social-engineered employee deploy malicious code?

The human layer often holds the keys to the machine.

6.9 Producing a Threat Model That Guides Review Instead of Decorating It

6.9.1 The Threat Model Must Produce Work

A useful threat model produces review tasks.

For each important threat, the model should say:

Threat:

What can go wrong?

Priority:

Why does it matter more or less than others?

Review target:

Which code, config, process, or dependency must be inspected?

Invariant:

What property should be written or tested?

Mitigation:

What design control prevents or limits the threat?

Residual risk:

What remains after mitigation?

Monitoring:

How would we detect failure?

If a threat model does not affect review priorities, it is probably decoration.

6.9.2 Prioritize by Impact, Feasibility, and Uncertainty

NIST SP 800-30 frames risk around threat events, threat sources, vulnerabilities or predisposing conditions, likelihood, and impact.¹¹ It also notes that impact determinations should make assumptions, sources, methods, and rationale explicit.¹²

For blockchain security review, use practical factors:

- Funds at risk.
- Privilege impact.
- Solvency impact.
- Governance impact.
- Liveness impact.
- Cross-domain contagion.
- Attacker capital required.
- Exploit reliability.
- Preconditions.

¹¹NIST, “SP 800-30 Revision 1: Guide for Conducting Risk Assessments”, likelihood determination, <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf>.

¹²NIST, “SP 800-30 Revision 1: Guide for Conducting Risk Assessments”, impact determination, <https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-30r1.pdf>.

- Time sensitivity.
- Detectability.
- Recovery difficulty.
- Uncertainty.

Uncertainty deserves emphasis. A high-impact area with unclear assumptions should get review attention even before a known bug exists.

Risk shorthand:

High priority:

High impact + plausible path + unclear or weak controls.

Medium priority:

Meaningful impact + constrained path or strong controls.

Low priority:

Low impact, implausible path, or well-constrained residual risk.

Research priority:

Impact potentially high but model uncertain.

Do not pretend you can reduce every blockchain threat to a neat numeric score. Use scoring to communicate, not to replace judgment.

6.9.3 Turn Threats Into Invariants and Tests

Threat:

Attacker manipulates vault share price by donating assets before a victim deposit.

Invariant:

Depositing assets should mint shares proportional to economic contribution,
and no direct asset transfer should let an attacker force later deposits to mint
zero or unfairly low shares.

Tests:

first depositor fuzzing
direct donation before deposit
small deposit rounding
large asset balance / tiny share supply
differential comparison with ideal math model

Threat:

Admin upgrades vault to malicious implementation during timelock.

Invariant or security property:

Users can withdraw before a delayed upgrade executes, unless withdrawals are paused only under documented emergency rules.

Tests/review:

simulate upgrade queue and execution
 verify timelock delay
 verify pause cannot block exit indefinitely without escalation
 verify monitoring can detect queued implementation

The threat model should generate the testing plan.

6.9.4 Decide Mitigation, Acceptance, Transfer, or Elimination

Not every risk is fixed the same way.

Possible responses:

- Eliminate the feature.
- Redesign the transition.
- Add validation.
- Add invariant checks.
- Add rate limits.
- Add timelock.
- Add circuit breaker.
- Add monitoring.
- Restrict supported assets.
- Add independent oracle.
- Add fallback path.
- Increase multisig threshold.
- Improve signing workflow.
- Add tests or formal specifications.
- Document and accept residual risk.
- Transfer risk through insurance or legal process.

OWASP's threat-modeling guidance includes identifying countermeasures or managing risk after identifying what can go wrong.¹³

For each material threat, write the decision:

Mitigate:

Add staleness and deviation checks.

¹³OWASP, "Threat Modeling", https://owasp.org/www-community/Threat_Modeling.

Accept:

Governance can change collateral factor after timelock; users accept governance risk.

Eliminate:

Do not support fee-on-transfer collateral.

Monitor:

Alert if oracle age > 30 minutes or market price deviates > 2 percent.

Unwritten acceptance is not risk management. It is amnesia.

6.9.5 Threat Model Deliverable Template

A useful threat model can be concise if it is structured.

1. Scope

Included systems:

Excluded systems:

Review goals:

2. System summary

State machines:

Major flows:

Deployment addresses:

3. Assets

User assets:

Protocol assets:

Control assets:

Off-chain claims:

4. Actors

Users:

Operators:

Privileged roles:

External participants:

Adversaries:

5. Entry points

On-chain:

Off-chain:

Human/operational:

6. Trust boundaries

Oracles:
 Bridges:
 RPC/indexers:
 Frontend/backend:
 Governance/admin:
 Custody/legal:

7. Assumptions

Safety:
 Liveness:
 Economic:
 Operational:

8. Threat scenarios

Threat:
 Preconditions:
 Attack path:
 Impact:
 Existing controls:
 Gaps:
 Priority:

9. Review plan

Code areas:
 Invariants:
 Tests:
 Manual review questions:
 Monitoring requirements:

10. Residual risk

Accepted:
 Mitigated:
 Requires decision:

This template is not bureaucracy. It is a compression format for security reasoning.

6.9.6 Anti-Patterns

Avoid these.

Anti-pattern	Why it fails
Bug-list threat model	Names vulnerabilities without modeling system-specific attack paths
Diagram-only threat model	Looks serious but does not drive tests or review

Anti-pattern	Why it fails
Contract-only scope	Misses oracles, governance, frontends, keepers, bridges, and keys
Honest-user model	Reviews product flows instead of adversarial transitions
“Decentralized” as assumption	Replaces precise validator, governance, oracle, or admin assumptions with a slogan
Ignoring liveness	Preserves accounting while trapping funds
Treating trusted roles as safe	Hides the largest authority edges
No update process	Model becomes stale after deployment or governance change
Equal-priority threats	Fails to guide scarce review effort
No residual-risk statement	Leaves users and teams unclear about what remains trusted

The Threat Modeling Manifesto values finding and fixing design issues over checkbox compliance, and doing threat modeling over talking about it.¹⁴ That is the standard to use here.

6.9.7 What Good Looks Like

A good threat model should let a reviewer say:

```
I know where value is.
I know who can affect it.
I know which assumptions matter.
I know which assumptions are weakest.
I know which state transitions deserve the deepest review.
I know which invariants to write.
I know which tests should exist.
I know which external systems can break this protocol.
I know what risks the team is accepting.
```

If the threat model does not create that clarity, keep working.

6.10 Worked Example: Threat Modeling a Lending Market

Consider a simplified lending market:

```
Users deposit collateral.
Users borrow stablecoin up to a collateral factor.
```

¹⁴Threat Modeling Manifesto, <https://www.threatmodelingmanifesto.org/>.

Oracle prices determine account health.
 Liquidators repay debt and receive collateral at a discount.
 Governance can list collateral assets and set parameters.
 An admin multisig can pause markets and upgrade contracts after a timelock.
 Keepers monitor accounts and submit liquidations.

6.10.1 Step 1: Scope

Included:

- Lending market contracts
- Oracle integration
- Collateral token integration
- Liquidation logic
- Governance parameter changes
- Upgrade and pause controls

Excluded but documented:

- Frontend implementation
- Stablecoin issuer reserves
- External AMM liquidity sources

6.10.2 Step 2: Assets

Asset	Representation	Failure
Collateral	Token balances and account collateral records	Theft, mispricing, frozen withdrawal
Debt	Borrow balances and interest indexes	Bad debt, incorrect repayment
Stablecoin liquidity	Stablecoin balances and mint/transfer ability	Insolvency, liquidity shortage
Liquidation right	Public liquidation function	Griefing, unfair seizure, failed liquidation
Governance control	Voting and timelock state	Capture, unsafe listing
Upgrade authority	Proxy admin/timelock	Malicious implementation
Oracle price	Oracle feed and cached values	Manipulated or stale health checks

6.10.3 Step 3: Actors

Actor	Capability
Depositor	Supply collateral, withdraw
Borrower	Borrow, repay, increase leverage
Liquidator	Repay debt and seize collateral
MEV searcher	Order liquidation and oracle-update transactions
Governance	List assets, set factors, change oracle
Admin multisig	Pause and upgrade
Oracle operator	Publish price
Keeper	Monitor and call liquidations
Token issuer	Freeze or alter token behavior if token supports it

6.10.4 Step 4: Entry Points

```

deposit(asset, amount)
withdraw(asset, amount)
borrow(asset, amount)
repay(asset, amount)
liquidate(account, repayAmount)
setCollateralFactor(asset, factor)
setOracle(asset, oracle)
pauseMarket(asset)
upgradeTo(implementation)

```

6.10.5 Step 5: Assumptions

Assumption	Type	Failure
Oracle price is fresh and manipulation-resistant	Safety	Undercollateralized borrow
Liquidators act when profitable	Liveness	Bad debt grows
Collateral tokens behave as integrated	Safety	Accounting drift or freeze
Governance cannot be cheaply captured	Safety	Malicious parameters
Pause cannot permanently trap users without documented authority	Liveness	Frozen funds

Assumption	Type	Failure
Upgrade timelock gives users a real exit window	Safety/liveness	Malicious upgrade before exit

6.10.6 Step 6: Threat Scenarios

Threat:

Oracle manipulation enables bad debt.

Attack path:

Attacker manipulates low-liquidity reference market.

Oracle reads manipulated price.

Attacker deposits inflated collateral.

Attacker borrows maximum stablecoin.

Price normalizes.

Account becomes insolvent.

Controls:

TWAP or robust oracle

staleness checks

collateral caps

borrow caps

liquidation incentives

post-borrow health check

monitoring for price deviation

Threat:

Governance lists unsafe collateral.

Attack path:

Attacker accumulates or borrows voting power.

Proposal lists illiquid token with high collateral factor.

Attacker manipulates token price or mints supply.

Attacker borrows stablecoin and leaves bad debt.

Controls:

listing framework

governance timelock

collateral caps

oracle review

liquidity requirements

risk committee veto

monitoring for proposal payloads

Threat:

Liquidation liveness fails during market stress.

Attack path:

Prices fall quickly.

Many accounts become unhealthy.

Liquidation transactions become expensive or unprofitable.

Keepers stop executing.

Bad debt accumulates.

Controls:

liquidation incentives sized for stress

permissionless liquidation

batch limits

keeper redundancy

bad debt monitoring

emergency parameter controls

6.10.7 Step 7: Review Plan

Review priorities:

1. Oracle integration and staleness/deviation handling.
2. Collateral and debt accounting invariants.
3. Liquidation math and liveness under stress.
4. Governance/admin authority and timelock bypasses.
5. Token integration assumptions.
6. Upgrade and pause behavior.

Invariants:

total debt accounting matches user debts plus indexes

borrow cannot leave account below required health at execution time

liquidation cannot seize more collateral than rules allow

withdraw cannot make account unhealthy

collateral caps cannot be exceeded

paused state does not create undocumented permanent fund lock

Tests:

manipulated oracle price

stale oracle price

fee-on-transfer collateral

collateral decimals mismatch

rapid price drop with many liquidations

governance parameter edge cases
upgrade queued while withdrawals are paused

This is a threat model that guides work. It tells the reviewer where to look and what properties to test.

6.11 Practical Discipline: The One-Page Threat Model

Before a deep review, write one page with:

Protocol goal:

Assets:

Critical transitions:

Privileged roles:

External dependencies:

Top safety assumptions:

Top liveness assumptions:

Most plausible profitable attack:

Worst privileged-role failure:

Most fragile dependency:

Top five review priorities:

If this page cannot be written, the protocol is not understood yet.

6.12 Review Questions

1. What should a blockchain threat model produce besides a list of possible attackers?
2. Why should assets, actors, entry points, privileged roles, and trust boundaries be mapped before naming bug classes?
3. What is the difference between a user, attacker, rational participant, trusted operator, and privileged actor?
4. Why is an external dependency a security assumption rather than background infrastructure?
5. How does an attack tree help convert a vague threat into review priorities?

6. What makes a threat model useful during review rather than decorative?
7. Why should a threat model include both safety and liveness assumptions?
8. How should a reviewer turn threat-model output into tests, invariants, mitigations, and monitoring?

6.13 Applied Exercises

6.13.1 Exercise 1: One-Page Threat Model

Given a protocol description, produce a one-page threat model with:

- protocol goal;
- assets;
- actors;
- critical transitions;
- privileged roles;
- external dependencies;
- top safety assumptions;
- top liveness assumptions;
- top five review priorities.

Expected output: a one-page artifact, not an essay.

6.13.2 Exercise 2: Data and Control Flow

Draw a data/control-flow diagram for one of the following:

- lending market with oracle and liquidators;
- vault with strategist and withdrawal queue;
- bridge with relayers and validator signatures.

For each arrow, state what crosses the boundary, who can modify it, who verifies it, and what happens if it is stale, false, missing, delayed, replayed, or reordered.

Expected output: one diagram and one trust-boundary table.

6.13.3 Exercise 3: Attack Tree to Review Plan

Start with this goal:

Cause the protocol to lose assets or freeze user exits.

Build an attack tree with at least three branches. Then convert it into a review plan with depth budgets.

Expected output: attack tree plus a prioritized review plan.

6.13.4 Exercise 4: Threat Model Critique

Critique this weak statement:

`The system is safe because only trusted actors can perform sensitive actions.`

Rewrite it as concrete assumptions about roles, keys, thresholds, timelocks, observability, and recovery paths.

Expected output: an assumption register and one residual-risk paragraph.

6.14 Key Takeaways

- Threat modeling turns a protocol description into a security model that guides review.
- A useful threat model connects system scope, assets, actors, entry points, assumptions, threats, mitigations, and tests.
- Blockchain threat models must include chain behavior, transaction ordering, economic incentives, external dependencies, infrastructure, and human operations.
- Users are not trusted to follow product intent. Public entry points must survive adversarial but valid use.
- Rational participants, Byzantine actors, compromised keys, insiders, and negligent operators require different controls.
- Trust assumptions should be written in falsifiable form and classified by safety, liveness, or both.
- Attack surfaces exist across chain, contract, economic, cross-domain, infrastructure, and human layers.
- STRIDE and vulnerability lists are prompts, not substitutes for system-specific reasoning.
- A good threat model produces review priorities, invariants, tests, mitigations, monitoring, and residual-risk decisions.
- A threat model that does not change what gets reviewed or tested is decoration.

6.15 Further Reading

- OWASP, “Threat Modeling”. Good general overview of threat-modeling goals, lifecycle, and the four-question framework.
- “Threat Modeling Manifesto”. Concise principles and anti-patterns for practical threat modeling.

- Microsoft, “Microsoft Threat Modeling Tool threats”. Practical overview of STRIDE threat categories.
- NIST, “SP 800-30 Revision 1: Guide for Conducting Risk Assessments”. Useful for threat sources, likelihood, impact, and risk framing.
- OWASP, “Smart Contract Top 10”. Blockchain-specific awareness list of common smart-contract risk categories.
- OWASP Smart Contract Security, “S1.3 Threat Modeling”. Smart-contract security verification requirements for identifying threats, assessing risks, and implementing mitigations.
- OWASP Smart Contract Security, “Alternate Top 15 - Web3 Attack Vectors Beyond Smart Contracts”. Useful reminder that protocol risk includes infrastructure, keys, supply chain, and social engineering.

Chapter 7

Invariants and Exploitability

7.1 Purpose

The previous chapter ended with a demand:

A threat model must produce invariants, tests, mitigations, monitoring, and residual-risk decisions.

This chapter teaches the first and most important output: invariants.

An invariant is not a slogan like “funds should be safe.” It is a property that should remain true across the states, transitions, or histories the system can actually reach. In blockchain security, the deepest bugs are often not parser bugs, syntax errors, or obviously invalid transactions. They are valid sequences of transactions that move the system into a state the protocol designers did not mean to allow.

That is the central lesson of Part I:

An exploit is often a counterexample to an intended invariant.

If the invariant is vague, the review is vague. If the invariant is wrong, the tests will prove the wrong thing. If the invariant ignores capital, timing, ordering, liquidity, privilege, and profit, it may describe a theoretical weakness but not a realistic exploit. This chapter connects those pieces.

By the end of this chapter, a reader should be able to look at a protocol description and extract concrete security properties such as:

- Total issued shares must correspond to a claim on real assets.
- No account’s collateral may be seized unless a liquidation condition is true.
- A bridge must not mint more wrapped assets than are locked or otherwise secured on the source domain.

- A withdrawal queue must not allow later users to bypass earlier users if priority is part of the design.
- A vault donation must not let an early depositor steal a later depositor's assets through rounding.
- A governance payload must not execute before the required timelock has elapsed.

Then the reader must go further:

- Can an attacker reach a violating state?
- What sequence gets there?
- How much capital is needed?
- Does the attacker need ordering control?
- Is the attack profitable after fees, slippage, gas, liquidity limits, and competition?
- Does the attack break safety, liveness, solvency, fairness, authority, or user expectation?

That second set of questions is exploitability.

7.2 Learning Objectives

After studying this chapter, you should be able to:

- Define an invariant as a property over reachable states, transitions, or histories.
- Distinguish invariants from aspirations, tests, assertions, assumptions, and implementation details.
- Write conservation, authorization, solvency, temporal, and economic invariants for blockchain systems.
- Explain why many attacks are valid state transitions that violate protocol intent.
- Translate a broken invariant into an attack hypothesis.
- Evaluate exploitability using capital, timing, ordering, liquidity, reliability, privilege, and profit.
- Recognize weak invariants that are too broad, too narrow, untestable, circular, or assumption-heavy.
- Use invariant thinking as the bridge from threat modeling to testing, fuzzing, formal verification, and manual review.

7.3 The Core Model

For Part I, use this mental model:

System:

```
state space S
initial states I
```

transition relation T
 assumptions A

Invariant:

property P that should hold over all reachable states, transitions, or histories

Security failure:

a reachable state or history that violates P under assumptions A

Exploit:

a feasible strategy that drives the system to such a violation and extracts value, control, denial, or other advantage

For a simple state invariant:

P holds initially.

Every allowed transition preserves P .

Therefore P holds in every reachable state.

This is the classic inductive invariant method.

Formal notation can be useful, but keep it tied to review work:

For all reachable states S :

$P(S) == \text{true}$

For every allowed transition T :

$P(S) == \text{true} \text{ implies } P(\text{apply}(S, T)) == \text{true}$

Counterexample:

find S and T such that $P(S) == \text{true}$ but $P(\text{apply}(S, T)) == \text{false}$

The notation is not decoration. It tells you what kind of evidence would falsify the claim. Lamport describes state machines in terms of initial states and next-state relations, then treats invariance as a correctness method over reachable states.¹ Certora's documentation makes the same proof structure practical for smart contracts: establish the invariant after construction, then check that every relevant method preserves it if it held before the method.²

In security review, however, the practical question is usually inverted:

Can I find a valid transition sequence from an initial state to a state where P is false?

That sequence is a counterexample. In blockchain, a counterexample often looks like an exploit trace.

¹Leslie Lamport, "Computation and State Machines", <https://lamport.azurewebsites.net/pubs/state-machine.pdf>.

²Certora Prover Documentation, "Invariants", invariant establishment and preservation checks, <https://docs.certora.com/en/latest/docs/cvl/invariants.html>.

7.4 What an Invariant Is and Why It Finds Deep Bugs

7.4.1 Invariants Are Properties, Not Wishes

A weak statement:

Users should not lose funds.

A better invariant:

For every user `u`, after a successful withdrawal of `s` shares, the assets transferred to `u` must be no less than `previewRedeem(s)` computed immediately before the withdrawal, except for documented fees.

Another weak statement:

Only governance can upgrade the protocol.

A better invariant:

No implementation address used by the proxy can change unless the call path originates from the governance executor after the proposal has passed, the timelock delay has elapsed, and the payload target and calldata match the queued proposal.

Another weak statement:

The bridge should be backed.

A better invariant:

For asset `a`, total wrapped supply on destination domains must be less than or equal to source-domain locked balance plus explicitly recognized canonical debt, minus completed burns and withdrawals, under the bridge's finality assumptions.

The difference is not style. It is reviewability. A wish cannot be disproved. A useful invariant can be attacked.

7.4.2 Reachability Is the Point

An invariant is not merely a true statement about the system at rest. It must hold over reachable states.

This matters because many bad reviews inspect only clean, intended states:

- The vault has normal deposits.
- The oracle price is fresh.
- The pool has healthy liquidity.
- The bridge validator set is honest.

- The admin key acts according to policy.
- The contract is called through the official frontend.

Attackers search the rest of the reachable state space:

- First deposit is one wei.
- A token charges transfer fees.
- A donation changes `balanceOf` without updating accounting.
- A borrow occurs immediately after an oracle update.
- A reentrant callback observes state before restoration.
- A liquidation races a price change.
- A bridge message is replayed across domains.
- A governance proposal executes through an unexpected call path.

Invariant thinking forces a reviewer to ask:

What can be true before this transaction?

What can this transaction change?

What must still be true after it?

7.4.3 Inductive Invariants vs Intended Invariants

An invariant can be true of the implementation and still be useless.

Example:

`totalSupply` is always equal to `totalSupply`.

This is true and worthless.

Another example:

The contract's stored '`totalDeposits`' variable never decreases except in `withdraw`.

This may be true but incomplete if token balances can change through direct transfer, rebasing, fee-on-transfer behavior, slashing, or strategy loss.

Security review cares about intended invariants:

The accounting variables that define user claims must correspond to actual recoverable assets under the token and strategy semantics the protocol supports.

The gap between the implemented invariant and the intended invariant is where deep bugs live.

7.4.4 Invariants Can Be About States, Transitions, or Histories

Most beginners think invariants are only static state predicates:

```
assetBalance >= totalUserClaims
```

Those are important, but not enough.

Blockchain systems need several kinds of properties.

Property type	Question	Example
State invariant	What must be true now?	<code>cash + borrows - reserves >= liabilities</code>
Transition invariant	What relationship must hold before and after one action?	A withdrawal burns shares before reducing total assets owed
Relational invariant	What variables must agree?	Sum of per-user balances equals total supply
Temporal property	What must eventually or never happen over time?	A queued withdrawal eventually becomes claimable if the system is unpaused
Ordering property	What must happen before what?	A governance payload cannot execute before its timelock
Economic invariant	What value relationship must hold?	A solvent account cannot be liquidated at the current oracle price
Observational invariant	What must external observers be able to verify?	A bridge proof must bind source chain, message nonce, sender, recipient, and amount

The safety/liveness distinction is useful here. Safety says “bad things do not happen”; liveness says “good things eventually happen.” Alpern and Schneider’s classical work gives the formal background for that split.³ In blockchain security, most theft bugs are safety failures, while stuck withdrawals, unprocessable queues, permanent censorship, and unusable exits are often liveness failures.

Do not force every property into a state invariant. Use the right shape.

7.4.5 A Broken Invariant Is an Attack Hypothesis

When you write:

³Bowen Alpern and Fred B. Schneider, “Defining Liveness”, <https://www.cs.cornell.edu/fbs/publications/DefLiveness.pdf>.

For every account, `debtValue <= collateralValue * collateralFactor`
after any borrow or withdrawal.

you are implicitly asking:

Can an attacker make `debtValue` exceed the allowed collateralized value?

That becomes an attack hypothesis:

If the attacker can manipulate the oracle price used in `collateralValue`,
then the borrow check may pass at an inflated collateral value.
After price normalization, the account may be insolvent.

When you write:

Only the owner of a position can withdraw its collateral,
unless the position is liquidatable.

you are asking:

Can the attacker make the system treat them as owner,
or make a healthy position appear liquidatable,
or call a path that bypasses this check?

This is why invariants find deep bugs. They do not ask whether a known vulnerability label appears. They ask whether the system can violate its own security claims.

7.4.6 Tools Check Properties; People Must Choose Them

Invariant testing and formal tools are valuable, but they do not decide what the protocol should mean.

Foundry describes invariant testing as checking properties that should always hold regardless of the sequence of actions, with Forge generating random call sequences and checking invariants after calls.⁴ Echidna similarly generates arbitrary transactions and reports sequences that make a property fail.⁵ Solidity’s SMTChecker documentation is blunt about the specification problem: formal verification helps compare what you specified with what you implemented, but you still must check whether the specification is what you wanted.⁶

That is the reviewer’s burden.

Bad invariant:

```
function invariant_totalAssetsNonzero() external view {
    assert(vault.totalAssets() >= 0);
}
```

⁴Foundry, “Invariant Testing”, <https://www.getfoundry.sh/guides/invariant-testing>.

⁵Trail of Bits, “Testing a Property with Echidna”, <https://secure-contracts.com/program-analysis/echidna/introduction/how-to-test-a-property.html>.

⁶Solidity documentation, “SMTChecker and Formal Verification”, <https://docs.soliditylang.org/en/latest/smtchecker.html>.

This proves nothing useful.

Better invariant:

```
function invariant_solveny() external view {
    assert(asset.balanceOf(address(vault)) + strategyRecoverableAssets()
        >= vault.totalAssetsOwedToShareholders());
}
```

Still incomplete unless:

- `strategyRecoverableAssets()` is defensible.
- The asset token behavior is modeled correctly.
- All share holders and debt-like claims are included.
- Fees, pending withdrawals, locked profit, slashing, and bad debt are accounted for.
- The invariant is checked across reachable sequences, not only a single happy path.

Tools amplify good properties. They also amplify bad ones.

7.4.7 The Invariant Quality Test

Before using an invariant in review, testing, fuzzing, or formal verification, test the invariant itself.

A useful invariant is:

- **Specific:** It names the state, asset, actor, authority, or transition it constrains.
- **Falsifiable:** A counterexample can be described.
- **Reachability-aware:** It applies to states the system can actually reach, including adversarial sequences.
- **Assumption-aware:** It states which oracle, finality, token, governance, or operator assumptions it needs.
- **Quantified:** It says whether it applies to all users, all assets, all markets, all messages, or a subset.
- **Measurable:** A reviewer or tool can evaluate it from code, state, traces, or explicit models.
- **Non-circular:** It does not merely restate the implementation variable as truth.
- **Not too weak:** Passing it would actually reduce risk.
- **Not too strong:** It does not forbid intended behavior.
- **Linked to impact:** Violating it would matter.

If an invariant fails this test, refine it before trusting the review built on it.

7.5 Conservation, Supply, and Asset-Balance Invariants

7.5.1 Conservation Is the First Accounting Question

For systems that mint, burn, lock, unlock, wrap, deposit, withdraw, swap, stake, or bridge assets, the first security question is:

Where did the value come from, and where can it go?

Conservation invariants track value across representations.

Examples:

Token:

```
totalSupply == sum(balanceOf(account) for all accounts)
```

Vault:

```
totalAssetsManaged >= assetsRedeemableByAllShares
```

Bridge:

```
wrappedSupply(destination) <= lockedAssets(source) + recognizedCredit - pendingExits
```

AMM:

```
reserves after swap must satisfy the pool's pricing invariant after fees
```

Reward distributor:

```
totalClaimed + totalUnclaimed + undistributedBalance <= totalFundedRewards
```

These are not all the same kind of invariant. Some are exact. Some are inequalities. Some depend on external facts. Some need a model because summing all accounts on-chain may not be feasible. But the underlying question is the same:

Can claims exceed backing?

7.5.2 Exact Conservation vs Bounded Drift

Do not demand exact equality when the system intentionally permits drift.

Exact:

ERC-20 totalSupply equals the sum of all balances.

Bounded:

Total rounding loss allocated to users must be less than one unit per operation and must not be accumulable by an attacker into a net profit above fees.

Intentional inequality:

Vault assets may exceed shareholder claims because donations or strategy yield can increase assets without minting shares.

Dangerous inequality:

Vault assets are less than shareholder claims because strategy losses were not recognized or withdrawal accounting double-counted assets.

Many accounting bugs come from confusing these cases.

7.5.3 Total Supply Is Not Enough

Beginners often stop at:

```
totalSupply == sum(balances)
```

That is necessary for many tokens, but it does not establish asset safety.

A wrapped token can have perfect internal supply accounting and still be unbacked. A lending market can track cToken balances correctly and still become insolvent. A vault can correctly sum shares and still issue shares at the wrong exchange rate. A bridge can correctly burn wrapped tokens and still release source assets to the wrong recipient.

Use layers:

Internal accounting:

```
totalSupply == sum(userShares)
```

Claim accounting:

```
userAssetsClaim = sharesToAssets(userShares)
```

Backing:

```
recoverableAssets >= sum(userAssetsClaim)
```

Authority:

```
only permitted transitions can change supply, claims, or backing
```

External dependency:

```
recoverableAssets calculation is valid under token, oracle, strategy, and bridge
assumptions
```

The strongest bugs often appear between layers.

7.5.4 Direct Balance Changes Are a Security Surface

Many protocols use token balances as accounting inputs:

```
assets = asset.balanceOf(address(this))
```

That may be correct. It may also be dangerous.

Token balances can change without the protocol’s deposit function:

- Direct transfers.
- Rebases.
- Fee-on-transfer behavior.
- Token hooks.
- Token recovery functions.
- Strategy returns.
- Airdrops.
- Slashing.
- Cross-chain minting or burning.

If a protocol’s accounting assumes all asset changes pass through its own functions, direct balance changes break the model.

Example invariant:

If `share price` is computed from `asset.balanceOf(vault)`,
then direct asset transfers to the vault must not allow an attacker to extract
more value from later users than the attacker contributed.

That is a real security claim, not a generic “handle donations” note.

7.5.5 Bridges Need Cross-Domain Conservation

A lock-mint bridge has a simple intended accounting story:

source chain: lock original token
destination chain: mint wrapped token

The invariant is not local to either chain:

`wrappedSupply(destination) <= lockedBalance(source)`

But real bridges complicate this:

- Multiple destination chains.
- Multiple token representations.
- Pending messages.
- Failed withdrawals.
- Finality windows.
- Validator signatures.
- Relayer retries.

- Fee deductions.
- Emergency pauses.
- Canonical debt during asynchronous settlement.

So the real invariant becomes:

For each asset `a`:

```
sum(wrappedSupply[a] across destination domains)
+ pendingUnlocks[a]
- pendingBurns[a]
<= sourceLocked[a] + explicitlyAuthorizedCredit[a]
```

under the bridge's finality and message-validity assumptions.

The assumption clause matters. A bridge invariant may be true if validators never sign fraudulent messages and false if a quorum can be compromised. That does not make the invariant useless. It tells you where the security boundary is.

7.5.6 Conservation Invariants Should Name Units

Always name units.

Bad:

```
balances match.
```

Better:

```
The sum of user principal balances denominated in underlying asset units
must be less than or equal to recoverable underlying assets.
```

Better still:

```
For USDC market m, after applying USDC's 6-decimal precision and the protocol's
index scale, the sum of user supply claims in USDC atomic units must be less
than or equal to cash + borrows - reserves - recognizedBadDebt, allowing at most
1 atomic unit of rounding error per account update.
```

Units are not clerical. Decimal mismatch, index scale mismatch, and rounding direction are exploit surfaces.

7.5.7 Common Conservation Failure Modes

Failure mode	Broken invariant
Mint without backing	Supply exceeds assets, locks, or authorized credit
Burn without releasing claim	User loses claim without receiving assets or debt reduction
Double claim	Same deposit creates multiple redeemable claims
Untracked liability	Pending withdrawals or rewards omitted from solvency calculation
Donation manipulation	Direct balance change shifts exchange rate in attacker's favor
Fee-on-transfer mismatch	Protocol credits nominal amount but receives less
Rebase mismatch	Internal accounting assumes balances are stable
Cross-domain replay	Same source event mints or unlocks more than once
Decimal mismatch	Value comparisons mix incompatible units
Rounding accumulation	Tiny errors become extractable over repeated calls

For every asset-moving feature, write at least one conservation invariant before reading the implementation.

7.6 Authorization, Ownership, and Capability Invariants

7.6.1 Authorization Is a State Property

Authorization is often taught as a function-level check:

```
require(msg.sender == owner);
```

That is too small. Authorization is a property of the state transition:

Who is allowed to cause this state change, under what conditions, through which path?

A better invariant:

No transition may reduce user *u*'s asset claim unless one of the following is true:

- u* authorized the transition,
- u*'s position satisfies the liquidation predicate,
- a documented fee was charged according to protocol rules,
- or an explicitly scoped emergency process executed.

This captures more than `msg.sender`. It includes signatures, approvals, liquidations, fee logic, governance, and emergency controls.

7.6.2 Ownership Must Be Bound to the Right Object

Ownership bugs are often object-binding bugs.

Examples:

- Checking that the caller owns some NFT, but not the NFT controlling this position.
- Checking that an account is a signer, but not that it signed this message.
- Checking that a PDA has the expected derivation, but not that it belongs to the expected program and seed domain.
- Checking that a proof is valid, but not that it proves inclusion for this chain, token, amount, recipient, and nonce.
- Checking that a governance proposal passed, but not that the executed calldata matches the proposal.

Authorization invariant:

For every protected object `o` and protected action `a`,
the authority used to authorize `a` must be bound to `o`, `a`, the target contract,
the chain or domain, and any nonce or validity window required to prevent replay.

This repeats a lesson from Chapter 5: signatures authorize messages, not intentions. The same is true for ownership checks. They authorize exactly what they bind.

7.6.3 Capabilities Are Assets

Some systems do not use role strings or owner addresses as the main authority model. They use capabilities:

- A Move resource that grants mint authority.
- A Solana PDA that controls token accounts.
- A smart account session key with scoped permissions.
- A governance executor contract that can call privileged targets.
- A multisig module that can move treasury funds.
- A validator key that can sign bridge messages.

A capability is an asset because controlling it changes what state transitions are possible.

Capability invariant:

A capability that can mint, burn, upgrade, pause, seize, withdraw, or sign cross-domain messages must only be created, transferred, delegated, or revoked through explicitly authorized transitions.

Then specialize:

Mint capability:

- Only governance can grant it.
- Granting requires timelock.
- Revoked minters cannot mint.
- The zero address cannot receive it.
- A minter cannot exceed per-epoch limits.

Session key:

- It cannot spend outside its token allowlist.
- It cannot exceed its value cap.
- It expires at or before its configured deadline.
- It cannot upgrade its own permissions.

Bridge signer:

- It cannot sign messages for unsupported domains.
- Its key rotation must not reduce threshold safety below the configured quorum.

Do not treat capabilities as “configuration.” They are part of the asset map.

7.6.4 Negative Authorization Matters

Authorization is not only “who can do X.” It is also “who cannot do X.”

Examples:

Liquidator:

- Can seize collateral only if liquidation predicate is true.
- Cannot choose arbitrary collateral unrelated to the unhealthy debt.
- Cannot seize more than close factor and incentive allow.

Guardian:

- Can pause deposits and borrows.
- Cannot withdraw user funds.
- Cannot upgrade implementation.
- Cannot unpause after a governance-defined maximum incident window, unless governance confirms.

Strategist:

- Can move assets into approved strategies.
- Cannot move assets into unapproved tokens.
- Cannot set withdrawal fees.

Cannot change oracle source.

A role without negative boundaries is not a role. It is a vague trust assumption.

7.6.5 Cross-Contract Authorization Is Where Bugs Hide

Many protocols split authority across contracts:

- Proxy admin.
- Implementation contract.
- Timelock.
- Governance executor.
- Access-control registry.
- Vault.
- Strategy.
- Oracle adapter.
- Token.
- Router.

Local checks can pass while the system-level authorization invariant fails.

Example:

The strategy checks that `caller == vault`.

The vault lets anyone call `'executeStrategyCall(bytes calldata data)'`.

The system-level effect is that anyone can make the strategy perform privileged actions.

The local invariant “strategy only trusts vault” is true. The system invariant “only authorized strategists can move strategy funds” is false.

System authorization invariant:

Every call path that reaches a privileged state change must pass through the intended authorization predicate, and no public wrapper may make that predicate trivially true.

7.6.6 Authorization Invariants Should Include Time

Time changes authority.

Examples:

- A signature expires.
- A session key expires.
- A timelock matures.

- A guardian pause expires.
- A withdrawal delay completes.
- A dispute window closes.
- A validator key rotation becomes active.

Temporal authorization invariant:

Authority granted for interval [start, end] must not authorize protected transitions before start or after end.

Governance example:

A queued proposal can execute only if:
 proposal state is succeeded,
 execution time $\geq$ queuedTime + timelockDelay,
 execution time $\leq$ expirationTime if expiration exists,
 payload hash matches the queued payload,
 and the proposal has not already been executed or canceled.

If you omit time from authority, replay and premature execution bugs become easy to miss.

7.7 Solvency, Collateralization, Reserve, and Share-Price Invariants

7.7.1 Solvency Is About Claims vs Recoverable Assets

A protocol is solvent with respect to a claim class when it can satisfy those claims under the rules and assumptions promised.

Simple:

```
recoverableAssets  $\geq$  userClaims
```

But serious protocols need precision:

```
recoverable underlying assets
 $\geq$  immediately withdrawable user claims
+ pending withdrawal claims
+ accrued but unpaid fees owed to users
+ recognized protocol obligations
```

Questions:

- Which assets count as recoverable?
- At what price?
- Under what withdrawal delay?

- With what slippage?
- Are strategy positions liquid?
- Are pending losses recognized?
- Are bad debts included?
- Are fees liabilities or protocol equity?
- Can an emergency pause delay payment without violating the stated property?

Solvency is not a single number. It is a claim about asset recoverability under stated conditions.

7.7.2 Lending Markets Need Collateralization Invariants

Lending protocols convert assets, prices, collateral factors, interest indexes, and liquidation rules into a safety condition.

Aave describes health factor as collateral value multiplied by weighted liquidation threshold divided by borrow value, with health factor below 1 signaling liquidation risk.⁷ Compound's Comptroller is its risk management layer, using balances, prices, and collateral factors to determine liquidity and shortfall.⁸

The security invariant behind these systems is:

A borrower must not be able to increase debt or withdraw collateral if the resulting account would violate the protocol's collateralization predicate.

More concrete:

After any borrow, collateral withdrawal, collateral transfer, oracle update application, liquidation, or interest accrual:

```
account is either:
  healthy according to the current risk model,
  liquidatable according to the current risk model,
  or in an explicitly recognized bad-debt/recovery state.
```

Do not write only:

```
healthFactor(user) >= 1
```

That may be false for accounts that are intentionally liquidatable. Better:

Healthy-only operations may not leave `healthFactor(user) < 1`.

Liquidation-only operations may execute only when `healthFactor(user) < 1` or according to the protocol's documented liquidation eligibility predicate.

The invariant must distinguish operation classes.

⁷Aave, "Health Factor & Liquidations", <https://aave.com/help/borrowing/liquidations>.

⁸Compound v2, "Comptroller", <https://docs.compound.finance/v2/comptroller/>.

7.7.3 Lending Invariants Depend on Oracles

Collateralization is only as good as the price model.

Bad invariant:

```
debtValue <= collateralValue * collateralFactor
```

Better:

```
debtValue(priceSnapshot) <= collateralValue(priceSnapshot) * collateralFactor
where priceSnapshot satisfies the protocol's freshness, source, deviation,
liquidity, and sequencer-availability assumptions.
```

Attack hypothesis:

If the attacker can manipulate priceSnapshot for one transaction,
then the borrow check may accept an undercollateralized account.

This is why oracle assumptions must be written into the invariant. Otherwise, a test may prove the accounting math while ignoring the exploit.

7.7.4 Reserve Invariants Separate User Claims from Protocol Equity

Many systems have reserves:

- Lending reserves.
- Insurance funds.
- AMM protocol fees.
- Vault performance fees.
- Liquidation penalties.
- Treasury-owned liquidity.

Reserve invariant:

Protocol reserves must not be counted simultaneously as user-redeemable backing
and protocol-withdrawable equity.

Double-counting reserves is a solvency bug.

Example:

```
cash + borrows - reserves >= supplierClaims
```

If reserves are omitted:

```
cash + borrows >= supplierClaims
```

the protocol may appear solvent while the same assets are promised to suppliers and the treasury.

7.7.5 Share-Price Invariants Are Subtle

Vaults, staking derivatives, LP tokens, lending receipts, and restaking positions often use shares.

General model:

`shares` represent a proportional claim on assets.

ERC-4626 standardizes tokenized vault interfaces such as `totalAssets`, `convertToShares`, and `convertToAssets`.⁹ It also specifies rounding behavior: `convertToShares` and `convertToAssets` must round down.¹⁰

That rounding direction is not just an implementation detail. It creates security properties and integration risk.

Common share-price invariants:

If no loss, fee, or donation occurs, share price should not decrease.

If a user deposits assets and immediately redeems the received shares without intervening state changes, the loss must be bounded by documented fees and rounding.

If `totalSupply > 0`, `redeem(shares)` should return assets proportional to `shares / totalSupply`, subject to fees, liquidity, and rounding.

If `totalSupply == 0`, the initial exchange rate must not be manipulable in a way that lets the first depositor confiscate later deposits.

The last property is where many real vault bugs live.

7.7.6 Donation and Inflation Attacks Are Broken Share Invariants

OpenZeppelin’s ERC-4626 documentation describes the inflation attack: an attacker deposits a small amount, donates assets directly to the vault, shifts the exchange rate, and can make a later deposit mint zero or too few shares due to rounding.¹¹

The broken invariant is not simply:

`sharePrice` increased.

Share price may legitimately increase through yield or donations. The broken property is closer to:

A third party must not be able to manipulate the deposit exchange rate so that a victim receives fewer shares than the protocol’s slippage and rounding policy allows, while the manipulator can recover the donated value through their existing shares.

⁹Ethereum EIP-4626, “Tokenized Vaults”, <https://eips.ethereum.org/EIPS/eip-4626>.

¹⁰Ethereum EIP-4626, `convertToShares` and `convertToAssets` rounding requirements, <https://eips.ethereum.org/EIPS/eip-4626>.

¹¹OpenZeppelin Contracts documentation, “ERC4626”, security concern: inflation attack, <https://docs.openzeppelin.com/contracts/5.x/erc4626>.

Attack shape:

Initial state:

```
totalAssets = 0
totalShares = 0
```

Attacker:

```
deposit 1 atomic unit, receive 1 share
donate large amount directly to vault
```

Victim:

```
deposit amount U
receive 0 or too few shares because conversion rounds down
```

Attacker:

```
redeem 1 share
withdraw most or all assets
```

Mitigations change the invariant:

- Minimum initial liquidity.
- Dead shares.
- Virtual shares and virtual assets.
- Deposit functions with minimum shares out.
- Routers that protect first deposits.
- Internal accounting that ignores direct donations.
- Higher share precision.

Each mitigation has assumptions. Do not write “prevent donation attack.” Write the exact post-mitigation property.

7.7.7 AMM Invariants Are Pricing Rules, Not Magic Shields

Uniswap documentation describes the constant product formula $x * y = k$, where x and y are reserves and k is the invariant that stays constant or increases after trades.¹²

That invariant is a pricing rule. It does not mean:

- The pool price is globally correct.
- LPs cannot lose value.
- Trades cannot be sandwiched.
- Oracles derived from the pool are safe.

¹²Uniswap Docs, “How Uniswap Works”, <https://developers.uniswap.org/docs/get-started/concepts/how-uniswap-works>.

- Fees cannot be misaccounted.
- Reserves cannot be desynchronized from token balances.

AMM invariant examples:

Swap:

```
adjustedReserveXAfter * adjustedReserveYAfter >= reserveXBefore * reserveYBefore
after accounting for fees.
```

Liquidity mint:

```
minted LP shares must be proportional to contributed reserves,
except for documented minimum liquidity and rounding.
```

Liquidity burn:

```
burned LP shares must redeem proportional reserves,
except for documented fees and rounding.
```

Reserve sync:

```
stored reserves must match actual balances after operations that update reserves,
and stale reserves must not be used as if they were fresh for security decisions.
```

The invariant tells you what kind of attack to search for:

- Can a token with transfer fees make stored reserves wrong?
- Can a callback observe reserves before update?
- Can a sync/skim mechanism move value unexpectedly?
- Can a price oracle be manipulated within one block?
- Can liquidity be minted at a favorable ratio because of rounding?

The formula is the beginning of review, not the end.

7.7.8 Solvency Invariants Must Name Failure Modes

Solvency can fail by:

- Theft.
- Bad debt.
- Mispriced collateral.
- Strategy loss.
- Rounding accumulation.
- Untracked liabilities.
- Frozen assets.
- Withdrawal queue mismatch.

- Bridge failure.
- Governance seizure.
- Liquidation malfunction.
- Oracle outage.

Different failure modes require different invariants.

Example:

Asset theft:

Unauthorized accounts cannot reduce recoverableAssets.

Bad debt:

After recognizing bad debt, supplier claims and reserve accounting must reflect the loss according to the documented socialization policy.

Frozen assets:

Assets paused in a strategy cannot be counted as immediately withdrawable.

Oracle outage:

Borrowing and liquidation behavior under stale oracle data must match the documented safety mode.

Solvency is not only “assets $\geq$ liabilities.” It is “the protocol’s accounting of assets and liabilities matches reality under each failure mode it claims to handle.”

7.8 Temporal, Liveness, Exit, Withdrawal, and Recovery Invariants

7.8.1 Not Every Security Failure Is Theft

A protocol can lose security without losing funds immediately.

Examples:

- Users cannot withdraw.
- Messages never finalize.
- A queue can be permanently blocked by one bad item.
- A bridge pause has no exit path.
- Liquidations cannot execute during market stress.
- Oracle staleness freezes borrowing and repayment.
- Governance cannot recover from a failed upgrade.
- A rollup escape hatch exists on paper but cannot be used in practice.

These are liveness and recovery failures.

Safety property:

No unauthorized withdrawal occurs.

Liveness property:

An authorized withdrawal eventually becomes claimable if required conditions remain satisfied and the system is not in a documented emergency state.

Both matter.

7.8.2 Temporal Properties Need Clocks and Conditions

Bad:

Withdrawals should be processed quickly.

Better:

If a withdrawal request is accepted at time t and the protocol remains unpaused, then the request must become claimable no later than $t + \text{withdrawalDelay} + \text{maxEpochLength}$.

Better still:

If a withdrawal request for amount a is accepted at epoch e , and sufficient liquid assets or documented exit liquidity become available, and no emergency pause is active, then the request must become claimable by the protocol's stated processing rule without later requests bypassing it except under documented priority rules.

Temporal properties always have conditions. Name them.

7.8.3 Exit Invariants Are Security Boundaries

An exit mechanism is a promise that users can leave.

Exit invariant:

A user with a valid claim can eventually convert that claim into the underlying asset or a documented settlement asset, unless a stated and bounded emergency condition applies.

For bridges:

A user who burns wrapped assets on the destination domain and submits a valid message after source-domain finality must be able to unlock source assets exactly once, unless the bridge is paused under documented recovery rules.

For rollups:

A user must be able to force inclusion or exit through L1 under the rollup's published censorship-resistance assumptions.

For vaults:

A user who owns shares must be able to redeem according to liquidity, queue, fee, and strategy-withdrawal rules, and the protocol must not silently convert an immediate claim into an indefinite claim.

If an exit exists only when all operators are honest and online, then operator honesty is part of the trust model. Write that down.

7.8.4 Withdrawal Queues Need Ordering Invariants

Queues are common:

- Staking withdrawals.
- Vault redemptions.
- Bridge exits.
- Insurance claims.
- Liquidation auctions.
- Delayed settlement.

Queue invariant:

If the design is FIFO, request i must not be processed after request j when i was accepted before j , unless i is invalid, canceled, expired, or explicitly skipped according to documented rules.

Capacity invariant:

The sum of claimable withdrawals must not exceed liquid assets reserved for the withdrawal batch.

Cancellation invariant:

Canceling a withdrawal must restore exactly the user's unclaimed rights minus documented fees, and must not let the user double claim.

Griefing invariant:

No user can submit a request that permanently blocks unrelated valid requests without bearing a cost proportional to the damage or without an administrative recovery path.

Queues convert accounting bugs into time bugs. Review both.

7.8.5 Pause and Recovery Controls Need Invariants Too

Emergency controls are security-critical because they intentionally break normal flow.

Pause invariant:

When paused:

- prohibited actions must revert,
- permitted recovery actions must remain available,
- user claim accounting must not be corrupted,
- and the pause authority must not gain extra withdrawal or upgrade power unless explicitly documented.

Unpause invariant:

Unpause must require the documented authority and must not skip required post-incident checks, timelocks, or governance confirmations.

Recovery invariant:

A recovery action must move the system from a known bad or degraded state to a documented safe state without increasing total user claims, reducing user claims arbitrarily, or granting permanent emergency authority.

Bad recovery controls create second exploits after the first incident.

7.8.6 Liveness Invariants Are Hard to Test

Unit tests are often bad at liveness because liveness is about time, eventuality, and environment behavior.

Testing strategies:

- Model time explicitly.
- Advance blocks, slots, or epochs.
- Simulate paused and unpaused states.
- Simulate empty and full queues.
- Simulate failed external calls.
- Simulate partial liquidity.
- Simulate oracle downtime.
- Simulate operator absence.
- Test that recovery paths remain callable.

Formal verification can express some temporal properties, but many blockchain teams rely on stateful tests, model checking, simulations, and manual reasoning. The key is not tool purity. The key is that the property is written.

7.9 Exploitability: Capital, Timing, Liquidity, Reliability, and Profit

7.9.1 Vulnerability Is Not the Same as Exploit

A vulnerability is a weakness. An exploit is a feasible strategy that uses the weakness to achieve impact.

Example vulnerability:

The vault's first-deposit exchange rate can be manipulated by donation.

Exploitability questions:

- Is the vault empty or can it become empty?
- Can the attacker front-run the victim's deposit?
- Can the attacker donate directly to the vault?
- Does rounding mint zero or too few shares?
- Does the victim transaction set a minimum share output?
- Can the attacker redeem after the victim deposit?
- How much capital is required?
- Is the attack profitable after gas and opportunity cost?
- Can the attack be repeated?
- How many users or assets are exposed?

The same code flaw can be catastrophic in one deployment and low severity in another because the exploitability conditions differ.

7.9.2 Exploitability Has Preconditions

Always write preconditions.

Template:

Threat:

Attacker violates invariant P.

Preconditions:

Initial state:

Required attacker capital:

Required permissions:

Required timing or ordering:

Required liquidity:

Required external dependency behavior:

Required victim behavior:

Required protocol configuration:

Attack path:

Step 1:

Step 2:

Step 3:

Impact:

Direct profit:

Funds at risk:

Denial or griefing:

Governance or control impact:

Recovery difficulty:

Reliability:

Deterministic or probabilistic:

Competition:

Monitoring/detection:

Repeatability:

If you cannot fill the preconditions, you do not yet understand exploitability.

7.9.3 Capital Requirement

Capital matters, but not simplistically.

Questions:

- How much capital must be owned?
- Can it be flash-borrowed?
- Must it be held across blocks?
- Is it at market risk?
- Is it locked during a challenge window?
- Is it consumed as a cost or recoverable?
- Does the attack require scarce collateral?
- Does slippage scale with attack size?
- Does the attacker need governance voting power?

Examples:

Flash-loan feasible:

Attacker needs large temporary liquidity within one transaction.

Capital locked:

Attacker must stake assets for 7 days and faces slashing risk.

Unborrowable capital:

Attacker needs governance voting power at a historical snapshot.

Inventory risk:

Attacker must hold volatile collateral while manipulating a market over time.

“Requires a lot of capital” is not a defense unless the amount, source, duration, and risk are specified.

7.9.4 Timing and Ordering

Blockchain exploits often depend on ordering.

Questions:

- Must the attacker act before a victim transaction?
- Must the attacker act after an oracle update?
- Must several actions occur in one transaction?
- Can a private mempool or builder relationship help?
- Can a sequencer censor or reorder?
- Can a keeper race the attacker?
- Does the attack survive reorgs or finality delays?
- Is there a challenge period?
- Does the exploit require a stale price window?

Ordering-dependent exploit:

Attacker front-runs first deposit, manipulates share price, victim deposits, attacker back-runs redemption.

Temporal exploit:

Attacker borrows against collateral immediately after a manipulated oracle update and before the price reverts.

Liveness exploit:

Attacker fills a withdrawal queue with dust requests so large withdrawals cannot be processed before a deadline.

Write ordering as part of the exploit, not as a footnote.

7.9.5 Liquidity and Market Depth

Economic attacks need markets.

Questions:

- Is there enough liquidity to manipulate the price?
- Is there enough liquidity to exit the profit?
- Does the manipulation cost exceed the extracted value?
- Is the oracle based on spot price, TWAP, median, or signed feeds?
- How long must manipulation persist?
- Can arbitrage restore the price before the exploit completes?
- Does the attacker need to trade through fee tiers or concentrated ranges?
- Are there liquidity cliffs?

Invariant:

The oracle price used for collateral checks must not be manipulable within the capital and time bounds available to a rational attacker for profit greater than manipulation cost.

That is not directly a simple code assertion. It may require simulation, market analysis, and risk parameter review. But it is still a security property.

7.9.6 Reliability and Repeatability

A one-shot exploit with uncertain success is different from a deterministic drain.

Reliability questions:

- Does the exploit always work if preconditions hold?
- Does it depend on miner, validator, sequencer, or builder cooperation?
- Does it depend on victim behavior?
- Can it be simulated exactly on a fork?
- Can failed attempts revert without cost?
- Can competitors copy the transaction?
- Does the protocol detect or pause after the first attempt?
- Can the attacker repeat across markets, assets, chains, or users?

Repeatability often determines severity.

Single victim:

Profit limited to one pending transaction.

Per-market:

Attack repeats across all markets with same accounting flaw.

Global drain:

Attack can drain all assets controlled by the protocol.

Permanent control:

Attack grants upgrade, mint, or governance authority.

7.9.7 Profit Is Not the Only Impact

Some attacks are not profitable thefts.

They may cause:

- Permanent fund lock.
- Forced liquidation.
- Bad debt.
- Governance takeover.
- Oracle corruption.
- Chain halt.
- Bridge halt.
- Censorship.
- User deanonymization.
- Loss of accounting integrity.
- Loss of upgrade control.
- Griefing at low cost.

Security severity should not require attacker profit. A cheap griefing attack that freezes a billion-dollar withdrawal queue is serious even if the attacker cannot directly steal.

7.9.8 “Not Exploitable” Must Be Proven Narrowly

Bad statement:

This is not exploitable.

Better:

Under the current deployment, the attack requires the vault to be empty.

The vault has non-burnable dead shares and a minimum initial liquidity position.

A direct donation can change share price, but deposits route through a function that enforces minimum shares out. Therefore the specific zero-share first-deposit attack is not feasible unless those conditions change.

This statement is useful because it names:

- Deployment condition.
- Mitigations.
- Remaining effect.

- Attack variant ruled out.
- Configuration sensitivity.

“Not exploitable” without scope is usually false confidence.

7.9.9 Exploitability Worksheet

Use this worksheet for every serious invariant violation.

Invariant:

Violation:

Smallest counterexample:

Required initial state:

Required attacker capabilities:

Required victim or keeper behavior:

Required external dependency behavior:

Capital needed:

Time/order needed:

Liquidity/depth needed:

Gas/compute/storage constraints:

Profit or impact:

Repeatability:

Detection likelihood:

Mitigations:

Residual risk:

If the counterexample exists but exploitability is unclear, say so. That is not a failure. It is honest security analysis.

7.10 Worked Example: From Protocol Intent to Attack Hypotheses

7.10.1 Protocol Description

Consider a simplified tokenized vault:

Users deposit an ERC-20 asset and receive vault shares.

Users redeem shares for a proportional amount of the vault's assets.

The vault uses `asset.balanceOf(vault)` as total assets.

When `totalShares == 0`, initial deposits mint shares 1:1 with assets.

When `totalShares > 0`:

```
sharesOut = assetsIn * totalShares / asset.balanceOf(vault)
rounded down.
```

The intended security story:

- Shares represent proportional claims on vault assets.
- Depositors should not be diluted unexpectedly.
- Rounding should be small.
- A user who deposits and later redeems should receive their proportional claim.
- Anyone may deposit.
- Anyone may transfer assets directly to the vault, because ERC-20 transfers cannot be prevented in general.

Now write invariants.

7.10.2 Step 1: Asset and Share Conservation

Basic accounting:

```
totalShares == sum(userShares)
```

Backing:

```
asset.balanceOf(vault) >= assetsRedeemableByAllShares
```

But this is not enough. If `assetsRedeemableByAllShares` is computed from the same manipulated `asset.balanceOf(vault)`, this invariant may be true during the attack.

The deeper claim is user fairness under deposit pricing.

7.10.3 Step 2: Deposit Fairness Invariant

For normal deposits:

If user deposits amount `a` through the official deposit function,
and no fee or loss condition applies,
then shares minted must represent a proportional claim on the deposited assets,
with rounding loss bounded by the protocol's documented tolerance.

This is still too vague. Add attack context:

For any attacker-controlled direct donation before a victim deposit,
the attacker must not be able to recover the donation plus a material portion
of the victim deposit solely because the donation changed the share conversion rate.

Now we have a target.

7.10.4 Step 3: Empty-Vault Attack Hypothesis

Hypothesis:

If the vault is empty, an attacker can become the only shareholder with a tiny
deposit, donate assets directly to inflate `asset.balanceOf(vault)`, and cause a
victim deposit to mint zero or too few shares because `sharesOut` rounds down.

Preconditions:

Vault state:

`totalShares == 0` or very small

Attacker capabilities:

Can deposit before victim

Can transfer asset directly to vault

Can redeem shares after victim deposit

Victim behavior:

Victim deposit does not enforce minimum shares out

Victim transaction can be observed or anticipated

Token behavior:

Direct transfers increase `asset.balanceOf(vault)`

Math:

`sharesOut` rounds down

Capital:

Attacker can temporarily supply donation amount comparable to victim deposit

Attack path:

1. Attacker deposits 1 atomic unit.
2. Vault mints 1 share to attacker.

3. Attacker transfers D assets directly to the vault.
4. Victim deposits U assets.
5. $\text{sharesOut} = U * 1 / (1 + D + \text{maybe } U \text{ depending on implementation})$
6. If sharesOut rounds to 0 or too few shares, victim receives little or no claim.
7. Attacker redeems 1 share for most vault assets.

Invariant violation:

Victim's deposit minted shares with rounding loss far above the documented or economically acceptable bound, and the loss was captured by the attacker.

Impact:

The attacker can steal a victim's first deposit or a material fraction of it.

This is not hypothetical as a class of bug. It is the ERC-4626 inflation attack pattern discussed in the OpenZeppelin documentation.¹³

7.10.5 Step 4: Exploitability Evaluation

Capital:

The attacker needs enough assets to donate so the victim's share output rounds to the target level. The donation may be mostly recoverable if the attacker is the dominant shareholder.

Timing:

The attacker benefits from front-running or otherwise preceding the victim's deposit while the vault has low share supply.

Liquidity:

The attack uses the vault asset itself, not necessarily external market liquidity. If capital can be flash-borrowed and the deposit/redeem sequence fits in one transaction or bundle, capital constraints may be lower.

Reliability:

If the vault is empty and deposits lack minimum-share protection, the attack is highly reliable. If the vault already has large share supply, the attack may be uneconomic.

Repeatability:

Usually strongest at initialization or after a vault is emptied. It may repeat across newly deployed vaults, markets, or assets.

¹³OpenZeppelin Contracts documentation, "ERC4626", security concern: inflation attack, <https://docs.openzeppelin.com/contracts/5.x/erc4626>.

Severity:

High if users can make unprotected first deposits and assets are valuable.

Lower if deployment process seeds the vault safely, users deposit through a router with slippage protection, or virtual offsets make the attack uneconomic.

7.10.6 Step 5: Mitigation Invariants

Virtual shares/assets:

The initial exchange rate must be anchored so that a direct donation cannot make victim rounding loss exceed the documented bound at economically rational capital levels.

Minimum shares out:

A deposit must revert if `sharesOut < minSharesOut` supplied by the depositor or caller.

Seed liquidity:

Before public deposits are enabled, the vault must have initial liquidity and share supply sufficient to bound rounding loss for expected deposit sizes.

Internal accounting:

Direct token transfers must not affect deposit pricing unless the protocol explicitly treats donations as yield and has bounded donation-capture risk.

Monitoring:

Alert if a vault with low `totalShares` receives a direct asset transfer before or near a public deposit.

Residual risk:

Even with mitigations, integrators using preview functions as hard price oracles may be exposed if they ignore slippage, rounding, or manipulable on-chain state.

This is the full workflow:

Intent -> invariant -> counterexample -> exploitability -> mitigation invariant -> tests and monitoring

7.11 Practical Discipline: Building an Invariant Catalogue

For every protocol, maintain an invariant catalogue. It should be short enough to use and precise enough to guide review.

Template:

Invariant ID:

Name:

Type:

state / transition / temporal / economic / authorization / cross-domain

Statement:

Scope:

contracts, assets, users, domains, roles

Assumptions:

oracle, token behavior, finality, governance, operators, liquidity

Allowed exceptions:

Impact if violated:

Likely attack paths:

Tests:

unit:

fuzz/invariant:

scenario:

fork/simulation:

formal/spec:

Monitoring:

Residual risk:

Example:

Invariant ID:

VAULT-ROUNDING-001

Name:

Deposit rounding loss is bounded

Type:

transition / economic

Statement:

For any successful deposit of assets a , shares minted to receiver must be at least the minimum output specified by the caller and must not create

attacker-capturable rounding loss above the configured tolerance.

Scope:

All vault deposit and mint paths, including router and direct calls.

Assumptions:

Asset token does not reenter during transfer.

Asset decimals are configured correctly.

Direct donations are either ignored by internal accounting or made uneconomic by virtual offset.

Allowed exceptions:

Documented deposit fee.

Revert if minSharesOut is not met.

Impact if violated:

User deposit can be partially or fully captured by existing shareholders.

Likely attack paths:

First-deposit donation.

Low-supply rounding manipulation.

Fee-on-transfer mismatch.

Tests:

Unit test empty vault first deposit.

Stateful fuzz deposits, donations, redeems, low share supply.

Fork test against supported assets with real decimals.

Monitoring:

Alert on direct transfers into low-supply vaults.

This kind of catalogue becomes the bridge to later chapters on testing, formal verification, and review workflow.

7.12 Review Questions

1. Why is “users should not lose funds” not a useful invariant?
2. What is the difference between an invariant over all states and an invariant over reachable states?
3. Give one example of a state invariant, a transition invariant, a temporal property, and an economic assumption.
4. Why can a true implementation invariant still fail to capture protocol intent?
5. What assumptions would you attach to a lending collateralization invariant?
6. Why is `totalSupply == sum(balances)` insufficient for a wrapped asset?

7. What must be specified before claiming that a bug is “not exploitable”?
8. How do capital, timing, liquidity, ordering control, and reliability change exploit severity?

7.13 Applied Exercises

7.13.1 Exercise 1: Write Invariants from Intent

Protocol intent:

Users deposit Token A into a vault and receive shares.

The vault deploys Token A into a strategy.

Users can request withdrawal.

Withdrawals are processed once per epoch.

The strategist can move funds between approved strategies.

Governance can add strategies after a 2-day timelock.

Write:

- three conservation invariants;
- two authorization invariants;
- two temporal or withdrawal properties;
- one recovery invariant for strategy failure.

Expected output: an invariant catalogue with assumptions for each invariant.

7.13.2 Exercise 2: Find the Missing Assumption

Invariant:

Borrowing is allowed only if `debtValue <= collateralValue * collateralFactor`.

List at least six assumptions hidden inside this invariant. Consider oracle source, freshness, decimal scaling, liquidity, interest accrual, token transfer behavior, governance parameter changes, sequencer availability, and liquidation liveness.

Expected output: an assumption table with a failure mode for each assumption.

7.13.3 Exercise 3: Exploitability Worksheet

Threat:

An attacker can manipulate an AMM spot price used by a lending protocol oracle.

Fill out:

- broken invariant;
- preconditions;
- required capital;
- required timing;
- liquidity assumptions;
- attack path;
- profit calculation inputs;
- reliability;
- mitigations;
- residual risk.

Expected output: an exploitability worksheet and a conclusion: feasible, infeasible, or uncertain.

7.13.4 Exercise 4: Counterexample to Finding

Counterexample:

1. Vault is empty.
2. Attacker deposits 1 unit and receives 1 share.
3. Attacker donates 100,000 units directly to the vault.
4. Victim deposits 50,000 units and receives 0 shares.
5. Attacker redeems 1 share for almost all assets.

Write:

- root cause;
- broken invariant;
- preconditions;
- impact;
- exploitability assessment;
- recommended mitigation;
- tests that should be added;
- residual risk after mitigation.

Expected output: a concise finding draft.

7.14 Key Takeaways

- An invariant is a property that should hold over reachable states, transitions, or histories.
- Many serious blockchain exploits are valid transaction sequences that violate intended invariants.
- Useful invariants are specific, falsifiable, assumption-aware, measurable, and linked to impact.

- Conservation invariants ask whether claims exceed backing.
- Authorization invariants ask who can cause protected state changes, through which paths, under what conditions.
- Solvency invariants must include recoverability, liabilities, reserves, prices, and failure modes.
- Share-price invariants are vulnerable to rounding, direct donations, first-deposit conditions, and integration assumptions.
- Temporal and liveness properties cover withdrawals, queues, exits, pauses, and recovery.
- Exploitability analysis turns a counterexample into a realistic attack assessment.
- “Not exploitable” is meaningful only when scoped to concrete preconditions, mitigations, deployment facts, and attacker capabilities.
- Tools can test or prove properties, but humans must choose the right properties.

7.15 Part I Handoff

Part I ends with an Integrated Lab rather than a separate capstone. The lab asks the reader to combine the first seven chapters into one security-model review package.

The required artifact is:

Security Model Review Package:

```
state-machine diagram
transition table
asset inventory
authority map
trust-boundary map
transaction analysis
authorization-flow map
threat model
invariant catalogue
exploitability memo
```

Use `part-01-integrated-lab.tex` as the working file for that review.

Part II then applies the same method below the application layer. Consensus, fork choice, finality, peer-to-peer networks, block production, MEV, incentives, clients, forks, upgrades, and chain incidents determine which histories become real, which transactions can be included or censored, which assumptions applications inherit, and which failures no smart contract can fix locally.

7.16 Further Reading

- Leslie Lamport, “Computation and State Machines”. Foundation for state-machine and invariant reasoning.

- Bowen Alpern and Fred B. Schneider, “Defining Liveness”. Classical background on safety and liveness properties.
- Foundry, “Invariant Testing”. Practical invariant testing guidance for Solidity projects.
- Trail of Bits, “Testing a Property with Echidna” and “How to Select the Most Suitable Testing Mode”. Property-based and stateful smart-contract fuzzing guidance.
- Solidity documentation, “SMTChecker and Formal Verification”. Official Solidity guidance on assertions, state properties, counterexamples, and specification limits.
- Certora, “Invariants”. Practical explanation of invariant preservation, induction, and pitfalls in formal specifications.
- Ethereum EIP-4626, “Tokenized Vaults”. Standardized vault share/accounting interface and rounding requirements.
- OpenZeppelin, “ERC4626”. Includes an explanation of the ERC-4626 inflation attack and virtual-offset defense.
- Uniswap, “How Uniswap Works”. Useful reference for AMM reserve and constant-product invariant intuition.
- Aave, “Health Factor & Liquidations”. Practical description of lending health factor and liquidation conditions.
- Compound v2, “Comptroller”. Versioned reference for risk management, collateral factors, liquidity, and shortfall.

Part I Integrated Lab: Security Model Review

Purpose

This Integrated Lab replaces a separate capstone. It is the practical checkpoint for Part I. The goal is to combine the first seven chapters into one professional security-model review package.

Part I taught the reusable method:

```
model the state machine
map assets and authority
inspect transactions and authorization
identify commitments, proofs, and trust boundaries
write a threat model
write invariants
reason about exploitability
```

This lab asks you to perform that method on one small protocol.

Scenario

You are reviewing a toy yield vault called EpochVault.

Users deposit USDC into the vault.

The vault mints shares.

A strategist can allocate USDC into approved strategies.

An oracle reports the strategy value once per epoch.

Users request withdrawals into a queue.

Withdrawals are processed at the next epoch boundary.

Governance can add strategies after a 48-hour timelock.

A guardian can pause deposits immediately.

A separate emergency role can pause withdrawals.

The vault is upgradeable through a proxy admin controlled by governance.

Assume the protocol documentation claims:

Users own vault shares backed by USDC and approved strategies. Governance protects users through timelocked upgrades. The oracle reports fair strategy value. The guardian can pause during emergencies.

Your job is to replace that product description with a security model.

Required Output

Produce one document titled:

`Security Model Review Package: EpochVault`

It must include the sections below.

1. State-Machine Model

Include:

- state variables;
- accepted inputs;
- transition table;
- rejection conditions;
- forbidden states;
- at least three safety properties;
- at least two liveness properties.

2. Asset Inventory

Identify assets that are not merely token balances.

At minimum include:

- USDC deposits;
- vault shares;
- strategy positions;
- queued withdrawal claims;
- oracle-reported value;
- governance power;
- guardian and emergency roles;
- upgrade authority.

3. Authority Map

Create a table with:

asset or state
transition
required authority
authority holder
delay or timelock
observability
worst-case failure
recovery path

Then answer:

Which authority is economically equivalent to control over the vault?

Which authority can break liveness without directly stealing funds?

Which authority can change the future transition rules?

4. Trust-Boundary Map

Draw or describe the trust boundaries among:

- users and wallets;
- frontend or transaction builder;
- vault contract;
- USDC token contract;
- strategy contracts;
- oracle or reporter;
- governance and timelock;
- guardian and emergency role;
- proxy admin or upgrade path.

For each boundary, state what is trusted, who can lie or fail, who verifies, and what happens if the assumption is false.

5. Transaction and Authorization Review

Choose three transitions:

- deposit;
- request withdrawal;

- governance strategy addition or vault upgrade.

For each transition, identify:

- caller or signer;
- authorization rule;
- nonce or replay protection if relevant;
- state read;
- state written;
- external calls;
- events or logs;
- valid-but-unsafe outcome.

6. Threat Model

Produce a one-page threat model with:

- protocol goal;
- actors;
- assets;
- critical transitions;
- privileged roles;
- external dependencies;
- top safety assumptions;
- top liveness assumptions;
- top five attack hypotheses;
- review priorities.

7. Invariant Catalogue

Write at least eight invariants:

- two conservation invariants;
- two authorization invariants;
- two share/accounting invariants;
- one temporal or withdrawal invariant;
- one recovery or pause invariant.

Each invariant must state its assumptions and what evidence would falsify it.

8. Exploitability Memo

Pick one attack hypothesis and write an exploitability memo with:

- broken invariant;
- preconditions;
- required capital;
- required timing;
- liquidity assumptions;
- ordering assumptions;
- expected profit or damage;
- reliability;
- mitigations;
- residual risk.

Grading Criteria

The submission is acceptable if it:

- identifies state and transitions before naming bug classes;
- distinguishes ownership, custody, authority, and economic exposure;
- names trust assumptions instead of hiding them behind words like “trusted” or “decentralized”;
- writes invariants that are specific, falsifiable, and linked to impact;
- explains exploitability using capital, timing, liquidity, ordering, reliability, and profit;
- produces artifacts another reviewer could use.